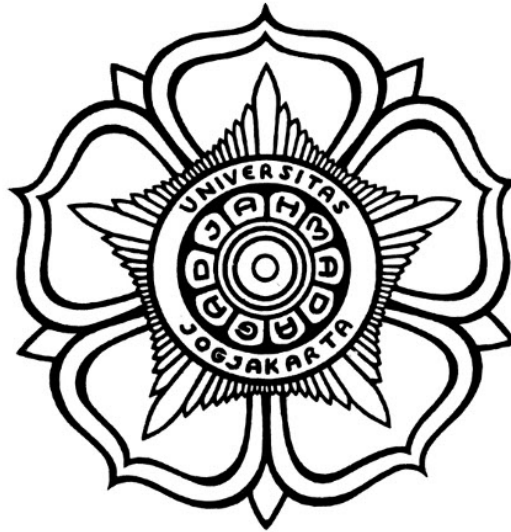


**SISTEM PEMANTAUAN KEPADATAN PENGUNJUNG BERBASIS
COMPUTER VISION MENGGUNAKAN MODEL YOLO DI
KAWASAN WISATA MALIOBORO**

SKRIPSI



***THE SUSTAINABLE DEVELOPMENT GOALS
Industry, Innovation and Infrastructure***

Disusun oleh:

RADEN ARYO BISMO NUGROHO

22/494473/TK/54233

**PROGRAM SARJANA PROGRAM STUDI TEKNOLOGI INFORMASI
DEPARTEMEN TEKNIK ELEKTRO DAN TEKNOLOGI INFORMASI
FAKULTAS TEKNIK UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026**

HALAMAN PENGESAHAN

SISTEM PEMANTAUAN KEPADATAN PENGUNJUNG BERBASIS *COMPUTER VISION* MENGGUNAKAN MODEL YOLO DI KAWASAN WISATA MALIOBORO

SKRIPSI

Diajukan Sebagai Salah Satu Syarat untuk Memperoleh
Gelar Sarjana Teknik
pada Departemen Teknik Elektro dan Teknologi Informasi
Fakultas Teknik
Universitas Gadjah Mada

Disusun oleh:

RADEN ARYO BISMO NUGROHO
22/494473/TK/54233

Telah disetujui dan disahkan

Pada tanggal

Dosen Pembimbing I

Dosen Pembimbing II

Dr. Ir. Rudy Hartanto, M.T., IPM.
NIP. 196403151990031003

Ir. Azkario Rizky Pratama, S.T., M.Eng., Ph.D., IPM.
NIP. 111199102201607102

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini :

Nama : Raden Aryo Bismo Nugroho
NIM : 22/494473/TK/54233
Tahun terdaftar : 2022
Program : Sarjana
Program Studi : Teknologi Informasi
Fakultas : Teknik Universitas Gadjah Mada

Menyatakan bahwa dalam dokumen ilmiah Skripsi ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan sumbernya secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Skripsi ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, tanggal-bulan-tahun

Materai Rp10.000

(Tanda tangan)

Raden Aryo Bismo Nugroho
22/494473/TK/54233

HALAMAN PERSEMBAHAN

Tuliskan kepada siapa skripsi ini dipersembahkan!

contoh

KATA PENGANTAR

Contoh: Puji syukur ke hadirat Allah SWT atas limpahan rahmat, karunia, serta petunjuk-Nya sehingga tugas akhir berupa penyusunan skripsi ini telah terselesaikan dengan baik. Dalam hal penyusunan tugas akhir ini penulis telah banyak mendapatkan arahan, bantuan, serta dukungan dari berbagai pihak. Oleh karena itu pada kesempatan ini penulis mengucapkan terima kasih kepada:

1. Bapak Prof. Ir. Hanung Adi Nugroho, S.T., M.Eng., Ph.D., IPM., SMIEEE. selaku Ketua Departemen Teknik Elektro dan Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada.
2. Bapak Dr. Iswandi, S.T., M.Eng./Dr. Ahmad Nasikun, S.T., M.Sc./Syukron Abu Ishaq Alfarozi, S.T., Ph.D. (**pilih yang sesuai**) selaku Ketua Program Studi Teknik Elektro/Teknologi Informasi/Teknik Biomedis (**pilih yang sesuai**).
3. Bapak selaku dosen pembimbing pertama
4. Ibu selaku dosen pembimbing kedua
5. <isi dengan nama orang lainnya>

Akhir kata penulis berharap semoga skripsi ini dapat memberikan manfaat bagi kita semua, aamiin.

Yogyakarta, <tanggal harus sebelum tanggal pendadaran>

Penulis

DAFTAR ISI

HALAMAN PENGESAHAN	ii
PERNYATAAN BEBAS PLAGIASI	iii
HALAMAN PERSEMBAHAN	iv
KATA PENGANTAR	v
DAFTAR ISI	vi
DAFTAR TABEL	ix
DAFTAR GAMBAR	x
DAFTAR SINGKATAN.....	xi
INTISARI.....	xii
ABSTRACT	xiii
BAB I Pendahuluan	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan Penelitian.....	3
1.4 Batasan Penelitian	3
1.5 Manfaat Penelitian	4
1.6 Sistematika Penulisan.....	5
BAB II Tinjauan Pustaka dan Dasar Teori	6
2.1 Tinjauan Pustaka	6
2.2 Dasar Teori	8
2.2.1 Deteksi Objek	8
2.2.2 Arsitektur YOLO.....	9
2.2.3 Metrik Evaluasi Deteksi Objek.....	11
2.2.4 Ketidakseimbangan Kelas pada Deteksi Objek.....	14
2.2.5 Anotasi Semi-Otomatis	14
2.2.6 Tingkat Pelayanan Pejalan Kaki	15
2.2.7 Arsitektur Aplikasi Web Pemantauan	16
2.3 Analisis Perbandingan Metode	16
2.3.1 Perbandingan Pendekatan Deteksi Objek	17
2.3.2 Pemilihan Varian YOLO untuk Penelitian	18
BAB III Metode Penelitian.....	20
3.1 Gambaran Umum Penelitian	20
3.2 Alat dan Bahan Tugas Akhir	20
3.2.1 Perangkat Keras	21
3.2.2 Perangkat Lunak	22
3.3 Pengumpulan Data	22

3.3.1	Sumber dan Mekanisme Pengambilan <i>Frame</i>	22
3.3.2	Penjadwalan dan Variasi Kondisi.....	23
3.4	Anotasi Data.....	24
3.4.1	Strategi Anotasi Semi-Otomatis	24
3.4.2	Perbandingan Pendekatan Pelabelan Otomatis	25
3.4.3	Anotasi Manual dan Koreksi	26
3.5	Pembersihan dan Pembagian Dataset	27
3.5.1	Pembagian <i>Train</i> , <i>Validation</i> , dan <i>Test</i>	27
3.5.2	Pembersihan Geometris Anotasi	27
3.5.3	Distribusi dan Ketidakseimbangan Kelas	28
3.6	Pelatihan dan Perbandingan Model	28
3.6.1	Konfigurasi Pelatihan	29
3.6.2	Prapemrosesan dan Augmentasi.....	30
3.6.3	Skema Pelatihan dan Perbandingan	31
3.7	Evaluasi Model.....	31
3.7.1	Metrik Evaluasi	32
3.7.2	Desain Evaluasi dan Pemilihan Model	32
3.8	Penyetelan Hyperparameter Model Terpilih	33
3.9	Perancangan dan Implementasi Sistem	34
3.9.1	Arsitektur Sistem.....	34
3.9.2	Penanganan <i>Stream</i> CCTV dan Penjadwalan Inferensi	34
3.9.3	Estimasi Kepadatan dan Ambang Notifikasi.....	36
3.9.4	Visualisasi Kepadatan pada Dashboard	40
BAB IV Hasil dan Pembahasan.....		42
4.1	Hasil Pembentukan Dataset	42
4.2	Hasil Pelatihan dan Perbandingan Delapan Varian	42
4.2.1	Perbandingan Akurasi	44
4.2.2	Perbandingan Efisiensi Pelatihan dan Kecepatan Inferensi	44
4.2.3	Pemilihan Model Terbaik	45
4.3	Analisis Kurva Evaluasi Model Terpilih	46
4.4	Analisis Performa per Kelas dan Pengaruh Ketidakseimbangan	48
4.5	Hasil Penyetelan Hyperparameter Model Terpilih	50
4.6	Analisis Performa per Kelas Setelah Penyetelan	51
4.7	Hasil Implementasi dan Pengujian Sistem.....	52
4.7.1	Kecepatan Inferensi pada Perangkat Penerapan	52
4.7.2	Hasil Antarmuka Pemantauan	54
4.7.3	Hasil Grafik Kepadatan dan Riwayat Peringatan	54
4.8	Sintesis Temuan	56
BAB V Kesimpulan dan Saran.....		58

5.1	Kesimpulan.....	58
5.2	Saran.....	59
DAFTAR PUSTAKA.....		62
LAMPIRAN		L-1
L.1	Isi Lampiran.....	L-1
L.2	Panduan Latex.....	L-2
L.2.1	Syntax Dasar	L-2
L.2.1.1	Penggunaan Sitasi	L-2
L.2.1.2	Penulisan Gambar	L-2
L.2.1.3	Penulisan Tabel	L-2
L.2.1.4	Penulisan formula.....	L-2
L.2.1.5	Contoh list.....	L-3
L.2.2	Blok Beda Halaman.....	L-3
L.2.2.1	Membuat algoritma terpisah	L-3
L.2.2.2	Membuat tabel terpisah.....	L-3
L.2.2.3	Menulis formula terpisah halaman.....	L-4
L.3	Format Penulisan Referensi	L-6
L.3.1	Book	L-6
L.3.2	Handbook.....	L-8
L.4	Contoh Source Code	L-10
L.4.1	Sample algorithm	L-10
L.4.2	Sample Python code	L-11
L.4.3	Sample Matlab code	L-12

DAFTAR TABEL

Tabel 2.1	Perbandingan penelitian terdahulu.	7
Tabel 2.2	Kategori hasil prediksi pada evaluasi deteksi objek.	12
Tabel 2.3	Tingkatan standar pelayanan jalur pejalan kaki menurut Permen PUPR Nomor 03/PRT/M/2014 [1].	15
Tabel 2.4	Perbandingan pendekatan deteksi objek terhadap kebutuhan sistem pemantauan.	18
Tabel 3.1	Versi perangkat lunak utama yang digunakan pada lingkungan pelatihan.	22
Tabel 3.2	Slot waktu pengambilan citra dalam satu hari.	24
Tabel 3.3	Jumlah <i>instance</i> hasil tiga kombinasi pelabelan otomatis pada 1.252 citra latih.	25
Tabel 3.4	Filter pembersihan kotak pembatas.	28
Tabel 3.5	Jumlah <i>instance</i> sebelum dan sesudah pembersihan pada tiap himpunan.	28
Tabel 3.6	Distribusi jumlah <i>instance</i> tiap kelas setelah pembersihan.	29
Tabel 3.7	Konfigurasi <i>hyperparameter</i> pelatihan.	30
Tabel 4.1	Perbandingan akurasi delapan varian model pada himpunan uji.	44
Tabel 4.2	Perbandingan efisiensi delapan varian model. Kecepatan diukur pada GPU Tesla V100.	45
Tabel 4.3	Performa per kelas model terpilih yolo11l pada himpunan uji, disandingkan jumlah <i>instance</i> latih.	49
Tabel 4.4	Perbandingan performa yolo11l pada resolusi 640 dan 960 (himpunan uji).	50
Tabel 4.5	Perbandingan F1 per kelas yolo11l pada resolusi 640 dan 960 (himpunan uji).	52
Tabel 4.6	Kecepatan inferensi delapan varian model dan model hasil penyetelan pada GPU GTX 1650 (rata-rata 100 inferensi).	53
Tabel L.1	Tabel ini	L-2
Tabel L.2	Contoh tabel panjang	L-4

DAFTAR GAMBAR

Gambar 2.1	Skema sederhana arsitektur YOLOv8 dengan alur <i>backbone-neck-head</i> , disusun dari uraian pada [2].	9
Gambar 2.2	Skema sederhana arsitektur YOLO11, dengan kerangka yang sama seperti YOLOv8 dan tambahan blok C3k2 serta modul perhatian C2PSA pada <i>backbone</i> , disusun dari uraian pada [3]....	10
Gambar 2.3	Ilustrasi <i>Intersection over Union</i> : luas irisan (area abu-abu) dibagi luas gabungan kedua kotak pembatas.....	11
Gambar 2.4	Kurva <i>precision-recall</i> : <i>Average Precision</i> (AP) adalah luas area di bawah kurva (area arsir).....	13
Gambar 3.5	Alur tahapan penelitian.	21
Gambar 3.6	Contoh kesalahan deteksi tiga bangku taman yang terdeteksi sebagai objek.	26
Gambar 3.7	Arsitektur sistem pemantauan yang menghubungkan <i>stream</i> CCTV, <i>backend</i> FastAPI, basis data SQLite, dan <i>frontend</i> React..	34
Gambar 3.8	Hasil pengujian kecepatan jaringan dengan <i>Speedtest by Ookla</i> saat pengukuran.	36
Gambar 3.9	Kamera CCTV yang sama pada dua sudut pandang berbeda	37
Gambar 3.10	Citra CCTV area pejalan kaki di seberang Gedung Agung.	38
Gambar 3.11	Citra CCTV area pejalan kaki Titik Nol yang menghadap ke utara.	38
Gambar 3.12	Pengukuran luas area pejalan kaki yang tercakup CCTV seberang Gedung Agung pada <i>Google Earth</i>	39
Gambar 3.13	Pengukuran luas area pejalan kaki yang tercakup CCTV Titik Nol pada <i>Google Earth</i>	39
Gambar 4.14	Distribusi jumlah <i>instance</i> tiap kelas (kiri atas) serta sebaran posisi dan ukuran kotak pembatas pada himpunan latih.	43
Gambar 4.15	Perkembangan mAP50-95 pada data validasi sepanjang epoch untuk yolo11l dan yolov8l.	45
Gambar 4.16	Kurva <i>precision-recall</i> model yolo11l untuk seluruh kelas pada himpunan validasi.	47
Gambar 4.17	Kurva <i>F1-score</i> terhadap ambang keyakinan model yolo11l untuk seluruh kelas pada himpunan validasi.	47
Gambar 4.18	<i>Confusion matrix</i> ternormalisasi model yolo11l pada himpunan uji.	48
Gambar 4.19	F1 per kelas pada himpunan uji untuk kedelapan varian model. ..	49
Gambar 4.20	Tampilan <i>dashboard</i> utama dengan deteksi langsung, panel peringatan kamera, dan ringkasan jumlah objek per kelas.	54
Gambar 4.21	Grafik kepadatan pejalan kaki harian (per jam) untuk kamera Selatan Teteg, menampilkan rata-rata dan puncak jumlah orang. .	55
Gambar 4.22	Halaman riwayat peringatan keramaian, menampilkan detail satu peringatan beserta citra CCTV dan kotak deteksinya.	56
Gambar L.1	Contoh gambar.	L-2

DAFTAR SINGKATAN

b	=	bias
$K(x_i, x_j)$	=	fungsi kernel
y	=	kelas keluaran
C	=	parameter untuk mengendalikan besarnya pertukaran antara penalti variabel slack dengan ukuran margin
L_D	=	persamaan Lagrange dual
L_P	=	persamaan Lagrange primal
$\mathbf{w}$	=	vektor bobot
$\mathbf{x}$	=	vektor masukan
ANFIS	=	Adaptive Network Fuzzy Inference System
ANSI	=	American National Standards Institute
DAG	=	Directed Acyclic Graph
DDAG	=	Decision Directed Acyclic Graph
HIS	=	Hue Saturation Intensity
QP	=	Quadratic Programming
RBF	=	Radial Basis Function
RGB	=	Red Green Blue
SV	=	Support Vector
SVM	=	Support Vector Machines

INTISARI

Penelitian ini bertujuan untuk membangun dataset deteksi objek sembilan kelas khas kawasan Malioboro, membandingkan dan mengoptimasi varian model YOLO untuk mendeteksi kelas-kelas tersebut, lalu mengimplementasikan model terpilih ke dalam sistem pemantauan kepadatan pengunjung berbasis web, sebagai jawaban atas keterbatasan pemantauan manual dan ketidaktersediaan dataset publik seperti COCO untuk objek lokal seperti andong, becak, dan bajaj. Metodologi yang digunakan meliputi pembangunan dataset dari rekaman 22 kamera CCTV publik kawasan Malioboro melalui anotasi semi-otomatis berbantuan model *pretrained* dengan koreksi manual, pelatihan dan perbandingan delapan varian model YOLOv8 dan YOLO11 menggunakan metrik mAP50, mAP50-95, dan *F1-score* pada himpunan uji, penyetelan resolusi citra masukan pada model terbaik, serta implementasi model tersebut ke dalam sistem berbasis web yang memproses *stream* video kamera CCTV secara *real-time* dengan ambang notifikasi kepadatan berdasarkan konsep tingkat pelayanan (*level of service*) pejalan kaki. Hasil penelitian menunjukkan dataset akhir memuat 1.581 citra beranotasi dengan distribusi kelas yang sangat tidak seimbang, mencapai rasio 237 berbanding 1 antara kelas terbanyak dan tersedikit. Pada himpunan uji, yolov8l mencatat mAP50 dan rerata F1 tertinggi (0,6175 dan 0,6222), sedangkan yolo11l mencatat mAP50-95 tertinggi (0,4012) dengan ukuran model 42% lebih ringkas sehingga dipilih sebagai model terbaik. Penyetelan resolusi dari 640 menjadi 960 piksel kemudian menaikkan rerata F1 model terpilih dari 0,5962 menjadi 0,6739, melampaui seluruh varian pada resolusi dasar. Pengujian pada perangkat penerapan menunjukkan model terpilih mencapai 9,73 *frame* per detik pada GPU GTX 1650, lebih cepat dibanding yolov8l yang hanya 8,80 *frame* per detik, dengan sistem yang berhasil menampilkan deteksi langsung, grafik kepadatan, dan riwayat peringatan sesuai rancangan. Kesimpulan dari penelitian ini adalah model YOLO11 *large* dengan resolusi masukan yang disetel terbukti menjadi pilihan yang seimbang antara akurasi, ukuran, dan kecepatan untuk diterapkan pada sistem pemantauan kepadatan berbasis web di kawasan Malioboro, dengan ketidakseimbangan kelas pada dataset lokal tetap menjadi faktor yang memengaruhi performa deteksi pada kelas minoritas.

Kata kunci : deteksi objek, YOLO, kepadatan pengunjung, ketidakseimbangan kelas, Malioboro

ABSTRACT

Crowd density monitoring in the Malioboro tourist area, which accommodates millions of visitors each year, still relies on manual observation of CCTV footage, while standard detection datasets such as COCO do not cover vehicles unique to the area, such as andong, becak, and bajaj, leaving a gap that this study addresses by building a dedicated local dataset, selecting a suitable detection model, and deploying it within a working monitoring system. A dataset of 1,581 annotated images covering nine object classes was constructed from footage of 22 public CCTV cameras using semi-automatic annotation assisted by a pretrained model with manual correction, after which eight variants of YOLOv8 and YOLO11 were trained and evaluated on a held-out test set using mAP50, mAP50-95, and F1-score, with the best variant further optimized through input resolution tuning before being integrated into a web-based system that processes live CCTV streams and issues density alerts based on the pedestrian level of service concept. The resulting dataset exhibited a severe class imbalance, with a 237-to-1 ratio between the most and least frequent classes. Among the eight variants, yolov8l achieved the highest mAP50 and mean F1 (0.6175 and 0.6222), while yolo11l achieved the highest mAP50-95 (0.4012) at a model size 42% smaller, leading to its selection despite not leading on every metric. Tuning its input resolution from 640 to 960 pixels then raised its mean F1 from 0.5962 to 0.6739, surpassing every variant at the base resolution. On the GTX 1650 deployment device, the selected model also ran faster than yolov8l (9.73 versus 8.80 frames per second), and the implemented system correctly displayed live detection, density charts, and alert history during testing. These findings indicate that a deliberately balanced choice of architecture, model size, and resolution tuning, rather than the single highest-scoring variant, yields a detection model that is well suited for real-time deployment on resource-constrained hardware, although the underlying class imbalance continues to limit detection performance on minority classes such as bicycles and bajaj.

Keywords : object detection, YOLO, visitor density, class imbalance, Malioboro

BAB I

PENDAHULUAN

1.1 Latar Belakang

Kawasan Malioboro adalah pusat pariwisata Kota Yogyakarta yang menampung pergerakan pengunjung dalam jumlah besar setiap hari. Koridor pejalan kaki sepanjang sekitar 1,3 km menghubungkan Titik Nol Yogyakarta, Benteng Vredenburg, Pasar Beringharjo, hingga Stasiun Tugu, dan menjadi ruang publik utama bagi wisatawan maupun warga lokal [4]. Data Pemerintah Kota Yogyakarta mencatat kunjungan ke Malioboro sepanjang tahun 2024 berkisar antara 118.255 pengunjung pada bulan Maret hingga 604.240 pengunjung pada bulan Mei, dengan akumulasi sekitar 4,7 juta pengunjung dalam satu tahun [5]. Pada skala kota, perjalanan wisatawan nusantara ke Kota Yogyakarta sepanjang tahun 2025 berkisar antara 629.936 hingga 1.034.536 perjalanan per bulan [6]. Konsentrasi pengunjung memuncak pada periode tertentu. Saat arus mudik lebaran, volume pejalan kaki di Malioboro meningkat hampir dua kali lipat dibandingkan hari biasa, dengan puncak pada hari kedua dan ketiga lebaran [7].

Kepadatan setinggi itu menimbulkan tekanan pada ruang dan lalu lintas kawasan. Sebagai pusat bisnis sekaligus destinasi wisata, Malioboro kerap mengalami kemacetan akibat lonjakan penggunaan kendaraan [8]. Pengelolaan lalu lintas di kawasan ini bertumpu pada pengaturan administratif yang mengacu pada Peraturan Daerah Kota Yogyakarta Nomor 1 Tahun 2019 serta pengawasan langsung oleh petugas [8]. Pemantauan kondisi kepadatan umumnya dilakukan secara manual melalui kamera CCTV yang diamati operator. Pendekatan ini memiliki keterbatasan yang melekat, yaitu cakupan pengamatan yang terbatas, kebutuhan daya yang tinggi, dan ketergantungan pada pengawasan operator secara terus-menerus [9]. Akibatnya, informasi kepadatan yang objektif dan tersedia secara langsung untuk banyak titik sekaligus sulit diperoleh.

Sejumlah kota di dunia telah beralih ke sistem pemantauan berbasis kecerdasan buatan untuk mengatasi keterbatasan tersebut. Kota Hangzhou di Tiongkok mengoperasikan sistem *City Brain* sejak tahun 2016, yang memproses data *real-time* dari kamera dan sensor pada lampu lalu lintas untuk mengoptimalkan arus kendaraan [10]. Sistem tersebut dilaporkan meningkatkan kecepatan rata-rata lalu lintas sebesar 15,3 persen dan menurunkan kemacetan pada jam sibuk sebesar 9,2 persen [10]. Pada tingkat teknis, *computer vision* berbasis kamera CCTV telah terbukti mampu memantau kerumunan secara otomatis. Penelitian di sebuah pusat perbelanjaan di Indonesia, misalnya, menggunakan metode YOLOv3-Tiny untuk mendeteksi kerumunan dari rekaman kamera dan memicu peringatan ketika jumlah orang melampaui ambang tertentu [11]. Penggunaan *computer vision* memungkinkan pengurangan ketergantungan

pada pengawasan manusia sekaligus memperluas cakupan pemantauan [9].

Penerapan *computer vision* di kawasan seperti Malioboro menghadapi kendala yang khas. Model deteksi objek pada umumnya dilatih menggunakan dataset publik berskala besar seperti COCO yang tidak memuat kendaraan khas Indonesia terutama di Malioboro, seperti andong, becak, dan bajaj. Model yang dilatih pada dataset semacam COCO maupun UA-DETRAC terbukti kesulitan menggeneralisasi pada kondisi lalu lintas yang padat, heterogen, dan tidak teratur di kota-kota berkembang [12]. Padahal, Malioboro memuat objek yang beragam, mulai dari pejalan kaki, kendaraan bermotor umum, hingga moda transportasi tradisional seperti andong dan becak. Kondisi ini menuntut dataset dan model deteksi yang dirancang khusus untuk karakteristik objek lokal yang bukan sekadar mengandalkan model siap pakai.

Penelitian ini membangun sistem pemantauan kepadatan pengunjung di kawasan Malioboro menggunakan pendekatan *computer vision* dengan model YOLO. Dataset deteksi untuk sembilan kelas objek, yaitu andong, bajaj, becak, sepeda, bus, mobil, motor, orang, dan truk, dibangun dari rekaman kamera CCTV kawasan Malioboro. Beberapa varian model YOLOv8 dan YOLO11 dilatih dan dibandingkan untuk menentukan model dengan performa deteksi terbaik. Model terpilih kemudian diimplementasikan ke dalam sistem berbasis web yang memproses *stream* video kamera CCTV secara *real-time* dan menampilkan jumlah objek pada tiap kategori. Sebagai dasar penetapan ambang notifikasi kepadatan, sistem mengacu pada konsep tingkat pelayanan pejalan kaki (*level of service*) yang diadopsi dalam Peraturan Menteri PUPR Nomor 03/PRT/M/2014 [1, 13]. Berdasarkan uraian tersebut, beberapa permasalahan dirumuskan pada subbab berikut.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, yaitu kebutuhan pemantauan kepadatan otomatis di kawasan Malioboro yang masih dikelola secara manual, terdapat tiga persoalan pokok yang saling berurutan, mulai dari ketersediaan data, pemilihan model deteksi, hingga penerapannya pada sistem. Ketiga persoalan tersebut dirumuskan sebagai berikut.

1. Dataset deteksi objek untuk sembilan kelas khas kawasan Malioboro, yaitu andong, bajaj, becak, sepeda, bus, mobil, motor, orang, dan truk, belum tersedia karena sebagian kelas tersebut tidak terdapat pada dataset publik berskala besar seperti COCO.
2. Varian model YOLOv8 dan YOLO11 yang memberikan performa deteksi terbaik untuk kesembilan kelas tersebut belum diketahui, termasuk pengaruh ketidakseimbangan jumlah *instance* antar kelas terhadap performa deteksi pada tiap kelas.

3. Model deteksi terbaik belum diimplementasikan ke dalam sistem pemantauan kepadatan berbasis web yang memproses *stream* video kamera CCTV secara *real-time*.

1.3 Tujuan Penelitian

Tujuan penelitian ini dirumuskan untuk menjawab setiap permasalahan yang telah dipaparkan, dengan keterkaitan satu-satu antara tiap tujuan dan tiap rumusan masalah. Ketiga tujuan tersusun mengikuti alur penelitian, mulai dari pembentukan dataset sebagai fondasi, dilanjutkan pemilihan dan optimasi model deteksi melalui perbandingan dan penyetelan, hingga penerapan model terpilih ke dalam sistem yang berfungsi. Susunan berjenjang ini memastikan keluaran tiap tahap menjadi masukan bagi tahap berikutnya, sehingga capaian penelitian dapat ditelusuri secara runtut dari ketersediaan data hingga sistem yang beroperasi.

1. Membangun dataset deteksi sembilan kelas objek kawasan Malioboro (andong, bajaj, becak, sepeda, bus, mobil, motor, orang, dan truk) dari rekaman kamera CCTV melalui anotasi semi-otomatis berbantuan model *pretrained* dengan koreksi manual.
2. Membandingkan performa varian model YOLOv8 dan YOLO11 dalam mendeteksi kesembilan kelas menggunakan metrik mAP50, mAP50-95, dan F1, menganalisis pengaruh ketidakseimbangan kelas terhadap performa deteksi tiap kelas, lalu memilih model terbaik dan mengoptimasi performanya melalui penyetelan *hyperparameter*.
3. Mengimplementasikan model deteksi terbaik ke dalam sistem pemantauan kepadatan berbasis web yang memproses *stream* video kamera CCTV secara *real-time* dan mengukur performa inferensinya.

1.4 Batasan Penelitian

Penelitian ini diberi batasan agar lingkupnya tetap terarah dan dapat diselesaikan dengan sumber daya yang tersedia. Batasan mencakup lokasi dan sumber data, kelas objek yang dideteksi, varian model yang dibandingkan, serta fungsi sistem yang dikembangkan. Penetapan batasan ini sekaligus menjaga agar perbandingan model dilakukan pada kondisi yang terkontrol. Ruang lingkup selengkapnya diuraikan sebagai berikut.

1. Lokasi penelitian dibatasi pada kawasan wisata Malioboro, Kota Yogyakarta, dengan sumber data berupa *stream* video dari 22 titik kamera *closed-circuit television* (CCTV) publik di kawasan tersebut.
2. Objek yang dideteksi dibatasi pada sembilan kelas, yaitu andong, bajaj, becak, sepeda, bus, mobil, motor, orang, dan truk.
3. Model deteksi yang dilatih dan dibandingkan dibatasi pada varian YOLOv8 dan

YOLO11.

4. Dataset yang digunakan adalah data primer yang dibangun dari rekaman kamera CCTV kawasan Malioboro melalui anotasi semi-otomatis dengan koreksi manual. Rincian proses anotasi dijelaskan pada Bab III.
5. Sistem hanya melakukan pemantauan (*monitoring*) kepadatan secara *real-time* dan tidak melakukan prediksi kepadatan pada waktu mendatang.
6. Penghitungan objek dilakukan per *frame*. Sistem tidak melakukan pelacakan (*tracking*) objek antar-*frame*.
7. Penetapan ambang notifikasi berbasis tingkat pelayanan (*level of service*) untuk kepadatan orang hanya diterapkan pada dua titik kamera yang dikalibrasi sebagai contoh. Titik kamera lainnya serta penetapan ambang untuk kemacetan kendaraan berada di luar lingkup dan menjadi arah pengembangan lanjutan.
8. Penelitian berfokus pada perbandingan model deteksi dan implementasi fungsional sistem. Perancangan berbasis *User-Centered Design* (UCD) dan pengujian *usability* tidak termasuk dalam lingkup penelitian.

1.5 Manfaat Penelitian

Hasil penelitian ini diharapkan bermanfaat pada dua sisi yang saling melengkapi, yaitu pengembangan keilmuan dan penerapan di lapangan. Sisi akademis berkaitan dengan ketersediaan data dan temuan yang dapat dijadikan rujukan penelitian berikutnya, sedangkan sisi praktis berkaitan dengan kegunaan sistem bagi pengelola kawasan Malioboro. Uraian masing-masing manfaat adalah sebagai berikut.

Manfaat akademis:

1. Penelitian ini menghasilkan dataset deteksi objek untuk sembilan kelas khas kawasan Malioboro, termasuk andong, becak, dan bajaj yang tidak terdapat pada dataset publik seperti COCO, sehingga dapat menjadi sumber data bagi penelitian deteksi objek lokal selanjutnya.
2. Hasil perbandingan performa varian model YOLOv8 dan YOLO11 pada dataset objek lokal dengan distribusi kelas yang bervariasi dapat menjadi rujukan dalam pemilihan model deteksi objek untuk kasus serupa.

Manfaat praktis:

1. Sistem pemantauan yang dibangun dapat membantu pengelola kawasan Malioboro memperoleh informasi kepadatan pengunjung secara *real-time* sebagai dasar pengambilan keputusan pengendalian kerumunan.
2. Pemantauan berbasis *computer vision* memberikan informasi kepadatan yang lebih

objektif dan mencakup lebih banyak titik dibandingkan pengamatan manual oleh operator.

1.6 Sistematika Penulisan

Bab I berisi pendahuluan yang menjelaskan dasar dilakukannya penelitian. Bagian latar belakang memaparkan kondisi kepadatan pengunjung di kawasan Malioboro dan keterbatasan pemantauan manual yang berjalan saat ini, yang menjadi alasan dibangunnya sistem pemantauan otomatis. Dari latar belakang tersebut diturunkan rumusan masalah, tujuan, batasan, dan manfaat penelitian. Bab ini ditutup dengan sistematika penulisan yang menggambarkan susunan keseluruhan naskah.

Bab II menyajikan tinjauan pustaka dan dasar teori yang melandasi penelitian. Tinjauan pustaka membahas penelitian terdahulu yang relevan beserta posisi penelitian ini di antaranya. Dasar teori menguraikan konsep deteksi objek, arsitektur model YOLOv8 dan YOLO11, metrik evaluasi deteksi, ketidakseimbangan kelas, anotasi semi-otomatis, dan tingkat pelayanan (*level of service*) pejalan kaki. Bab ini ditutup dengan analisis perbandingan metode yang menjadi dasar pemilihan model deteksi.

Bab III menguraikan metode penelitian secara bertahap. Pembahasan dimulai dari pengumpulan data berupa *stream* kamera CCTV, dilanjutkan anotasi semi-otomatis dengan koreksi manual dan prapemrosesan data untuk membentuk dataset. Bagian berikutnya menjelaskan konfigurasi pelatihan model serta metode evaluasi yang digunakan untuk membandingkan varian model, serta penyetelan *hyperparameter* pada model terpilih untuk mengoptimasi performanya. Bab ini juga memuat perancangan dan implementasi sistem pemantauan berbasis web yang memproses video secara *real-time*.

Bab IV menyajikan hasil penelitian beserta pembahasannya. Bagian awal memaparkan hasil pembentukan dataset, dilanjutkan hasil pelatihan dan perbandingan varian model YOLOv8 dan YOLO11 berdasarkan metrik yang telah ditetapkan. Analisis performa pada tiap kelas dibahas untuk melihat pengaruh ketidakseimbangan kelas terhadap hasil deteksi. Bagian selanjutnya menyajikan hasil penyetelan *hyperparameter* pada model terpilih beserta dampaknya terhadap akurasi dan kecepatan inferensi. Bagian akhir memaparkan hasil implementasi dan pengujian sistem pemantauan.

Bab V berisi kesimpulan dan saran. Kesimpulan menjawab setiap rumusan masalah berdasarkan temuan yang diperoleh pada Bab IV. Saran memuat arah pengembangan yang dapat dilakukan pada penelitian selanjutnya sesuai dengan lingkup penelitian ini.

BAB II

TINJAUAN PUSTAKA DAN DASAR TEORI

2.1 Tinjauan Pustaka

Pemantauan kerumunan berbasis deteksi objek menggunakan model YOLO telah menjadi topik penelitian yang aktif dalam beberapa tahun terakhir. Sebagian penelitian berfokus pada penghitungan jumlah orang secara *real-time* dari rekaman kamera. Gündüz dan Işık [14] mengusulkan metode deteksi kerumunan *real-time* yang menentukan batas suatu wilayah pada video, menghitung jumlah orang di dalamnya, dan membandingkan performa beberapa varian model YOLO untuk keperluan tersebut. Pendekatan dengan tujuan serupa diterapkan Ginting dkk. [11] yang menggabungkan YOLOv3-Tiny dan algoritma Viola-Jones untuk mendeteksi kerumunan di pusat perbelanjaan serta memicu peringatan ketika jumlah orang melampaui ambang tertentu. Pada penerapan di perangkat dengan sumber daya terbatas, Hansika dkk. [15] membangun kerangka penghitungan kerumunan ringan menggunakan YOLOv8n dan algoritma pelacakan Norfair yang dijalankan pada *edge device* Raspberry Pi 5, dengan hasil ditampilkan melalui *dashboard*.

Sebagian penelitian lain menekankan keterpaduan deteksi dengan antarmuka pemantauan dan sistem peringatan. Apeksha dkk. [16] mengembangkan platform AI-Transit Alerts yang menghitung jumlah penumpang transportasi umum secara *real-time* menggunakan YOLOv8, menampilkannya pada *dashboard* interaktif, dan mengirim peringatan otomatis ketika jumlah penumpang melampaui batas aman. Wong dan Law [17] menyoroti keterbatasan pemantauan CCTV manual yang melelahkan dan rawan kesalahan pada ruang publik padat, lalu mengusulkan kerangka yang menggabungkan rekaman CCTV dengan informasi denah ruang untuk memetakan lintasan individu dan memantau kepadatan secara *real-time*.

Di luar deteksi orang, model YOLO juga digunakan untuk deteksi kendaraan dalam pemantauan lalu lintas. Zhang dkk. [18] mengembangkan model YOLOv5 yang ditingkatkan untuk deteksi kendaraan *real-time* dengan fokus menurunkan tingkat deteksi keliru (*false detection*). Ringkasan perbandingan penelitian terdahulu disajikan pada Tabel 2.1.

Penelitian-penelitian tersebut menunjukkan model YOLO efektif untuk pemantauan kerumunan maupun kendaraan secara *real-time*. Meskipun demikian, sebagian besar berfokus pada satu jenis objek saja, yaitu orang atau kendaraan, dan mengandalkan kelas objek yang tersedia pada dataset umum. Belum ada penelitian yang menangani kombinasi objek khas kawasan wisata Indonesia, termasuk andong, becak, dan bajaj, dalam satu sistem. Perbandingan antar varian model pada sebagian besar penelitian juga terbatas pada satu generasi YOLO. Penelitian ini menempatkan

Tabel 2.1. Perbandingan penelitian terdahulu.

Penelitian	Model	Domain Aplikasi	Catatan
Gündüz & Işık [14]	Beberapa varian YOLO	Penghitungan kerumunan dalam ruang	Membandingkan varian YOLO dan menentukan wilayah hitung
Ginting dkk. [11]	YOLOv3-Tiny, Viola-Jones	Deteksi kerumunan di pusat perbelanjaan	Memicu peringatan saat ambang terlampaui
Hansika dkk. [15]	YOLOv8n, Norfair	Penghitungan kerumunan pada <i>edge device</i>	Ringan, hemat sumber daya, dengan <i>dashboard</i>
Apeksha dkk. [16]	YOLOv8	Penghitungan penumpang transportasi umum	<i>Dashboard</i> interaktif dan peringatan otomatis
Wong & Law [17]	Model <i>deep learning</i>	Pemantauan kepadatan ruang publik	Menggabungkan CCTV dengan denah ruang
Zhang dkk. [18]	YOLOv5 (ditingkatkan)	Deteksi kendaraan lalu lintas	Menurunkan tingkat deteksi keliru
Penelitian ini	YOLOv8 & YOLO11 (n, s, m, l)	Deteksi sembilan kelas objek di Malioboro	Multi-kelas mencakup andong, becak, dan bajaj. Membandingkan dua generasi dan diimplementasikan sebagai sistem berbasis web

diri pada celah tersebut dengan membandingkan varian YOLOv8 dan YOLO11 untuk mendeteksi sembilan kelas objek khas kawasan Malioboro secara bersamaan, kemudian mengimplementasikan model terbaik ke dalam sistem pemantauan berbasis web.

Sintesis dari keenam penelitian tersebut memperlihatkan tiga celah yang ditangani penelitian ini. Pertama, belum ada yang membangun satu dataset deteksi yang memuat objek transportasi tradisional Indonesia seperti andong, becak, dan bajaj bersama objek umum dalam satu himpunan kelas. Kedua, perbandingan langsung antara dua generasi YOLO yang setara secara implementasi masih jarang dilakukan pada kondisi data lokal yang timpang. Ketiga, sebagian besar penelitian berhenti pada tahap deteksi atau penghitungan, sedangkan penelitian ini melanjutkannya hingga penerjemahan jumlah

deteksi menjadi tingkat kepadatan berbasis standar nasional dan implementasi sistem yang berjalan. Ketiga celah inilah yang menjadi dasar rumusan masalah pada Bab I.

2.2 Dasar Teori

Subbab ini menyusun landasan teori mengikuti alur kerja sistem yang dibangun, mulai dari teori deteksi objek hingga penerjemahan hasil deteksi menjadi penilaian kepadatan. Teori deteksi objek dan arsitektur YOLO menjelaskan cara objek dikenali dari citra kamera pengawas, sedangkan metrik evaluasi menetapkan ukuran kualitas deteksi yang dihasilkan. Ketidakeimbangan kelas dan anotasi semi-otomatis membahas dua persoalan data yang menentukan keberhasilan pelatihan model pada konteks kawasan Malioboro. Bagian terakhir, yaitu tingkat pelayanan pejalan kaki, menghubungkan keluaran model dengan tujuan akhir sistem berupa estimasi kepadatan dan penetapan ambang notifikasi.

2.2.1 Deteksi Objek

Deteksi objek adalah tugas *computer vision* yang bertujuan mengidentifikasi lokasi dan kategori setiap objek yang ada dalam suatu gambar secara bersamaan [19]. Keluaran dari sistem deteksi objek berupa kotak pembatas (*bounding box*) yang melingkupi setiap objek beserta label kelas dan nilai keyakinan (*confidence score*) yang menyertainya. Gabungan informasi lokasi dan kelas inilah yang membedakan deteksi objek dari klasifikasi citra biasa, karena sistem tidak hanya menentukan ada atau tidaknya suatu objek tetapi juga menetapkan posisinya pada citra.

Pendekatan deteksi objek berbasis *deep learning* secara umum dibagi menjadi dua kategori, yaitu *two-stage detector* dan *one-stage detector*. *Two-stage detector* bekerja dalam dua tahap terpisah: tahap pertama menghasilkan kandidat wilayah yang mungkin mengandung objek, dan tahap kedua mengklasifikasikan serta menyempurnakan posisi kotak pembatas pada setiap kandidat wilayah tersebut. Faster R-CNN [20] adalah salah satu arsitektur *two-stage* yang berpengaruh, yang memperkenalkan *Region Proposal Network* (RPN) untuk menghasilkan proposal wilayah secara efisien dengan berbagi fitur konvolusional antara tahap proposal dan tahap deteksi. Pendekatan ini menghasilkan akurasi yang tinggi, namun kecepatan inferensinya terbatas karena sifat dua tahapnya, yaitu sekitar 5 *frame* per detik pada GPU.

One-stage detector menggabungkan kedua tahap tersebut menjadi satu jaringan tunggal yang langsung memprediksi kotak pembatas dan label kelas dari seluruh gambar dalam satu kali evaluasi [21]. Pendekatan ini jauh lebih cepat dibandingkan *two-stage detector* dan lebih sesuai untuk kebutuhan deteksi *real-time*, meskipun pada generasi awal memiliki akurasi yang sedikit lebih rendah. Penelitian ini menggunakan pendekatan *one-stage* karena kebutuhan pemantauan kepadatan secara *real-time* dari *stream* video

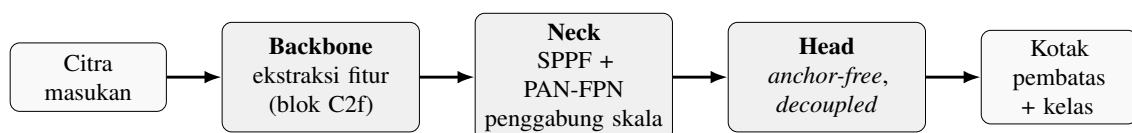
kamera CCTV.

Komponen penting dalam arsitektur detektor modern adalah *Feature Pyramid Network* (FPN) [22]. FPN membangun representasi piramida fitur multi-skala dari jaringan konvolusional dengan biaya komputasi yang minimal, menggunakan arsitektur *top-down* dengan koneksi lateral untuk menghasilkan peta fitur berkualitas tinggi di semua skala. Kemampuan mendeteksi objek dalam berbagai ukuran ini menjadikan FPN komponen penting dalam detektor YOLO generasi terbaru, termasuk YOLOv8 dan YOLO11 yang digunakan dalam penelitian ini.

2.2.2 Arsitektur YOLO

YOLO (*You Only Look Once*) diperkenalkan oleh Redmon dkk. [21] sebagai pendekatan deteksi objek *one-stage* yang memformulasikan deteksi sebagai masalah regresi tunggal dari piksel gambar ke koordinat kotak pembatas dan probabilitas kelas. Model dasar YOLO mampu memproses gambar pada kecepatan 45 *frame* per detik, sedangkan versi yang lebih ringan mencapai 155 *frame* per detik. Sejak diperkenalkan, arsitektur YOLO berkembang melalui banyak iterasi yang membawa peningkatan pada akurasi, efisiensi komputasi, dan kemampuan multitugas [2]. Secara umum, setiap varian YOLO tersusun atas tiga komponen utama, yaitu *backbone* yang mengekstraksi fitur dari citra, *neck* yang menggabungkan fitur lintas skala, dan *head* yang menghasilkan prediksi kotak pembatas beserta kelasnya.

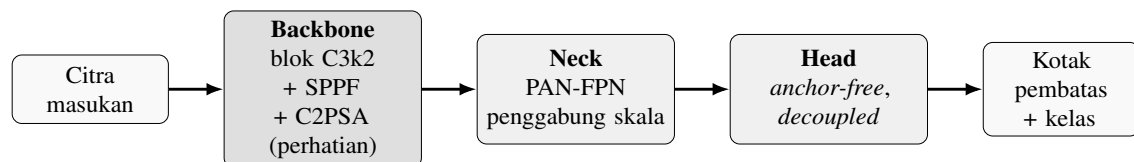
YOLOv8 dirilis oleh Ultralytics pada Januari 2023 dan tersedia dalam lima ukuran, yaitu *nano* (n), *small* (s), *medium* (m), *large* (l), dan *extra-large* (x) [2]. Arsitektur YOLOv8 memperkenalkan modul C2f (*cross-stage partial bottleneck with two convolutions*) pada *backbone* sebagai pengganti modul C3 pada versi sebelumnya, yang menggabungkan fitur tingkat tinggi dengan informasi kontekstual untuk meningkatkan akurasi deteksi. YOLOv8 juga menggunakan model *anchor-free* dengan *decoupled head* yang memproses tugas klasifikasi dan regresi kotak pembatas secara terpisah, sehingga mengurangi konflik antar-tugas dan meningkatkan konvergensi pelatihan. Susunan menyeluruh *backbone*, *neck*, dan *head* YOLOv8 ditunjukkan pada Gambar 2.1.



Gambar 2.1. Skema sederhana arsitektur YOLOv8 dengan alur *backbone–neck–head*, disusun dari uraian pada [2].

YOLO11 dirilis oleh Ultralytics pada September 2024 sebagai penerus YOLOv8 [3]. Khanam dan Hussain menganalisis inovasi arsitektur YOLO11, yang mencakup blok C3k2 (*Cross Stage Partial with kernel size 2*), komponen SPPF (*Spatial Pyramid Pooling*

Fast), dan modul C2PSA (*Convolutional block with Parallel Spatial Attention*). Blok C3k2 meningkatkan ekstraksi fitur dengan kernel berukuran lebih kecil yang lebih efisien secara komputasi, sedangkan C2PSA menambahkan mekanisme perhatian spasial paralel untuk memperkuat kemampuan model menangkap konteks global dalam gambar. YOLO11 tersedia dalam skala *nano* hingga *extra-large* dan mendukung berbagai tugas *computer vision* termasuk deteksi objek, segmentasi, dan estimasi pose. Susunan arsitekturnya beserta perbedaan utama terhadap YOLOv8 ditunjukkan pada Gambar 2.2.



Gambar 2.2. Skema sederhana arsitektur YOLO11, dengan kerangka yang sama seperti YOLOv8 dan tambahan blok C3k2 serta modul perhatian C2PSA pada *backbone*, disusun dari uraian pada [3].

Secara konseptual, YOLOv8 dan YOLO11 berbagi kerangka tiga bagian yang sama seperti pada Gambar 2.1 dan Gambar 2.2. *Backbone* memindai citra masukan dan menyaringnya menjadi fitur, mulai dari pola sederhana seperti tepi hingga bentuk objek yang lebih utuh. *Neck* kemudian menggabungkan fitur dari berbagai tingkat kedalaman agar objek besar maupun kecil sama-sama terwakili, lalu *head* di ujungnya mengubah fitur itu menjadi keluaran akhir berupa kotak pembatas beserta kelasnya. Perbedaan utama YOLO11 hanya terletak pada isi *backbone*, yaitu blok C3k2 yang lebih ringan menggantikan blok C2f, ditambah modul perhatian C2PSA yang membantu model memusatkan perhatian pada bagian citra yang penting. Karena kerangkanya identik dan hanya berbeda pada komponen internal inilah perbandingan keduanya menjadi setara, sehingga perbedaan performa dapat dikaitkan pada perbaikan arsitektur, bukan pada perbedaan kerangka.

Kedua model tersebut, yaitu YOLOv8 dan YOLO11, dipilih dalam penelitian ini karena keduanya adalah model resmi Ultralytics yang berbagi kerangka pelatihan, format data, dan antarmuka pemrograman yang identik. Keseragaman ini memungkinkan perbandingan yang terkontrol, saat perbedaan performa dapat diatribusikan pada perbedaan arsitektur dan bukan pada perbedaan implementasi. Selain itu, kedua model memiliki skalabilitas yang baik untuk perangkat dengan sumber daya terbatas, yang relevan untuk kebutuhan inferensi *real-time* pada sistem pemantauan [2].

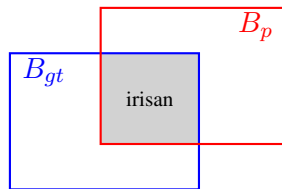
Model *transformer* seperti RT-DETR [23] adalah alternatif yang kompetitif dari segi akurasi. Akan tetapi, YOLOv8 dan YOLO11 dipilih karena efisiensi parameter, ekosistem yang matang, serta kemampuan *deployment* yang lebih luas pada berbagai jenis perangkat mulai dari server hingga *edge device*. Pertimbangan arsitektur yang lebih lengkap antara YOLO, RT-DETR, dan Faster R-CNN diuraikan pada Subbab 2.3.

2.2.3 Metrik Evaluasi Deteksi Objek

Evaluasi performa model deteksi objek menggunakan sejumlah metrik standar yang saling melengkapi [24]. Metrik-metrik ini mengukur dua aspek yang berbeda, yaitu ketepatan lokalisasi kotak pembatas dan kebenaran klasifikasi kelas. Penjelasan berikut menguraikan tiap metrik beserta cara membacanya, lengkap dengan contoh angka, agar makna setiap nilai dapat dipahami tanpa latar belakang khusus deteksi objek. Seluruh metrik ini digunakan secara konsisten pada semua eksperimen dalam penelitian ini agar perbandingan antar varian model bersifat setara.

Intersection over Union (IoU) mengukur seberapa besar tumpang tindih antara kotak pembatas hasil prediksi B_p dan kotak pembatas acuan B_{gt} (*ground truth*). Nilai IoU dihitung sebagai rasio luas irisan terhadap luas gabungan kedua kotak, sebagaimana dinyatakan pada Persamaan (2-1) dan diilustrasikan pada Gambar 2.3. Nilainya berkisar dari 0 sampai 1, dengan 0 berarti kedua kotak sama sekali tidak bertumpang tindih dan 1 berarti keduanya berimpit sempurna. Sebagai contoh, jika kotak prediksi dan kotak acuan memiliki luas irisan 60 satuan dan luas gabungan 100 satuan, maka IoU bernilai 0,6, yang menandakan lokalisasi cukup baik tetapi belum sempurna. Sebuah prediksi dinyatakan benar (*true positive*) jika nilai IoU-nya melampaui ambang tertentu, yang lazimnya ditetapkan 0,5.

$$IoU = \frac{\text{luas}(B_p \cap B_{gt})}{\text{luas}(B_p \cup B_{gt})} \quad (2-1)$$



Gambar 2.3. Ilustrasi *Intersection over Union*: luas irisan (area abu-abu) dibagi luas gabungan kedua kotak pembatas.

Berdasarkan kecocokan IoU tersebut, setiap prediksi dikelompokkan menjadi salah satu dari tiga kategori yang dirangkum pada Tabel 2.2. *True positive* (TP) adalah prediksi yang benar, yaitu kotak yang melampaui ambang IoU dan berlabel kelas yang tepat. *False positive* (FP) adalah prediksi yang salah, misalnya mendeteksi objek yang sebenarnya tidak ada atau memberi label kelas yang keliru. *False negative* (FN) adalah objek nyata yang gagal dideteksi oleh model. Ketiga kategori inilah yang menjadi bahan dasar perhitungan precision dan recall.

Precision mengukur ketelitian model, yaitu proporsi prediksi positif yang benar dari seluruh prediksi yang dihasilkan, sebagaimana Persamaan (2-2). *Recall* mengukur kelengkapan model, yaitu proporsi objek nyata yang berhasil dideteksi dari seluruh objek

Tabel 2.2. Kategori hasil prediksi pada evaluasi deteksi objek.

Kategori	Keterangan
<i>True positive</i> (TP)	Objek nyata yang terdeteksi dengan kotak yang tepat dan kelas yang benar.
<i>False positive</i> (FP)	Prediksi yang tidak sesuai objek nyata, baik karena salah lokasi maupun salah kelas.
<i>False negative</i> (FN)	Objek nyata yang tidak terdeteksi oleh model.

yang ada, sebagaimana Persamaan (2-3). Sebagai ilustrasi, misalkan pada suatu citra terdapat 50 objek orang dan model menghasilkan 60 prediksi orang. Jika 45 prediksi benar (TP), 15 prediksi salah (FP), dan 5 objek tidak terdeteksi (FN), maka precision bernilai $45/60 = 0,75$ dan recall bernilai $45/50 = 0,90$. Nilai ini berarti tiga dari empat prediksi model benar, sementara sembilan dari sepuluh objek nyata berhasil tertangkap.

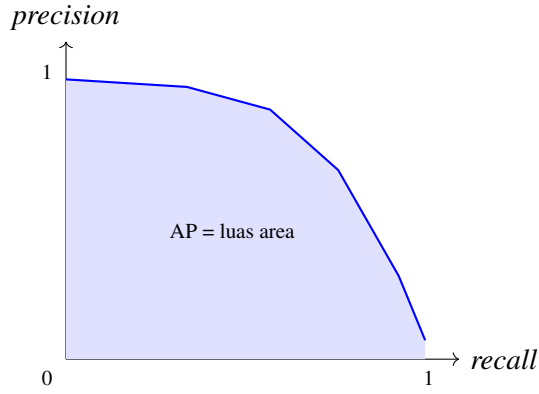
$$Precision = \frac{TP}{TP + FP} \quad (2-2)$$

$$Recall = \frac{TP}{TP + FN} \quad (2-3)$$

Precision dan recall sering kali berlawanan arah, karena model yang berani memprediksi banyak objek cenderung menaikkan recall tetapi menurunkan precision, begitu pula sebaliknya. Hubungan keduanya digambarkan oleh kurva *precision-recall*, yaitu grafik precision terhadap recall saat ambang keyakinan model divariasikan dari ketat ke longgar. *Average Precision* (AP) merangkum kurva ini menjadi satu angka, yaitu luas area di bawah kurva *precision-recall* seperti pada Persamaan (2-4) dan diilustrasikan pada Gambar 2.4, dengan $p(r)$ adalah precision pada tingkat recall r [24]. Semakin luas area di bawah kurva, semakin baik model menjaga precision tinggi sambil meningkatkan recall, sehingga nilai AP yang mendekati 1 menandakan detektor yang kuat pada satu kelas.

$$AP = \int_0^1 p(r) dr \quad (2-4)$$

mean Average Precision (mAP) adalah rata-rata AP di seluruh N kelas yang dievaluasi, sebagaimana Persamaan (2-5). Nilai mAP memberikan satu angka tunggal yang merangkum performa model pada semua kelas sekaligus, sehingga memudahkan perbandingan antar model. Karena dirata-ratakan, mAP yang tinggi belum menjamin tiap kelas terdeteksi baik, sehingga analisis per kelas tetap diperlukan untuk kasus dengan kelas yang timpang.



Gambar 2.4. Kurva *precision-recall*: *Average Precision* (AP) adalah luas area di bawah kurva (area arsir).

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i \quad (2-5)$$

Dataset COCO [25] memperkenalkan dua varian mAP yang kini menjadi standar evaluasi, yaitu mAP50 dan mAP50-95. mAP50 dihitung pada ambang IoU tunggal 0,50, sehingga hanya menuntut kotak prediksi kira-kira tepat dan bersifat longgar terhadap ketepatan lokalisasi. mAP50-95 adalah rata-rata mAP pada sepuluh ambang IoU dari 0,50 hingga 0,95 dengan langkah 0,05, sebagaimana Persamaan (2-6), sehingga menuntut kotak yang makin presisi pada ambang yang makin tinggi. Analoginya, mAP50 menilai apakah model menemukan objek di tempat yang kurang lebih benar, sedangkan mAP50-95 menilai seberapa rapi kotak itu menempel pada objek. Sebagai gambaran skala, COCO sendiri memuat anotasi 91 jenis objek pada 328 ribu gambar dengan total 2,5 juta *instance* berlabel.

$$mAP50-95 = \frac{1}{10} \sum_{t \in \{0,50, 0,55, \dots, 0,95\}} mAP_t \quad (2-6)$$

F1-score adalah rata-rata harmonik dari precision dan recall yang dinyatakan pada Persamaan (2-7), memberikan satu angka yang menyeimbangkan ketelitian dan kelengkapan. Berbeda dari rata-rata biasa, rata-rata harmonik menjatuhkan nilai F1 mendekati nilai terkecil di antara precision dan recall, sehingga model hanya memperoleh F1 tinggi jika kedua-duanya tinggi. Pada contoh sebelumnya dengan precision 0,75 dan recall 0,90, nilai F1 adalah $2 \cdot (0,75 \cdot 0,90) / (0,75 + 0,90) \approx 0,82$. Sifat ini membuat F1 sangat berguna untuk mengevaluasi kinerja per kelas pada kondisi ketidakseimbangan kelas, karena memberi penalti yang setara terhadap precision maupun recall yang rendah. Pada penelitian ini, metrik utama yang digunakan adalah mAP50, mAP50-95, *F1-score* per kelas, dan *confusion matrix* untuk menganalisis pola kesalahan deteksi antar kelas.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (2-7)$$

2.2.4 Ketidakseimbangan Kelas pada Deteksi Objek

Ketidakseimbangan kelas adalah kondisi ketika jumlah *instance* antar kelas dalam dataset berbeda jauh. Oksuz dkk. [19] mengidentifikasi ketidakseimbangan kelas sebagai salah satu masalah utama dalam deteksi objek, yang dapat menyebabkan model lebih condong mempelajari kelas dengan jumlah *instance* besar dan mengabaikan kelas dengan jumlah *instance* kecil. Dampaknya terlihat pada *Average Precision* yang rendah untuk kelas minoritas meskipun mAP keseluruhan tampak cukup baik.

Dataset yang dibangun dalam penelitian ini menunjukkan ketidakseimbangan kelas yang ekstrem. Kelas orang memiliki 10.694 *instance* pada data latih, sedangkan kelas truk hanya memiliki 45 *instance*, dengan rasio sebesar 237:1. Kelas bajaj dan bus masing-masing hanya memiliki 96 dan 92 *instance*. Kondisi ini merupakan konsekuensi langsung dari komposisi objek yang ada di kawasan Malioboro, bukan kesalahan pengumpulan data.

Ketidakseimbangan ini diperkirakan berpengaruh pada performa per kelas model yang dilatih, khususnya pada kelas dengan *instance* sedikit seperti truk, bajaj, dan sepeda. Kelas mayoritas seperti orang cenderung lebih mudah dipelajari karena contohnya berlimpah, sedangkan kelas minoritas berisiko terabaikan meskipun secara visual cukup khas. Analisis pengaruh ketidakseimbangan ini terhadap *F1-score* per kelas disajikan pada Bab IV.

2.2.5 Anotasi Semi-Otomatis

Anotasi dataset dalam jumlah besar secara manual membutuhkan waktu dan sumber daya yang besar. Pendekatan anotasi semi-otomatis menggabungkan prediksi model *pretrained* dengan verifikasi dan koreksi manual untuk mempercepat proses anotasi tanpa mengorbankan kualitas secara keseluruhan [26]. Tenckhoff dkk. menunjukkan bahwa platform anotasi berbasis umpan balik model, di mana model *pretrained* menghasilkan anotasi awal yang kemudian diverifikasi dan diperbaiki oleh anotator manusia, menghasilkan penghematan waktu dan jumlah interaksi anotasi yang terukur dibandingkan anotasi penuh secara manual.

Dalam penelitian ini, pendekatan ini diterapkan pada 1.252 gambar yang dianotasi menggunakan model *pretrained* untuk enam kelas yang tersedia pada dataset umum, diikuti dengan verifikasi dan koreksi manual. Kelas yang tidak tersedia pada model *pretrained*, yaitu andong dan becak, dianotasi secara penuh manual. Himpunan validasi dan pengujian sebanyak 329 gambar dianotasi secara penuh manual untuk memastikan integritas evaluasi. Rincian pipeline anotasi diuraikan pada BAB III.

2.2.6 Tingkat Pelayanan Pejalan Kaki

Tingkat pelayanan (*level of service*, LOS) pejalan kaki adalah konsep kuantitatif yang mengklasifikasikan kondisi pergerakan pejalan kaki berdasarkan kepadatan dan kenyamanan. Fruin [13] mendefinisikan enam tingkat pelayanan (A hingga F) berdasarkan studi fotografi *time-lapse* terhadap pergerakan pejalan kaki, dengan parameter utama berupa kepadatan orang per meter persegi dan ruang yang tersedia per pejalan kaki. Tingkat A menggambarkan kondisi pergerakan bebas tanpa konflik antar-pejalan kaki, sedangkan tingkat E dan F menggambarkan kondisi padat saat kontak fisik tidak terhindarkan dan pergerakan mandiri menjadi terbatas.

Di Indonesia, konsep LOS ini diadopsi dalam Peraturan Menteri PUPR Nomor 03/PRT/M/2014 tentang Pedoman Perencanaan, Penyediaan, dan Pemanfaatan Prasarana dan Sarana Jaringan Pejalan Kaki di Kawasan Perkotaan [1], yang menjadi acuan standar nasional untuk perencanaan jalur pedestrian. Pedoman tersebut menetapkan ambang kuantitatif tiap tingkat pelayanan berdasarkan luas ruang per orang, kecepatan rata-rata, volume arus, dan rasio volume terhadap kapasitas, sebagaimana dirangkum pada Tabel 2.3. Nilai ambang inilah yang menjadi dasar terukur bagi penetapan notifikasi kepadatan pada sistem yang dibangun, khususnya untuk kelas orang pada kamera yang dikalibrasi.

Tabel 2.3. Tingkatan standar pelayanan jalur pejalan kaki menurut Permen PUPR Nomor 03/PRT/M/2014 [1].

Tingkat Pelayanan	Ruang (m ² /orang)	Kecepatan (m/menit)	Volume Arus (orang/m/menit)	Rasio V/C
A	≥ 12	≥ 78	≤ 6,7	≤ 0,08
B	≥ 3,6	≥ 75	≤ 23	≤ 0,28
C	≥ 2,2	≥ 72	≤ 33	≤ 0,40
D	≥ 1,4	≥ 68	≤ 50	≤ 0,60
E	≥ 0,5	≥ 45	≤ 83	≤ 1,00
F	< 0,5	< 45	variabel	1,00

Pedoman tersebut tidak hanya mengklasifikasikan kondisi, tetapi juga menetapkan tingkat pelayanan minimum yang berbeda menurut peruntukan kawasan. Untuk kawasan campuran serta kawasan di sekitar simpul transportasi, dua karakter yang melekat pada kawasan Malioboro sebagai pusat perdagangan dan wisata yang berdekatan dengan Stasiun Tugu, pedoman menetapkan tingkat pelayanan minimum Standar D, yaitu ruang pejalan kaki paling sedikit 1,4 m² per orang [1]. Standar D menandai batas bawah kondisi yang masih nyaman, saat pejalan kaki sudah harus sering menyesuaikan kecepatan dan posisi karena potensi konflik arus. Ambang ini menjadi rujukan kuantitatif yang masuk akal untuk menandai kondisi kepadatan yang perlu diwaspadai pada sistem pemantauan.

Untuk menghitung kepadatan dari hasil deteksi, diperlukan pemetaan antara

koordinat piksel dan koordinat nyata di bidang tanah. Pemetaan ini dilakukan melalui estimasi homografi planar, yaitu transformasi perspektif yang mengubah titik-titik pada bidang gambar menjadi koordinat metrik di bidang tanah menggunakan korespondensi titik acuan yang diketahui dimensinya [13]. Dengan homografi yang telah dikalibrasi, jumlah objek yang terdeteksi dalam suatu zona dapat dibagi dengan luas zona tersebut dalam meter persegi untuk memperoleh estimasi kepadatan. Nilai kepadatan itu kemudian dipetakan ke tingkat pelayanan sesuai klasifikasi Fruin dan Permen PUPR 2014 pada Tabel 2.3.

2.2.7 Arsitektur Aplikasi Web Pemantauan

Sistem pemantauan yang dibangun mengikuti arsitektur klien-server, yaitu pemisahan antara *server* yang memproses data dan klien yang menampilkannya kepada pengguna. Komunikasi antara keduanya menggunakan gaya *Representational State Transfer* (REST), yaitu sekumpulan aturan perancangan antarmuka berbasis HTTP yang menjadikan tiap sumber daya dapat diakses melalui alamat seragam dengan operasi standar [27]. Pendekatan ini membuat sisi pemroses dan sisi tampilan dapat dikembangkan secara terpisah selama keduanya menyepakati antarmuka yang sama. Keterpisahan inilah yang memungkinkan model deteksi berjalan di sisi *server* tanpa membebani perangkat pengguna.

Pemantauan kepadatan menuntut pembaruan tampilan yang cepat begitu hasil deteksi tersedia. Pada model permintaan biasa, klien harus berulang kali menanyakan data terbaru ke *server* (*polling*), yang boros dan menimbulkan jeda. WebSocket menyediakan kanal komunikasi dua arah yang tetap terbuka di atas satu sambungan, sehingga *server* dapat langsung mendorong data ke klien tanpa diminta setiap kali [28]. Mekanisme ini membuat angka kepadatan dan notifikasi dapat diperbarui hampir seketika tanpa memuat ulang halaman.

Selain angka, sistem juga menampilkan *stream* video beserta kotak deteksinya. Tampilan ini menggunakan *Motion JPEG* (MJPEG), yaitu teknik yang mengirim video sebagai rentetan citra JPEG yang masing-masing dikodekan secara mandiri [29]. Karena tiap *frame* berdiri sendiri, MJPEG mudah dipadukan dengan hasil inferensi dan dapat ditampilkan langsung oleh peramban tanpa pemutar khusus. Kesederhanaan dan latensi rendahnya membuat MJPEG cocok untuk menampilkan hasil deteksi secara *real-time* pada antarmuka pemantauan. Penerapan ketiga konsep ini pada sistem yang dibangun diuraikan pada Bab III.

2.3 Analisis Perbandingan Metode

Pemilihan metode deteksi objek untuk sistem pemantauan di kawasan Malioboro tidak dapat dilepaskan dari kebutuhan spesifik permasalahan yang dihadapi. Sistem

harus melakukan inferensi secara *real-time* pada perangkat dengan sumber daya terbatas, mampu mendeteksi pejalan kaki sekaligus kelas kendaraan lokal yang tidak tercakup dalam model umum, dapat dilatih dari *dataset* kustom berukuran terbatas, serta layak diterapkan pada lingkungan operasional. Empat kebutuhan inilah yang menjadi tolok ukur dalam membandingkan metode-metode yang tersedia. Subbab ini meninjau kelebihan dan kekurangan tiap pendekatan terhadap tolok ukur tersebut, lalu menjustifikasi metode yang dipilih.

2.3.1 Perbandingan Pendekatan Deteksi Objek

Pendekatan *two-stage* yang diwakili oleh Faster R-CNN memisahkan deteksi menjadi dua tahap, yaitu pembangkitan usulan wilayah (*region proposal*) dan klasifikasi pada tiap usulan tersebut [20]. Pemisahan ini menghasilkan akurasi lokalisasi yang tinggi, tetapi rangkaian dua tahap membuat latensi inferensi meningkat sehingga sulit memenuhi kebutuhan *real-time* pada perangkat berdaya rendah. Untuk sistem yang memproses *stream* kamera secara terus-menerus, latensi yang tinggi menjadi hambatan utama.

Pendekatan berbasis *transformer* yang diwakili RT-DETR menghilangkan tahap *non-maximum suppression* (NMS) dan menjadikan proses deteksi bersifat *end-to-end* [23]. Pada skala server, RT-DETR menawarkan keseimbangan kecepatan dan akurasi yang kuat. Varian RT-DETR-R50 mencapai 53,1% AP pada COCO *val2017* dengan 108 FPS pada GPU T4, mengungguli detektor YOLO sekelasnya dalam dua metrik tersebut [23]. Keterbatasannya muncul pada sisi efisiensi model. Varian terkecil RT-DETR, yaitu RT-DETR-R18, memiliki sekitar 20 juta parameter dan tidak menyediakan tier ringan setara kelas *nano*. Selain itu, detektor berbasis *transformer* cenderung membutuhkan data pelatihan yang lebih banyak untuk konvergen, sehingga kurang sesuai untuk *dataset* kustom berukuran terbatas.

Pendekatan *one-stage* berbasis CNN yang diwakili YOLO memprediksi kelas dan kotak pembatas dalam satu kali proses maju, sehingga inferensinya cepat [21]. Keluarga YOLO menyediakan rentang ukuran model yang lebar, mulai dari tier *nano* dengan sekitar 2,6 juta parameter hingga varian terbesar [2, 3]. Pada sisi akurasi, varian terbesar YOLO11 mencapai 54,7% mAP pada COCO dengan jumlah parameter yang lebih sedikit dibanding varian *transformer* setara, sehingga anggapan bahwa YOLO selalu kalah akurat terhadap *transformer* tidak lagi berlaku mutlak. Ketersediaan tier ringan membuat YOLO dapat dijalankan pada perangkat terbatas, sementara ekosistem dan bobot *pretrained* yang matang memudahkan pelatihan ulang pada *dataset* kustom.

Mengacu pada empat tolok ukur permasalahan, YOLO memberikan kesesuaian terbaik. Faster R-CNN tersisih karena latensi yang tinggi, sedangkan RT-DETR, meski unggul pada skala server, tidak menyediakan tier ringan dan lebih menuntut data pelatihan. Penting dicatat bahwa keputusan ini bukan karena YOLO unggul secara mutlak di semua

Tabel 2.4. Perbandingan pendekatan deteksi objek terhadap kebutuhan sistem pemantauan.

Aspek	Faster R-CNN	RT-DETR	YOLO (v8/v11)
Pendekatan	<i>Two-stage CNN</i>	<i>Transformer end-to-end</i>	<i>One-stage CNN</i>
Kecepatan inferensi	Rendah	Tinggi	Tinggi
Varian teringan	Tidak ada tier ringan	≈ 20 juta parameter	$\approx 2,6$ juta parameter
Kesesuaian perangkat terbatas	Kurang (latensi tinggi)	GPU server	Perangkat terbatas
Kebutuhan data latih	Sedang	Tinggi	Sedang

kondisi, melainkan karena ketersediaan tier ringan, efisiensi parameter, dan kematangan ekosistem YOLO selaras dengan kendala *real-time*, perangkat terbatas, dan *dataset* kustom yang menjadi ciri penelitian ini.

2.3.2 Pemilihan Varian YOLO untuk Penelitian

Setelah keluarga YOLO ditetapkan, langkah berikutnya adalah memilih versi yang dibandingkan. Penelitian ini menggunakan YOLOv8 dan YOLO11 karena keduanya merupakan model unggulan resmi dari Ultralytics yang berbagi kerangka pelatihan, format data, alur augmentasi, dan antarmuka pemrograman yang identik [2, 3]. Kesamaan ini membuat perbedaan performa di antara keduanya dapat diatribusikan ke arsitektur, bukan ke perbedaan implementasi *framework*, sehingga perbandingan menjadi terkontrol secara metodologis.

Versi YOLO lain sengaja tidak dimasukkan agar kontrol metodologis tetap terjaga. YOLOv9 [30] dikembangkan oleh kelompok penulis YOLOv7 di atas basis kode YOLOv5 dengan memperkenalkan *Programmable Gradient Information* (PGI) dan arsitektur GELAN, sehingga garis keturunan dan filosofi desainnya berbeda dari model resmi Ultralytics. YOLOv10 [31] dikembangkan oleh kelompok peneliti Universitas Tsinghua dan menekankan inferensi *end-to-end* tanpa *non-maximum suppression* melalui mekanisme *consistent dual assignments*, suatu arah desain yang berbeda dari YOLOv8 dan YOLO11. YOLOv12 [32] mengusung desain *attention-centric* dengan mekanisme *area attention*, sedangkan YOLO26 [33] yang dirilis Ultralytics pada 2026 menjadikan inferensi *end-to-end* tanpa NMS dan regresi tanpa *Distribution Focal Loss* sebagai bawaan. Memasukkan versi-versi tersebut ke dalam satu perbandingan akan menambah variabel pengganggu berupa perbedaan asal basis kode, filosofi desain, dan kematangan *tooling*, sehingga efek arsitektur tidak lagi dapat diisolasi.

Kematangan ekosistem juga menjadi pertimbangan praktis. Pada saat penelitian ini dilakukan, YOLOv8 dan YOLO11 telah memiliki dukungan *tooling*, dokumentasi, dan bobot *pretrained* yang stabil di bawah satu pustaka resmi, sedangkan sebagian versi yang lebih baru masih berkembang cepat dan belum tentu setara dukungannya. Membatasi perbandingan pada dua model unggulan Ultralytics yang berbagi kerangka pelatihan, format data, dan antarmuka yang identik justru menghasilkan perbandingan yang lebih valid, karena perbedaan performa dapat diatribusikan pada arsitektur dan bukan pada perbedaan implementasi.

Empat ukuran model, yaitu *nano* (n), *small* (s), *medium* (m), dan *large* (l), diuji pada tiap versi untuk memetakan keterkaitan antara ukuran model, akurasi, dan kecepatan yang relevan dengan kebutuhan *real-time*. Varian terbesar (x) dikecualikan karena keterbatasan perangkat pelatihan dan deteksi serta tidak praktis untuk penerapan *real-time*, dan pengecualian ini dinyatakan sebagai batasan penelitian.

Berdasarkan analisis di atas, metode yang dipilih adalah studi perbandingan YOLOv8 dan YOLO11 pada empat ukuran model, dilatih dan dievaluasi pada *dataset* kustom kawasan Malioboro. Model dengan keseimbangan akurasi dan kecepatan terbaik akan dipilih untuk diterapkan pada sistem pemantauan. Rincian tahapan pelatihan, konfigurasi, dan evaluasi diuraikan pada Bab III.

BAB III

METODE PENELITIAN

Bab ini menjelaskan tahapan yang ditempuh untuk menjawab tujuan penelitian, mulai dari penyiapan data hingga implementasi sistem pemantauan. Uraian disusun mengikuti urutan kerja penelitian, yaitu pengumpulan dan penyiapan data, pelatihan dan evaluasi model, penyetelan model terpilih, lalu perancangan sistem. Tiap tahap dijelaskan pada subbab tersendiri agar prosedurnya dapat diikuti dan direproduksi.

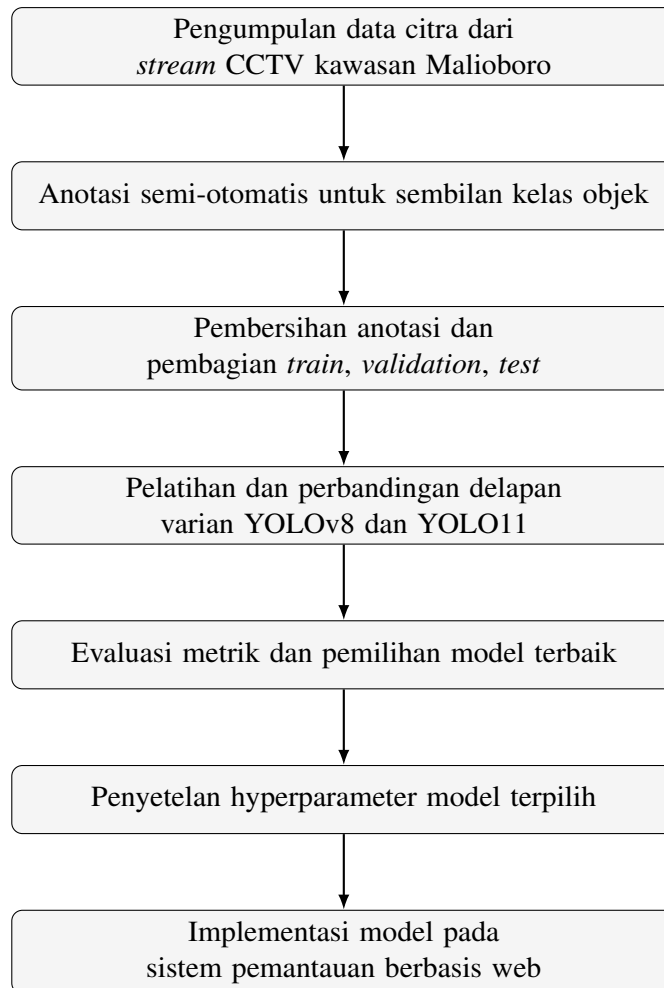
3.1 Gambaran Umum Penelitian

Penelitian ini dilaksanakan melalui tujuh tahap berurutan yang membentuk satu alur dari penyediaan data hingga sistem yang berjalan, seperti ditunjukkan pada Gambar 3.5. Tahap pertama adalah pengumpulan data citra dari *stream* kamera CCTV kawasan Malioboro. Citra yang terkumpul kemudian dianotasi pada tahap kedua menggunakan pendekatan semi-otomatis untuk menghasilkan label sembilan kelas objek. Tahap ketiga membersihkan anotasi dari kesalahan geometris dan membagi data menjadi himpunan latih, validasi, dan uji. Tahap keempat melatih delapan varian model YOLOv8 dan YOLO11 dengan konfigurasi identik untuk membandingkannya secara terkontrol. Tahap kelima mengevaluasi performa tiap varian menggunakan sejumlah metrik dan memilih model dengan keseimbangan akurasi serta kecepatan terbaik. Tahap keenam menyetel *hyperparameter* model terpilih untuk mengoptimasi performanya. Tahap ketujuh menerapkan model hasil penyetelan ke dalam sistem pemantauan berbasis web yang memproses video secara *real-time* dan menyajikan informasi kepadatan beserta notifikasinya.

Ketujuh tahap tersebut bersifat berurutan, yaitu keluaran satu tahap menjadi masukan bagi tahap berikutnya. Kualitas anotasi pada tahap kedua dan ketiga menetapkan batas atas performa yang dapat dicapai model pada tahap keempat, sedangkan performa model menentukan keandalan informasi kepadatan yang disajikan sistem pada tahap ketujuh. Karena keterkaitan tersebut, tahap penyiapan data dan pelatihan model diuraikan secara rinci pada subbab-subbab berikutnya sebelum perancangan sistem dibahas pada bagian akhir bab ini.

3.2 Alat dan Bahan Tugas Akhir

Subbab ini menjelaskan perangkat keras dan perangkat lunak yang digunakan sepanjang penelitian. Spesifikasi dicantumkan agar tahap pelatihan dan pengujian dapat diulang pada lingkungan yang setara. Pemisahan antara lingkungan pelatihan berperforma tinggi dan perangkat penerapan yang terbatas juga ditegaskan di sini, karena keduanya



Gambar 3.5. Alur tahapan penelitian.

berperan berbeda dalam menilai akurasi dan kecepatan model.

3.2.1 Perangkat Keras

Pelatihan model dilakukan pada lingkungan komputasi performa tinggi (*high-performance computing*, HPC) milik Departemen Teknik Elektro dan Teknologi Informasi, Fakultas Teknik, Universitas Gadjah Mada. Lingkungan ini menyediakan GPU NVIDIA Tesla V100-SXM2 dengan memori 32 GB, yang memungkinkan pelatihan seluruh varian model termasuk ukuran *large* tanpa kendala keterbatasan memori. Kapasitas memori sebesar itu penting karena pelatihan memakai *batch size* otomatis yang menyesuaikan beban GPU, sehingga varian besar tetap dapat dilatih tanpa kehabisan memori.

Perancangan dan pengujian sistem dilakukan pada laptop dengan prosesor AMD Ryzen 5 5600H, GPU NVIDIA GeForce GTX 1650 bermemori 4 GB, RAM 16 GB (2×8 GB DDR4-3200), dan sistem operasi Windows 11 Home Single Language 64-bit. Spesifikasi ini mewakili kelas perangkat dengan sumber daya terbatas sebagai target

penerapan *real-time*. Pemilihan perangkat kelas ini disengaja agar pengukuran kecepatan inferensi mencerminkan kondisi penerapan yang realistis, bukan performa pada server kelas atas.

3.2.2 Perangkat Lunak

Pelatihan dan inferensi model menggunakan pustaka Ultralytics yang menyediakan implementasi YOLOv8 dan YOLO11 di atas *framework* PyTorch. Proses anotasi data dilakukan pada platform Roboflow, sedangkan pembacaan dan pemrosesan *frame* video dilakukan dengan pustaka OpenCV. Versi perangkat lunak utama yang digunakan pada lingkungan pelatihan dirangkum pada Tabel 3.1.

Tabel 3.1. Versi perangkat lunak utama yang digunakan pada lingkungan pelatihan.

Perangkat lunak	Versi
Python	3.10.12
PyTorch	2.2.0a0+81ea7a4
CUDA	12.3
Ultralytics	8.4.63
OpenCV (cv2)	4.9.0
Roboflow	Platform anotasi berbasis web
<i>Backend</i> sistem web	FastAPI (Python)
<i>Frontend</i> sistem web	React, Vite, Tailwind
Basis data riwayat deteksi	SQLite

3.3 Pengumpulan Data

Pengumpulan data bertujuan membangun kumpulan citra primer yang mewakili kondisi nyata kawasan Malioboro sebagai bahan pelatihan dan pengujian model deteksi. Subbab ini menjelaskan sumber dan mekanisme pengambilan citra, penjadwalan pengambilan untuk memperoleh variasi kondisi, serta penyaringan citra yang tidak layak digunakan.

3.3.1 Sumber dan Mekanisme Pengambilan *Frame*

Data primer penelitian berupa citra yang diambil dari *stream* video langsung 22 kamera CCTV publik di kawasan Malioboro yang dikelola Pemerintah Kota Yogyakarta. *Stream* video disediakan dalam format *HTTP Live Streaming* (HLS) melalui berkas *playlist* berekstensi *.m3u8*, sehingga setiap kamera dapat diakses sebagai sumber video daring. Akses daring ini memungkinkan pengambilan citra tanpa pemasangan perangkat tambahan di lapangan, sekaligus memastikan data berasal dari kondisi nyata kawasan.

Pengambilan citra dilakukan secara otomatis menggunakan program yang ditulis dengan bahasa Python. Program memiliki penjadwal internal sendiri, yaitu dijalankan satu kali lalu menunggu hingga slot waktu berikutnya tanpa bergantung pada penjadwal sistem eksternal. Pada setiap slot, program mengambil satu *frame* dari tiap kamera.

Pengambilan satu *frame* dari *stream* HLS memerlukan penanganan khusus karena *stream* menyimpan *buffer* yang berisi *frame* lama. Program membuka *stream*, menunggu dua detik agar koneksi stabil, membuang 30 *frame* pertama dari *buffer*, lalu mengambil satu *frame* terkini. Jika pengambilan gagal, proses diulang hingga tiga kali sebelum kamera tersebut ditandai gagal. Setiap *frame* yang berhasil disimpan dalam format JPEG dengan kualitas 92, dan diberi nama berkas yang memuat nomor kamera, tanggal, dan waktu pengambilan, sehingga tiap citra dapat diidentifikasi sumber serta waktunya tanpa berkas catatan terpisah. Prosedur lengkap pengambilan satu *frame* dari *stream* HLS dirangkum pada Algoritma 1.

Algorithm 1 Pengambilan satu *frame* terkini dari *stream* HLS.

Require: URL *playlist* .m3u8 dari satu kamera

Ensure: satu *frame* terkini, atau status gagal

```

1: for  $i \leftarrow 1$  sampai 3 do
2:   buka stream dari URL
3:   tunggu 2 detik agar koneksi stabil
4:   buang 30 frame pertama dari buffer
5:    $f \leftarrow$  ambil satu frame terkini
6:   if  $f$  berhasil diambil then
7:     simpan  $f$  sebagai berkas JPEG kualitas 92
8:     kembalikan  $f$ 
9:   end if
10:  tutup stream
11: end for
12: tandai kamera sebagai gagal

```

3.3.2 Penjadwalan dan Variasi Kondisi

Pengumpulan data berlangsung selama satu pekan, yaitu 30 April hingga 6 Mei 2026. Rentang waktu ini dipilih agar data mencakup variasi kondisi antar hari, antar jam, dan antar cuaca, sehingga model dilatih pada keragaman situasi yang mendekati kondisi nyata. Variasi tersebut penting karena kepadatan dan pencahayaan di Malioboro berubah tajam antara dini hari yang sepi dan malam hari yang ramai, sehingga model perlu melihat rentang kondisi yang luas.

Dalam satu hari, pengambilan dilakukan pada 11 slot waktu yang mewakili rentang kepadatan dan pencahayaan, mulai dari dini hari yang sepi, meningkat sepanjang siang, hingga memuncak pada sore dan malam hari sesuai karakter kawasan wisata Malioboro.

Pembagian slot setiap dua jam menyeimbangkan cakupan sepanjang hari dengan jumlah citra yang masih wajar untuk dianotasi. Rincian slot ditunjukkan pada Tabel 3.2.

Tabel 3.2. Slot waktu pengambilan citra dalam satu hari.

Pukul	Label Slot	Keterangan Kondisi
00.00	<i>midnight</i>	keramaian larut malam mulai menurun
02.00	<i>dawn</i>	kondisi paling sepi
06.00	<i>early_morning</i>	aktivitas masih rendah
08.00	<i>morning</i>	aktivitas pagi meningkat
10.00	<i>midmorning</i>	kunjungan wisatawan meningkat
12.00	<i>noon</i>	keramaian siang
14.00	<i>afternoon</i>	keramaian siang berlanjut
16.00	<i>late_afternoon</i>	keramaian menjelang sore
18.00	<i>evening</i>	awal puncak keramaian malam
20.00	<i>night</i>	puncak keramaian malam
22.00	<i>late_night</i>	keramaian mulai menurun

Dengan 11 slot, 22 kamera, dan 7 hari, jumlah *frame* yang ditargetkan secara teoretis adalah 1.694. Sebagian percobaan pengambilan gagal akibat gangguan koneksi *stream*, sehingga jumlah citra yang berhasil dikumpulkan adalah 1.581. Seluruh citra tersebut digunakan tanpa penyaringan lebih lanjut, lalu dilanjutkan ke tahap anotasi.

3.4 Anotasi Data

Anotasi bertujuan memberi label berupa kotak pembatas dan kelas pada setiap objek di dalam citra. Subbab ini menjelaskan strategi anotasi yang dipilih akibat keterbatasan waktu, perbandingan beberapa pendekatan pelabelan otomatis untuk menentukan *baseline*, serta anotasi manual dan koreksi yang melengkapinya. Seluruh proses anotasi dikerjakan pada platform Roboflow.

3.4.1 Strategi Anotasi Semi-Otomatis

Rencana awal penelitian adalah menganotasi seluruh citra secara manual agar kualitas label terjaga. Setelah 329 citra selesai dianotasi manual, waktu yang tersedia dinilai tidak mencukupi untuk menyelesaikan seluruh 1.581 citra dengan cara yang sama dalam jadwal penyusunan skripsi. Oleh karena itu, sisa 1.252 citra dianotasi menggunakan pendekatan semi-otomatis, yaitu pelabelan otomatis oleh model *pretrained* yang kemudian diperiksa dan dikoreksi secara manual [26].

Dengan pembagian tersebut, 329 citra hasil anotasi manual digunakan sebagai data dengan label yang dianggap bersih, sedangkan 1.252 citra hasil anotasi semi-otomatis digunakan sebagai data latih. Pemisahan ini menempatkan label yang paling terpercaya

pada himpunan yang dipakai untuk mengukur performa, sementara himpunan latih boleh memuat sedikit ketidaksempurnaan label tanpa membiaskan evaluasi. Pembagian ke dalam himpunan latih, validasi, dan uji diuraikan pada Subbab 3.5.

3.4.2 Perbandingan Pendekatan Pelabelan Otomatis

Tahap pelabelan otomatis memerlukan model dasar yang menghasilkan kotak pembatas awal. Tiga kombinasi model diuji pada 1.252 citra latih untuk menentukan pilihan terbaik. Pada ketiga kombinasi, kelas bajaj dideteksi oleh model YOLOv12s yang telah dilatih pada dataset BMD-45 [12], sedangkan kelas lain dideteksi oleh model dasar yang berbeda, yaitu SAM3 [34], YOLO11x, atau YOLOv8x. Pelabelan dengan SAM3 dilakukan melalui layanan Roboflow API. Jumlah *instance* hasil tiap kombinasi dirangkum pada Tabel 3.3.

Tabel 3.3. Jumlah *instance* hasil tiga kombinasi pelabelan otomatis pada 1.252 citra latih.

Kelas	SAM3	YOLO11x	YOLOv8x
andong	0	0	0
bajaj	112	112	112
becak	0	0	0
sepeda	3.916	277	265
bus	549	114	123
mobil	11.964	1.789	1.821
motor	32.074	1.894	2.124
orang	81.000	7.784	9.803
truk	619	276	250
Total	130.234	12.246	14.498

Dua kelas, yaitu andong dan becak, tidak terdeteksi sama sekali oleh ketiga kombinasi karena tidak tersedia pada model dasar mana pun, sementara bajaj menghasilkan jumlah yang sama, yaitu 112, karena memakai model yang identik. Perbedaan utama terletak pada kelas umum. Kombinasi berbasis SAM3 menghasilkan 130.234 *instance*, hampir sembilan kali lipat dibanding YOLOv8x, dengan kelas orang saja mencapai 81.000. SAM3 dirancang untuk menemukan seluruh *instance* dari suatu konsep sekaligus, sehingga pada citra yang padat cenderung menghasilkan deteksi yang sangat banyak dan didominasi *false positive*. Akibatnya, biaya koreksi manualnya justru lebih besar daripada manfaatnya.

Kombinasi YOLOv8x dipilih sebagai *baseline*. Dibandingkan YOLO11x, YOLOv8x bersifat lebih agresif, terlihat dari jumlah *instance* yang lebih banyak, yaitu 14.498 berbanding 12.246. Sifat agresif ini menguntungkan pada tahap awal pelabelan karena menghapus *false positive* lebih cepat daripada menambahkan deteksi yang terlewat secara manual. SAM3 tidak dipilih karena tingkat deteksi berlebihnya terlalu tinggi,

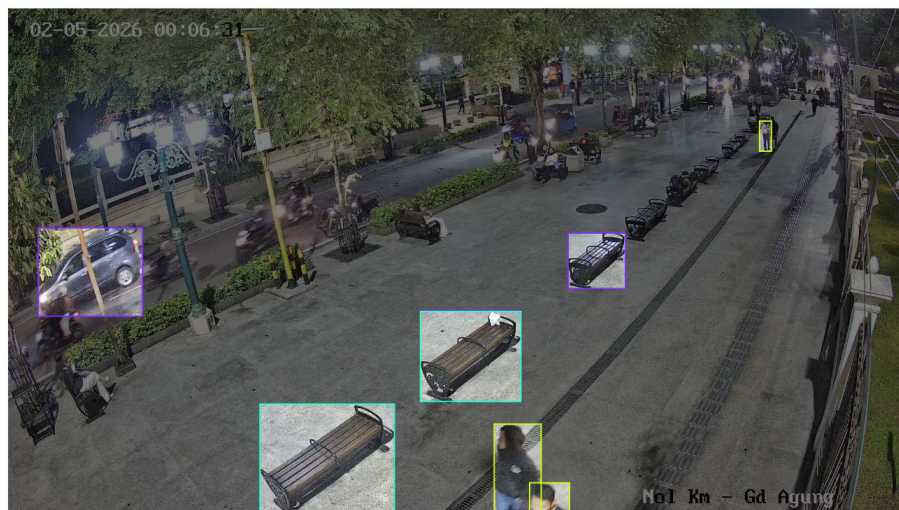
sehingga *noise* yang dihasilkan tidak sebanding dengan waktu koreksi yang diperlukan.

Pemilihan ini berangkat dari asimetri biaya koreksi manual. Menghapus *false positive* hanya menuntut anotator mengenali dan membuang kotak yang keliru, sedangkan menambahkan deteksi yang terlewat menuntut anotator menemukan objek yang luput di seluruh citra, pekerjaan yang jauh lebih lambat dan rawan terabaikan. Karena itu, model dasar yang sedikit terlalu agresif lebih disukai daripada yang terlalu konservatif, sepanjang tingkat *false positive*-nya masih wajar. SAM3 berada di ujung yang berlawanan, karena tingkat deteksi berlebihan begitu tinggi sehingga keuntungan dari kelengkapan deteksi tertutup oleh beban memverifikasi puluhan ribu kotak tambahan. YOLOv8x menempati titik tengah yang paling praktis, yaitu cukup agresif untuk meminimalkan objek terlewat tanpa membanjiri anotator dengan *noise*.

3.4.3 Anotasi Manual dan Koreksi

Hasil pelabelan otomatis melengkapi sebagian besar kelas, tetapi belum mencakup andong dan becak yang tidak terdeteksi oleh model dasar mana pun. Kedua kelas tersebut dianotasi secara manual penuh pada citra latih. Penambahan manual ini diperlukan karena andong dan becak tidak memiliki padanan kelas pada model *pretrained*, sehingga tidak mungkin muncul dari pelabelan otomatis.

Selain penambahan dua kelas tersebut, hasil *baseline* YOLOv8x ditinjau dan dikoreksi secara manual semaksimal mungkin. Koreksi ini penting karena kualitas label pada data latih menjadi batas atas performa yang dapat dicapai model. Gambar 3.6 menunjukkan salah satu contoh kesalahan, yaitu tiga bangku taman yang terdeteksi sebagai objek (*false positive*).



Gambar 3.6. Contoh kesalahan deteksi tiga bangku taman yang terdeteksi sebagai objek.

Selain contoh pada gambar tersebut, penulis menemukan beberapa pola kesalahan yang berulang. Kotak pembatas kelas bus dan truk sering tumpang tindih pada objek yang

sama. Kelas bajaj kadang terlabeli sebagai truk, sedangkan becak terkadang terdeteksi sebagai motor. Latar belakang citra juga beberapa kali terdeteksi sebagai salah satu dari sembilan kelas. Seluruh kesalahan tersebut diperbaiki secara manual, baik dengan menghapus deteksi yang keliru, memperbaiki label kelas, maupun menyesuaikan kotak pembatas.

Anotasi manual dan koreksi ini mengubah jumlah dan komposisi label dari hasil *baseline*. Perubahan tersebut mencakup penambahan *instance* kelas andong dan becak sekaligus pengurangan deteksi keliru pada kelas lain. Distribusi akhir setelah pembersihan beserta pembagian data diuraikan pada Subbab 3.5.

3.5 Pembersihan dan Pembagian Dataset

Sebelum digunakan untuk pelatihan dan evaluasi, anotasi dibagi ke dalam tiga himpunan lalu dibersihkan dari kesalahan geometris. Urutan ini dipilih agar pembagian dilakukan lebih dulu berdasarkan asal anotasi, baru pembersihan diterapkan seragam pada tiap himpunan. Subbab ini menjelaskan pembagian data, proses pembersihan anotasi, serta distribusi kelas akhir beserta ketidakseimbangannya.

3.5.1 Pembagian *Train*, *Validation*, dan *Test*

Data dibagi ke dalam tiga himpunan, yaitu latih (*train*), validasi (*validation*), dan uji (*test*). Pembagian mengikuti cara anotasi yang dijelaskan pada Subbab 3.4. Sebanyak 1.252 citra hasil anotasi semi-otomatis digunakan sebagai himpunan latih, sedangkan 329 citra hasil anotasi manual dibagi menjadi 165 citra validasi dan 164 citra uji.

Penempatan seluruh citra ber-anotasi manual pada himpunan validasi dan uji dilakukan dengan sengaja. Kedua himpunan tersebut menjadi acuan pengukuran performa model, sehingga keduanya perlu memiliki label yang andal. Dengan label manual pada validasi dan uji, performa model diukur terhadap acuan yang tepercaya, terlepas dari kemungkinan kesalahan yang masih tersisa pada label semi-otomatis di himpunan latih.

3.5.2 Pembersihan Geometris Anotasi

Anotasi dibersihkan dari kotak pembatas yang tidak valid secara geometris menggunakan lima filter. Kotak yang sangat kecil disaring karena objek dengan luas di bawah 0,1% luas citra setara dengan kurang dari 20×20 piksel setelah citra diskala ke 640×640 , sehingga terlalu kecil untuk dipelajari dan sejalan dengan perhatian terhadap objek berukuran sangat kecil pada COCO [25]. Kotak yang menutupi lebih dari 90% luas citra disaring sebagai kesalahan anotasi, sedangkan kotak dengan rasio sisi yang ekstrem, baik terlalu tipis maupun terlalu lebar, juga disingkirkan. Rincian ambang tiap filter ditunjukkan pada Tabel 3.4.

Tabel 3.4. Filter pembersihan kotak pembatas.

Kriteria	Ambang	Tindakan
Luas ternormalisasi minimum	0,001	buang kotak $< 0,1\%$ luas citra
Luas ternormalisasi maksimum	0,90	buang kotak $> 90\%$ luas citra
Rasio sisi minimum	0,15	buang kotak terlalu tipis (lebar/tinggi $< 0,15$)
Rasio sisi maksimum	10,0	buang kotak terlalu lebar (lebar/tinggi > 10)
Ukuran ternormalisasi maksimum	1,0	buang kotak yang melampaui batas citra

Penerapan kelima filter mengurangi sebagian kecil anotasi pada tiap himpunan. Tabel 3.5 merangkum jumlah *instance* sebelum dan sesudah pembersihan. Sebagian besar anotasi dipertahankan, dengan kelas orang paling banyak terdampak karena banyaknya kotak berukuran kecil pada citra kerumunan yang padat.

Tabel 3.5. Jumlah *instance* sebelum dan sesudah pembersihan pada tiap himpunan.

Himpunan	Sebelum	Sesudah	Dibuang	Dipertahankan
Latih	18.536	17.500	1.036	94,4%
Validasi	5.372	4.222	1.150	78,6%
Uji	5.605	4.288	1.317	76,5%

3.5.3 Distribusi dan Ketidakseimbangan Kelas

Distribusi kelas akhir setelah pembersihan ditunjukkan pada Tabel 3.6. Distribusi ini sangat tidak seimbang. Pada himpunan latih, kelas orang memiliki 10.694 *instance*, sedangkan kelas truk hanya 45, sehingga rasio antara kelas terbanyak dan paling sedikit mencapai sekitar 237 banding 1.

Ketidakseimbangan ini mencerminkan komposisi objek yang sebenarnya di kawasan Malioboro, yaitu pejalan kaki, becak, motor, dan mobil mendominasi, sedangkan truk, bus, bajaj, dan sepeda jauh lebih jarang muncul. Ketidakseimbangan kelas merupakan persoalan yang umum pada deteksi objek dan dapat menurunkan performa pada kelas minoritas [19]. Pengaruh ketidakseimbangan ini terhadap performa deteksi tiap kelas dianalisis pada Bab IV.

3.6 Pelatihan dan Perbandingan Model

Pelatihan bertujuan menghasilkan model deteksi terbaik dari dua keluarga arsitektur yang dibandingkan, yaitu YOLOv8 dan YOLO11. Subbab ini menjelaskan konfigurasi pelatihan, prapemrosesan dan augmentasi data masukan, serta skema pelatihan

Tabel 3.6. Distribusi jumlah *instance* tiap kelas setelah pembersihan.

Kelas	Latih	Validasi	Uji
andong	611	76	95
bajaj	96	17	15
becak	2.277	464	445
sepeda	162	15	28
bus	92	13	10
mobil	1.695	430	336
motor	1.828	444	453
orang	10.694	2.759	2.898
truk	45	4	8
Total	17.500	4.222	4.288

dan perbandingan model. Ketiga aspek tersebut ditetapkan seragam untuk seluruh varian agar perbandingan antar arsitektur berlangsung pada kondisi yang terkendali.

3.6.1 Konfigurasi Pelatihan

Pelatihan dilakukan pada lingkungan HPC dengan GPU NVIDIA Tesla V100-SXM2 32 GB yang dijelaskan pada Subbab 3.2, menggunakan implementasi resmi Ultralytics. Delapan varian model dilatih, yaitu YOLOv8 dan YOLO11 masing-masing pada empat ukuran, yaitu *nano* (n), *small* (s), *medium* (m), dan *large* (l).

Tiap varian diinisialisasi dari bobot *pretrained* pada dataset COCO, lalu dilatih ulang pada dataset kawasan Malioboro melalui mekanisme *transfer learning*. Seluruh varian dilatih dengan nilai *hyperparameter* yang identik agar perbandingan bersifat adil, sehingga perbedaan performa dapat diatribusikan pada arsitektur, bukan pada konfigurasi pelatihan. Karena *optimizer* diatur ke mode *auto*, Ultralytics menentukan jenis *optimizer*, yaitu AdamW, beserta *learning rate* dan *momentum* awalnya secara otomatis. Nilai *hyperparameter* selengkapnya dirangkum pada Tabel 3.7.

Inisialisasi dari bobot *pretrained* COCO adalah penerapan *transfer learning*, yaitu memanfaatkan pengetahuan visual umum yang telah dipelajari model dari dataset besar sebagai titik awal pelatihan. Dengan cara ini model tidak belajar dari nol, melainkan menyesuaikan fitur yang sudah matang ke objek khas Malioboro, sehingga pelatihan lebih cepat konvergen dan lebih hemat kebutuhan data. Mode *auto* pada Ultralytics melengkapi pendekatan ini dengan memilih *optimizer* AdamW serta menetapkan *learning rate* dan *momentum* awal secara otomatis berdasarkan karakteristik dataset, sehingga konfigurasi yang sama dapat diterapkan secara adil ke seluruh varian.

Penggunaan konfigurasi bawaan dan mode *auto* dipilih atas dua pertimbangan. Pertama, tujuan tahap ini adalah membandingkan dua keluarga arsitektur secara

adil, bukan mencari kombinasi *hyperparameter* optimal untuk tiap varian. Menyetel *hyperparameter* secara berbeda untuk tiap model justru akan mencampurkan efek arsitektur dengan efek konfigurasi, sehingga perbedaan performa tidak lagi dapat diatribusikan pada arsitektur. Prinsip memakai pengaturan standar bawaan implementasi resmi alih-alih menyetel manual tiap model juga diterapkan pada penelitian *benchmark* detektor pada dataset CCTV kendaraan, yang mengikuti konfigurasi yang direkomendasikan untuk tiap model demi perbandingan yang adil [12]. Penelitian ini menerapkan prinsip yang sama melalui mode *auto* Ultralytics, yang menetapkan *optimizer*, *learning rate*, dan *momentum* secara otomatis tanpa penyetelan manual. Kedua, performa deteksi cenderung tidak peka terhadap perubahan *hyperparameter* dalam rentang yang luas. Pada Faster R-CNN, perubahan bobot fungsi *loss* sebesar dua orde magnitudo hanya mengubah mAP sekitar 1% [20], sehingga penyetelan manual tiap *hyperparameter* tidak diharapkan mengubah urutan perbandingan antar arsitektur secara berarti. Optimasi terhadap satu *hyperparameter* tertentu tetap dilakukan, tetapi hanya pada model terbaik setelah model tersebut terpilih, sebagaimana diuraikan pada Subbab 3.8.

Tabel 3.7. Konfigurasi *hyperparameter* pelatihan.

Parameter	Nilai
Jumlah epoch	100
<i>Patience (early stopping)</i>	20
<i>Batch size</i>	otomatis (<i>auto-batch</i>)
Ukuran citra masukan	640×640
<i>Optimizer</i>	AdamW (dipilih otomatis, mode <i>auto</i>)
<i>Learning rate</i> awal	ditentukan otomatis oleh mode <i>auto</i>
Faktor <i>learning rate</i> akhir (lrf)	0,01
<i>Momentum</i>	ditentukan otomatis oleh mode <i>auto</i>
<i>Weight decay</i>	0,0005
<i>Warmup epoch</i>	3
Bobot awal	<i>pretrained COCO</i>
<i>Mixed precision (AMP)</i>	aktif
<i>Seed</i>	0 (deterministik)

3.6.2 Prapemrosesan dan Augmentasi

Sebelum masuk ke model, setiap citra diskala ke ukuran masukan 640×640 piksel. Augmentasi data dilakukan menggunakan pengaturan bawaan Ultralytics, yang diterapkan secara seragam pada seluruh varian agar perbandingan tidak dipengaruhi perbedaan perlakuan augmentasi. Penyeragaman ini menjaga agar satu-satunya faktor pembeda antar varian adalah arsitekturnya, bukan perlakuan data masukan.

Augmentasi yang aktif meliputi *mosaic* dengan peluang penuh yang dimatikan pada

10 epoch terakhir, pembalikan horizontal dengan peluang 0,5, penskalaan acak sebesar 0,5, pergeseran sebesar 0,1, penyesuaian warna pada ruang HSV, *RandAugment*, serta penghapusan acak (*random erasing*) sebesar 0,4. Beberapa augmentasi tidak diaktifkan, yaitu rotasi, pembalikan vertikal, *mixup*, *cutmix*, dan *copy-paste*, sesuai nilai bawaan Ultralytics.

Augmentasi bertujuan memperkaya keragaman data latih secara artifisial agar model lebih tahan terhadap variasi kondisi nyata. *Mosaic* menggabungkan empat citra menjadi satu sehingga model terbiasa dengan objek pada berbagai skala dan latar, dan dimatikan pada sepuluh epoch terakhir agar pelatihan ditutup pada citra yang utuh. Pembalikan horizontal dan penskalaan acak memperkenalkan variasi orientasi dan ukuran, sedangkan penyesuaian warna pada ruang HSV membuat model tahan terhadap perbedaan pencahayaan antara siang dan malam di Malioboro. Penghapusan acak menutup sebagian citra agar model belajar mengenali objek meskipun sebagian terhalang, kondisi yang sering muncul pada kerumunan padat. Rotasi dan pembalikan vertikal sengaja dimatikan karena objek pada citra CCTV selalu tegak, sehingga transformasi itu justru akan menghasilkan citra yang tidak realistis.

3.6.3 Skema Pelatihan dan Perbandingan

Kedelapan varian dilatih dengan konfigurasi identik selama 100 epoch, dengan *early stopping* yang menghentikan pelatihan apabila performa pada data validasi tidak membaik selama 20 epoch berturut-turut. Mekanisme ini memungkinkan pelatihan berhenti sebelum mencapai 100 epoch jika model telah konvergen. Tanpa *early stopping*, pelatihan yang terlalu lama berisiko membuat model menghafal data latih (*overfitting*) sehingga performanya pada data baru menurun, sehingga mekanisme ini sekaligus menjaga generalisasi dan menghemat waktu komputasi.

Performa kedelapan varian kemudian dibandingkan menggunakan metrik yang dijelaskan pada Subbab 3.7. Model dengan keseimbangan akurasi dan kecepatan terbaik dipilih untuk diterapkan pada sistem pemantauan. Hasil pelatihan dan perbandingan tiap varian disajikan pada Bab IV.

3.7 Evaluasi Model

Evaluasi bertujuan mengukur dan membandingkan performa kedelapan varian model serta memilih model terbaik untuk diterapkan. Subbab ini menjelaskan metrik yang digunakan beserta desain evaluasi dan kriteria pemilihan model. Kriteria pemilihan tidak hanya mempertimbangkan akurasi, tetapi juga kecepatan inferensi pada perangkat target, karena sistem harus berjalan *real-time* pada perangkat dengan sumber daya terbatas.

3.7.1 Metrik Evaluasi

Performa deteksi diukur menggunakan beberapa metrik yang definisinya telah diuraikan pada Bab II, yaitu *precision*, *recall*, F1, serta *mean Average Precision* (mAP). Metrik mAP dihitung pada dua ambang *Intersection over Union* (IoU), yaitu mAP50 pada ambang 0,5 dan mAP50-95 sebagai rata-rata pada ambang 0,5 hingga 0,95 dengan langkah 0,05. mAP50 menilai kemampuan deteksi dengan toleransi lokalisasi yang longgar, sedangkan mAP50-95 menuntut lokalisasi yang lebih tepat sehingga menjadi ukuran kualitas yang lebih ketat.

Skor F1 dihitung sebagai rata-rata harmonik *precision* dan *recall*, baik secara keseluruhan maupun per kelas. F1 per kelas digunakan untuk menganalisis pengaruh ketidakseimbangan jumlah *instance* terhadap performa tiap kelas, sesuai tujuan penelitian. Seluruh metrik akurasi dihitung pada himpunan uji yang berlabel manual agar pengukuran tidak bias.

Selain akurasi, kecepatan inferensi diukur dalam jumlah *frame* per detik (FPS) atau waktu pemrosesan per citra pada perangkat penerapan, yaitu laptop dengan GPU GTX 1650. Metrik ini menilai kelayakan model untuk pemantauan secara *real-time*.

3.7.2 Desain Evaluasi dan Pemilihan Model

Evaluasi dilakukan dengan membandingkan kedelapan varian model pada metrik di atas menggunakan himpunan uji. Himpunan validasi digunakan selama pelatihan untuk *early stopping*, sedangkan himpunan uji yang tidak pernah dilihat model selama pelatihan menjadi dasar perbandingan akhir. Pemisahan peran kedua himpunan ini mencegah kebocoran informasi, karena keputusan penghentian pelatihan tidak boleh diambil dari data yang sama dengan data penguji akhir.

Analisis tidak hanya melihat performa keseluruhan, tetapi juga performa tiap kelas melalui F1 per kelas. Pendekatan per kelas ini diperlukan karena mAP keseluruhan dapat tampak baik meskipun kelas minoritas berperforma buruk. Dengan demikian, F1 per kelas menjadi alat untuk menilai sejauh mana ketidakseimbangan kelas memengaruhi deteksi pada kelas mayoritas maupun minoritas.

Model terbaik dipilih berdasarkan keseimbangan antara kualitas deteksi dan kecepatan inferensi pada perangkat penerapan, bukan semata akurasi tertinggi, karena sistem harus berjalan secara *real-time* pada perangkat dengan sumber daya terbatas. Model terpilih kemudian dioptimasi lebih lanjut melalui penyetelan *hyperparameter* sebelum diterapkan, sebagaimana diuraikan pada Subbab 3.8. Hasil evaluasi dan perbandingan kedelapan varian disajikan pada Bab IV.

3.8 Penyetelan Hyperparameter Model Terpilih

Pemilihan model pada Subbab 3.7 menghasilkan satu varian terbaik yang menyeimbangkan kualitas deteksi dan kecepatan inferensi. Subbab ini menjelaskan upaya meningkatkan performa varian terpilih melalui penyetelan *hyperparameter*, beserta dasar pemilihan *hyperparameter* yang disetel, konfigurasi yang digunakan, dan strategi pengendaliannya. Penyetelan ini diposisikan sebagai langkah optimasi terhadap satu model terpilih, bukan pelatihan ulang seluruh varian, sehingga lingkungannya tetap terkendali.

Penyetelan diarahkan pada satu *hyperparameter* tunggal, yaitu ukuran citra masukan (*image size*), yang dinaikkan dari 640 menjadi 960 piksel. Pilihan ini didasarkan pada karakteristik kelemahan model hasil pelatihan awal. Hasil evaluasi menunjukkan *recall* yang rendah pada hampir seluruh kelas dan performa terlemah pada kelas sepeda, pola yang khas untuk objek berukuran kecil dan berjarak jauh dari kamera. Banyak objek seperti orang dan motor di kejauhan tidak terdeteksi. Menaikkan resolusi masukan menyasar kelemahan ini secara langsung, karena objek kecil terwakili oleh lebih banyak piksel pada resolusi yang lebih tinggi sehingga lebih mudah dikenali.

Dasar ini didukung oleh temuan empiris pada literatur deteksi objek. Hasil pada *benchmark* MIO-TCD menunjukkan bahwa menaikkan resolusi masukan dari 300 menjadi 512 piksel meningkatkan mAP sebesar 4%, dan tantangan terbesar pada dataset tersebut berasal dari kelas kendaraan kecil dan pejalan kaki yang mudah tertukar dengan kategori sepeda dan motor karena ukurannya yang kecil [35]. Pola kesulitan yang sama muncul pada dataset penelitian ini. Keterbatasan deteksi objek kecil juga diakui sejak arsitektur YOLO awal, yang kesulitan pada objek kecil yang muncul berkelompok karena penggunaan fitur yang relatif kasar akibat banyaknya lapisan *downsampling* [21]. Tinjauan terhadap masalah ketidakseimbangan pada deteksi objek juga menegaskan adanya kecondongan distribusi ke arah objek berukuran kecil yang memengaruhi performa deteksi [19].

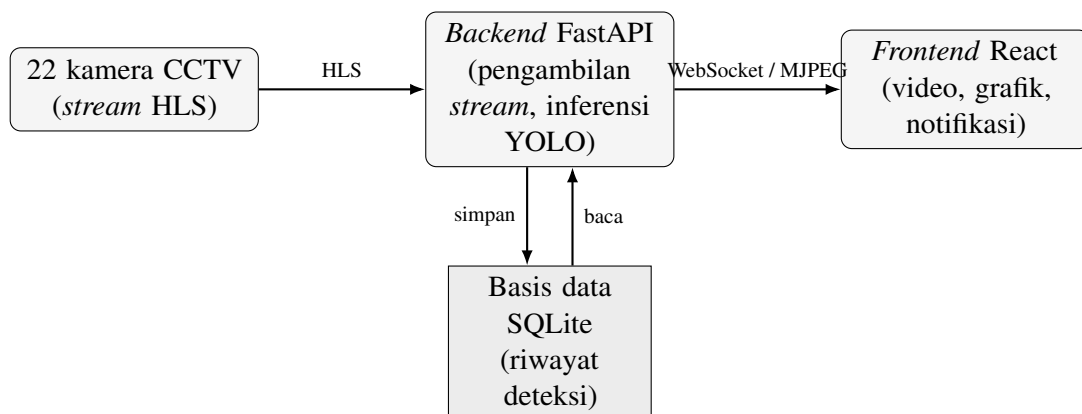
Penyetelan dilakukan dengan menjaga satu variabel berubah. Seluruh *hyperparameter* lain dipertahankan identik dengan konfigurasi pelatihan awal pada Tabel 3.7, dan model diinisialisasi dari bobot *pretrained* COCO yang sama, bukan dari bobot hasil pelatihan pada dataset Malioboro. Dengan cara ini, setiap perbedaan performa antara model pada resolusi 640 dan 960 dapat diatribusikan murni pada perubahan resolusi, bukan pada faktor lain. Hasil penyetelan beserta analisisnya, mencakup perbandingan metrik keseluruhan, F1 per kelas, serta biaya komputasi dan kecepatan inferensi, disajikan pada Bab IV.

3.9 Perancangan dan Implementasi Sistem

Model hasil pelatihan dan penyetelan diterapkan pada sebuah sistem pemantauan berbasis web yang mengolah *stream* CCTV kawasan Malioboro secara langsung. Subbab ini menjelaskan arsitektur sistem, mekanisme penanganan *stream* video dan penjadwalan inferensi, estimasi kepadatan beserta ambang notifikasi, dan visualisasi kepadatan pada antarmuka.

3.9.1 Arsitektur Sistem

Sistem terdiri atas dua bagian utama, yaitu *backend* dan *frontend*. Bagian *backend* dibangun menggunakan *framework* FastAPI dengan bahasa Python, dan bertanggung jawab atas pengambilan *stream* CCTV, penjalanan inferensi model, penyimpanan riwayat deteksi, serta pengiriman hasil ke antarmuka. Bagian *frontend* dibangun menggunakan pustaka React dengan *Vite* dan *Tailwind*, dan menampilkan video langsung beserta hasil deteksi, grafik kepadatan, dan panel notifikasi kepada pengguna. Keterhubungan antar komponen ini ditunjukkan pada Gambar 3.7.



Gambar 3.7. Arsitektur sistem pemantauan yang menghubungkan *stream* CCTV, *backend* FastAPI, basis data SQLite, dan *frontend* React.

Sumber data sistem adalah 22 kamera CCTV publik yang disediakan melalui laman resmi pemerintah Kota Yogyakarta dalam format *stream* HLS. Registri kamera, termasuk tautan *stream* dan konfigurasi ambang tiap kamera, disimpan dalam berkas konfigurasi. Riwayat deteksi disimpan dalam basis data SQLite yang dibuat otomatis saat sistem dijalankan, dan digunakan sebagai sumber data grafik kepadatan. Hasil deteksi terbaru dikirim ke antarmuka melalui mekanisme *WebSocket* agar tampilan dapat diperbarui tanpa pemuatan ulang halaman.

3.9.2 Penanganan *Stream* CCTV dan Penjadwalan Inferensi

Menjalankan inferensi secara terus-menerus pada 22 *stream* sekaligus tidak memungkinkan pada perangkat dengan satu GPU terbatas, sebagaimana ditunjukkan oleh

hasil kecepatan pada Bab IV. Tanpa pembedaan penanganan, beban 22 proses inferensi serentak akan melampaui kapasitas GPU tunggal dan membuat seluruh *stream* tersendat. Oleh karena itu, sistem membedakan penanganan antara kamera yang sedang dipilih pengguna dan kamera lainnya melalui dua mekanisme terpisah.

Kamera yang sedang dipilih ditangani oleh pemroses kamera aktif yang menjalankan dua utas, yaitu satu utas untuk membaca *stream* video dan satu utas untuk menjalankan inferensi pada interval tetap. Pemisahan ini bertujuan menjaga agar tampilan video tetap mulus sementara inferensi berjalan pada laju yang lebih lambat. Hasil deteksi ditampilkan dengan teknik *Motion JPEG* (MJPEG), yang memungkinkan sistem mengatur interval inferensi secara mandiri dan menampilkan kotak deteksi bersamaan dengan *frame* terbaru. Tanpa pemisahan ini, kotak deteksi akan tampak tertinggal dari *frame* yang sedang ditampilkan karena laju inferensi yang jauh lebih rendah daripada laju video.

Kamera yang tidak sedang dipilih ditangani oleh kumpulan penjadwal latar belakang yang menggilir kamera satu per satu secara berurutan dalam satu utas. Pada tiap giliran, sistem membuka *stream* satu kamera, mengambil satu *frame*, menjalankan inferensi, lalu menutup *stream* tersebut dan melanjutkan ke kamera berikutnya. Interval antar giliran dihitung agar tiap kamera mendapat giliran kira-kira sekali per 60 detik, dengan membagi jendela waktu 60 detik dengan jumlah kamera nonaktif. Mekanisme ini menjaga agar seluruh kamera tetap terpantau untuk keperluan deteksi anomali kepadatan, tanpa membebani GPU dengan 22 inferensi serentak. Prosedur penjadwalan latar belakang ini dirangkum pada Algoritma 2.

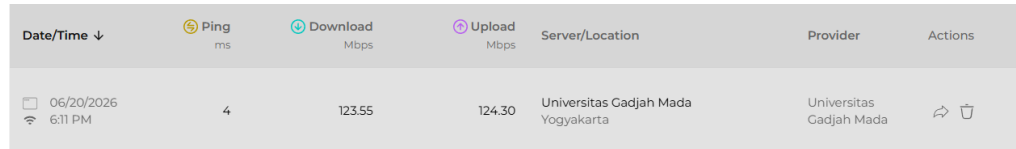
Algorithm 2 Penjadwalan inferensi latar belakang untuk kamera nonaktif.

Require: himpunan kamera nonaktif K , jendela waktu $W = 60$ detik, batas waktu $\tau = 10$ detik

- 1: $\Delta \leftarrow W/|K|$ $\triangleright$ interval antar giliran
- 2: **while** sistem berjalan **do**
- 3: **for** setiap kamera $k \in K$ **do**
- 4: buka *stream* kamera k
- 5: $f \leftarrow$ ambil satu *frame* dengan batas waktu τ
- 6: **if** f berhasil diambil sebelum τ **then**
- 7: jalankan inferensi pada f dan simpan hasilnya
- 8: **else**
- 9: catat status kamera k sebagai *timeout*
- 10: **end if**
- 11: tutup *stream* kamera k
- 12: tunggu hingga giliran berikutnya sesuai Δ
- 13: **end for**
- 14: **end while**

Pengukuran pada kondisi jaringan kampus menunjukkan rata-rata waktu satu siklus pemrosesan latar belakang per kamera sekitar 2,59 detik, sehingga satu putaran penuh

untuk 21 kamera nonaktif memakan waktu sekitar 54,3 detik, mendekati target rancangan 60 detik. Pengukuran ini dilakukan pada jaringan dengan kecepatan unduh 123 Mbps dan *ping* 4 milidetik ke server, sebagaimana ditunjukkan oleh hasil uji *Speedtest by Ookla* pada Gambar 3.8, sehingga durasi nyata pada kondisi jaringan lain dapat berbeda.



Date/Time ↓	Ping ms	Download Mbps	Upload Mbps	Server/Location	Provider	Actions
06/20/2026 6:11 PM	4	123.55	124.30	Universitas Gadjah Mada Yogyakarta	Universitas Gadjah Mada	↻ 🗑

Gambar 3.8. Hasil pengujian kecepatan jaringan dengan *Speedtest by Ookla* saat pengukuran.

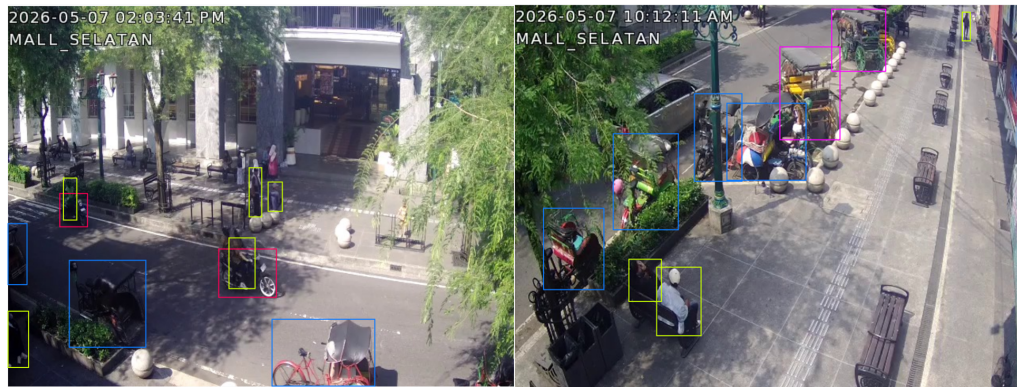
Pengujian pada kondisi nyata menemukan bahwa sebagian kamera publik dapat tidak aktif sewaktu-waktu. Pada implementasi awal, pembaca *stream* dirancang untuk mencoba menyambung ulang tanpa batas, sehingga satu kamera yang tidak aktif membuat satu siklus pemrosesan tertahan hingga sistem dimulai ulang. Karena penjadwal latar belakang berjalan berurutan dalam satu utas, penundaan ini menghentikan pemantauan seluruh kamera lain selama itu. Masalah ini diatasi dengan menambahkan batas waktu pengambilan *frame* sebesar 10 detik. Jika pengambilan melewati batas tersebut, proses dihentikan secara kooperatif dan giliran dilanjutkan ke kamera berikutnya, dengan status dicatat sebagai *timeout*. Setelah perbaikan, satu kamera yang tidak aktif hanya menambah sekitar 10 detik pada satu putaran, sehingga dampak satu kamera mati tidak menghentikan keseluruhan proses inferensi kamera yang tidak sedang dipilih.

3.9.3 Estimasi Kepadatan dan Ambang Notifikasi

Notifikasi kepadatan didasarkan pada konsep *Level of Service* (LOS) pejalan kaki, yang menyatakan tingkat kenyamanan suatu ruang pejalan kaki berdasarkan kepadatan orang per satuan luas [13]. Standar serupa juga diatur dalam pedoman teknis penyediaan prasarana pejalan kaki di Indonesia [1]. Penerapan LOS menuntut konversi dari jumlah orang yang terdeteksi pada citra menjadi kepadatan pada luas ruang nyata, yang dilakukan melalui transformasi homografi. Titik acuan dan jarak nyata di lapangan diperkirakan menggunakan *Google Earth*.

Transformasi homografi mensyaratkan sudut pandang kamera yang tetap, karena pemetaan antara bidang citra dan bidang nyata bergantung pada posisi kamera yang konsisten. Pada kenyataannya, sebagian kamera CCTV di kawasan ini memiliki sudut pandang yang tidak tetap, sebagian di antaranya dapat digerakkan, sebagaimana ditunjukkan pada Gambar 3.9 yang memperlihatkan kamera yang sama pada sudut yang berbeda. Oleh karena itu, kalibrasi LOS melalui homografi hanya diterapkan pada dua kamera yang sudut pandangnya relatif tetap sebagai contoh penerapan, sedangkan ambang untuk kamera lain ditentukan secara manual. Pembatasan ini sejalan dengan batasan

penelitian yang diuraikan pada Bab I.



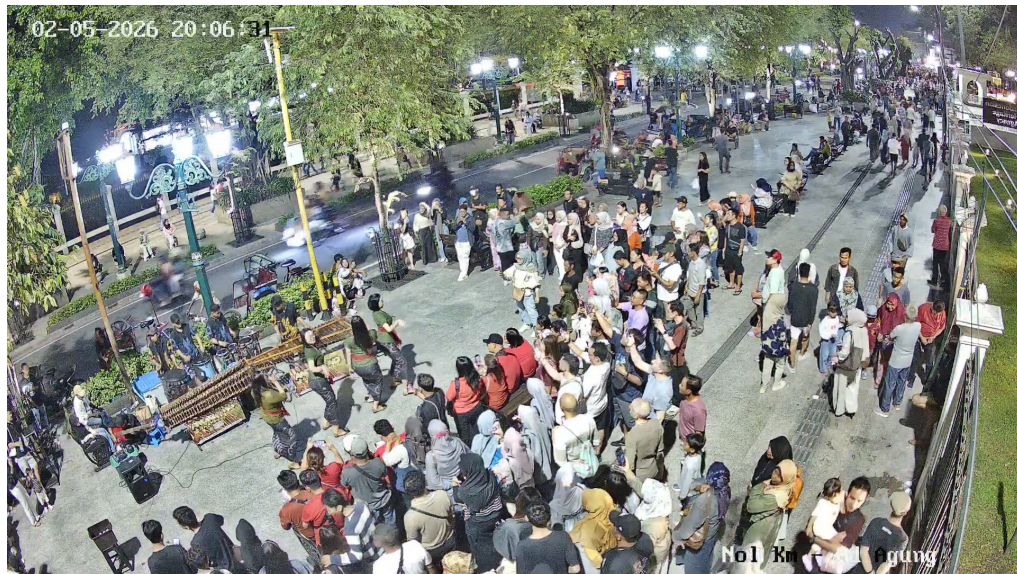
Gambar 3.9. Kamera CCTV yang sama pada dua sudut pandang berbeda

Pemilihan dua kamera tersebut sebagai titik kalibrasi LOS tidak semata karena sudut pandangnya relatif tetap. Kedua kamera juga mengarah ke segmen trotoar yang cukup luas dan terbuka, sehingga dimensi ruang nyatanya dapat diestimasi dengan andal melalui pengukuran pada *Google Earth* sebagai dasar transformasi homografi. Selain itu, kedua titik tersebut termasuk lokasi yang kerap menjadi pusat keramaian di koridor Malioboro, sehingga estimasi kepadatan di sana paling relevan untuk tujuan pemantauan dan pengendalian kerumunan.

Kamera pertama mengarah ke area pejalan kaki di seberang Gedung Agung, sebagaimana ditunjukkan pada Gambar 3.10. Estimasi dengan fitur *measurement* pada *Google Earth* menghasilkan luas area pejalan kaki yang tercakup kamera tersebut sekitar 169 m², sebagaimana terlihat pada Gambar 3.12. Luas inilah yang menjadi penyebut dalam mengubah jumlah orang yang terdeteksi menjadi kepadatan per meter persegi pada titik tersebut.

Kamera kedua mengarah ke area pejalan kaki Titik Nol yang menghadap ke utara, sebagaimana ditunjukkan pada Gambar 3.11. Pengukuran serupa pada *Google Earth* menghasilkan luas area pejalan kaki yang tercakup sekitar 149 m², sebagaimana terlihat pada Gambar 3.13. Dengan luas kedua zona yang telah diketahui, jumlah orang pada masing-masing kamera dapat diubah menjadi nilai kepadatan dan dipetakan ke tingkat pelayanan pada Tabel 2.3.

Alasan yang sama, yaitu sudut pandang yang dapat bergeser seperti diperlihatkan pada Gambar 3.9, juga mendasari keputusan untuk tidak menggunakan area pemantauan terbatas (*Region of Interest*). Poligon area yang digambar satu kali tidak lagi sahih setelah kamera bergeser, sehingga penghitungan objek dilakukan pada seluruh *frame*, bukan pada area terbatas tertentu. Penghitungan pada seluruh *frame* memang mengorbankan kemampuan membatasi zona, tetapi menjamin hasil tetap konsisten meskipun sudut pandang kamera berubah.



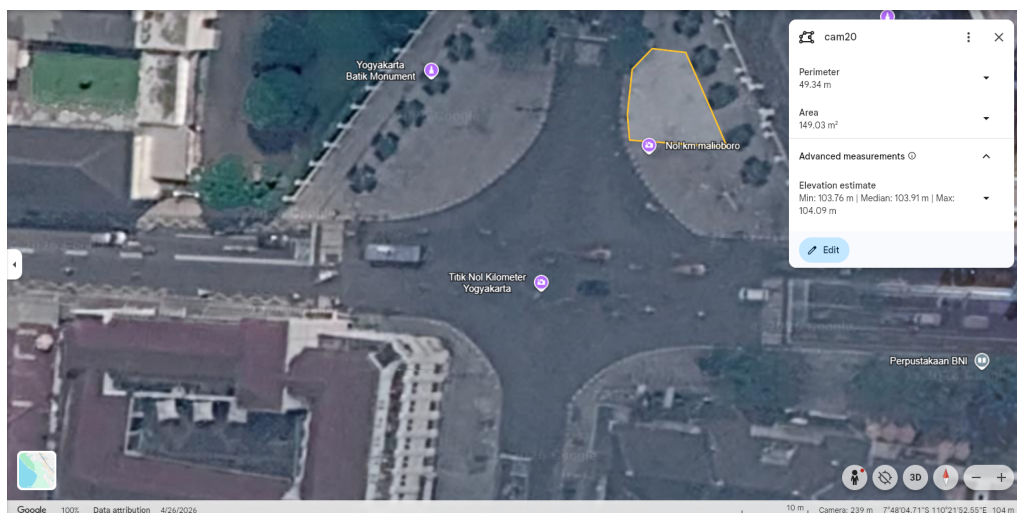
Gambar 3.10. Citra CCTV area pejalan kaki di seberang Gedung Agung.



Gambar 3.11. Citra CCTV area pejalan kaki Titik Nol yang menghadap ke utara.



Gambar 3.12. Pengukuran luas area pejalan kaki yang tercakup CCTV seberang Gedung Agung pada *Google Earth*.



Gambar 3.13. Pengukuran luas area pejalan kaki yang tercakup CCTV Titik Nol pada *Google Earth*.

Ambang notifikasi ditetapkan per kelas untuk tiap kamera, bukan sebagai satu ambang tunggal yang berlaku sama untuk seluruh kamera. Hal ini diperlukan karena tiap titik kamera mewakili ruang dengan kapasitas yang berbeda, sehingga jumlah objek yang menandakan kepadatan berlebih pun berbeda antar lokasi. Untuk dua kamera yang terkalibrasi LOS, ambang jumlah orang diturunkan dari kapasitas ruang hasil homografi. Untuk kamera lainnya, ambang ditentukan secara manual. Nilai ambang ini disimpan dalam berkas konfigurasi tiap kamera.

Agar notifikasi tidak terkirim berulang kali selama kondisi padat masih berlangsung, sistem menerapkan tenggang waktu antar notifikasi untuk kombinasi kamera dan jenis notifikasi yang sama. Tenggang waktu ditetapkan selama 7 menit, yang dinilai cukup untuk mencegah notifikasi yang berlebihan tanpa membuat sistem lambat menanggapi kondisi yang dapat berubah cepat. Jika kepadatan melonjak jauh melampaui ambang, tenggang waktu ini diabaikan agar kondisi yang lebih berbahaya tetap diberitahukan. Penetapan nilai tenggang waktu ini adalah pertimbangan rekayasa, bukan mengikuti standar tertentu.

Selain dikirim sebagai notifikasi, tiap peringatan yang melampaui ambang juga disimpan beserta citra CCTV pada saat peringatan terpicu dan kotak deteksi yang menyertainya, sehingga operator dapat meninjaunya kembali melalui halaman riwayat peringatan. Citra disimpan dalam bentuk asli tanpa anotasi, sedangkan kotak deteksi digambar ulang dari data koordinat yang tersimpan saat citra ditampilkan, agar penyimpanan tetap hemat tanpa menggandakan berkas gambar. Riwayat ini membuat peringatan tidak sekadar lewat sesaat, melainkan terdokumentasi sebagai bukti kondisi kepadatan yang dapat diperiksa ulang.

3.9.4 Visualisasi Kepadatan pada Dashboard

Antarmuka menyajikan grafik kepadatan yang merangkum riwayat deteksi tiap kamera. Metrik yang ditampilkan secara khusus adalah jumlah orang, bukan gabungan seluruh kelas objek, karena penggabungan orang dan kendaraan menjadi satu angka akan kehilangan makna mengingat skala kepadatan keduanya berbeda. Selain itu, dasar notifikasi dan tingkat pelayanan (LOS) pada penelitian ini khusus berlaku untuk pejalan kaki, sehingga grafik kepadatan pada *dashboard* difokuskan pada kelas orang agar selaras dengan dasar teori tersebut. Jumlah objek kelas kendaraan tetap tersimpan pada riwayat deteksi untuk kemungkinan pemanfaatan lebih lanjut, tetapi tidak ditampilkan pada grafik kepadatan saat ini.

Riwayat deteksi tidak disimpan per *frame*, melainkan diringkas setiap lima menit untuk tiap kamera. Pada tiap jendela lima menit, sistem mencatat dua besaran untuk kelas orang, yaitu rata-rata jumlah orang dari seluruh *frame* yang terkumpul sebagai gambaran kondisi tipikal, dan nilai puncak berupa jumlah orang tertinggi pada satu *frame* di dalam

jendela tersebut sebagai penanda lonjakan sesaat. Pemisahan ini diperlukan karena untuk tujuan keselamatan momen puncak kepadatan lebih penting daripada rata-rata yang dapat menutupi lonjakan singkat. Penyajian tren semacam ini menjadi representasi standar pada pemantauan kerumunan, sebagaimana ditunjukkan pada platform pemantauan transit yang menampilkan variasi jumlah orang sepanjang hari [16], dan membantu operator memahami kondisi kepadatan tanpa harus mengamati seluruh kamera secara manual yang melelahkan serta rawan kesalahan [17].

Pengguna dapat memilih dua rentang waktu, dan keduanya menampilkan baik rata-rata maupun puncak jumlah orang. Rentang harian menampilkan tren per jam pada satu tanggal terpilih, dengan nilai tiap jam dihitung dari ringkasan lima-menitan yang jatuh pada jam tersebut. Rentang mingguan menampilkan tren per hari selama tujuh hari kalender terakhir, dengan nilai tiap hari dihitung dari seluruh ringkasan lima-menitan pada hari tersebut. Seluruh pengelompokan jam dan hari menggunakan zona waktu setempat (WIB), dan grafik disajikan terpisah untuk tiap kamera karena tiap lokasi memiliki karakter kepadatan yang berbeda.

Granularitas dasar visualisasi ini adalah ringkasan lima menit, sehingga rata-rata maupun puncak per jam dan per hari dihitung dari ringkasan tersebut, bukan langsung dari tiap *frame*. Selain itu, rata-rata lima-menitan dihitung tanpa pembobotan waktu terhadap perbedaan laju pengambilan *frame* antara kamera aktif dan kamera latar belakang, sehingga nilainya lebih mencerminkan rata-rata atas sampel yang terkumpul daripada rata-rata atas durasi. Keterbatasan ini tidak memengaruhi fungsi pemantauan, tetapi perlu diperhatikan bila data digunakan untuk analisis kuantitatif lebih lanjut.

BAB IV

HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil penelitian beserta pembahasannya, disusun mengikuti urutan tujuan penelitian. Subbab 4.1 memaparkan hasil pembentukan dataset sebagai capaian tujuan pertama. Subbab 4.2 hingga 4.4 menjawab tujuan kedua melalui perbandingan delapan varian model, analisis kurva evaluasi, dan analisis performa per kelas. Subbab 4.5 dan 4.6 melaporkan hasil optimasi model terpilih melalui penyetelan resolusi. Subbab 4.7 menjawab tujuan ketiga melalui hasil implementasi dan pengujian sistem pemantauan, dan Subbab 4.8 menutup bab ini dengan sintesis temuan lintas subbab.

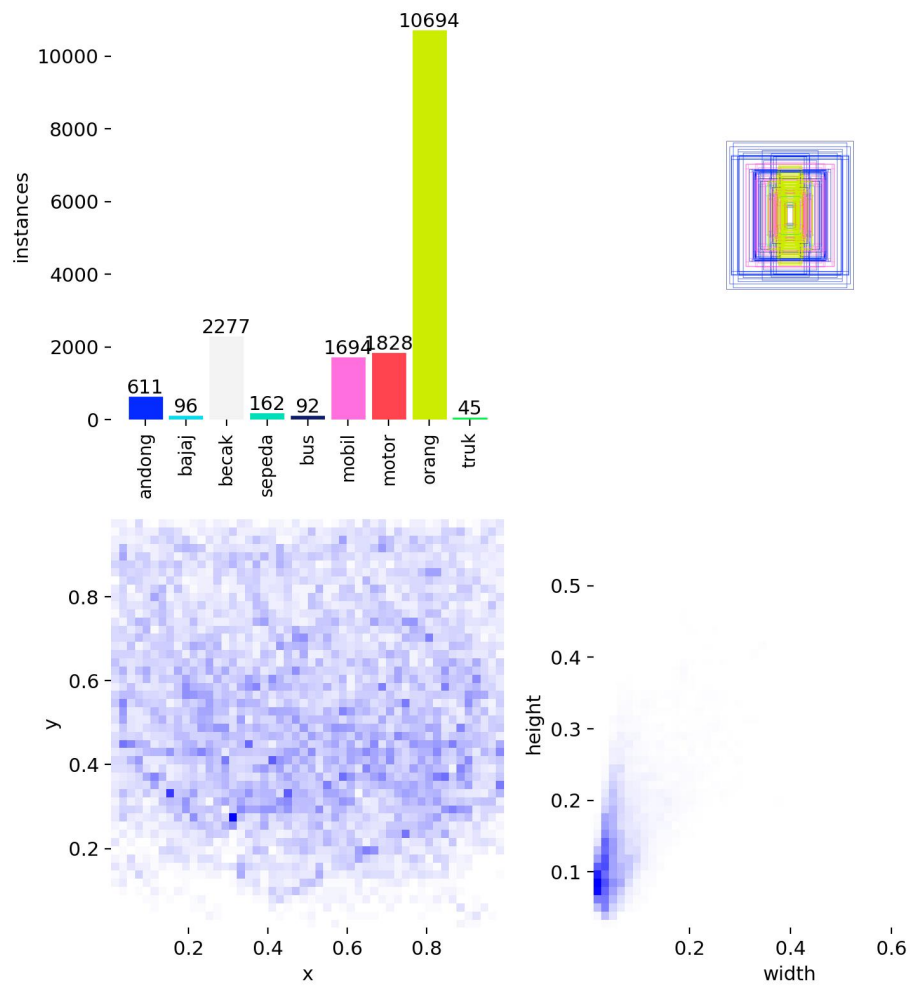
4.1 Hasil Pembentukan Dataset

Tujuan pertama, yaitu membentuk dataset deteksi objek untuk sembilan kelas khas kawasan Malioboro, tercapai melalui proses pengumpulan, pelabelan, dan pembersihan yang diuraikan pada Bab III. Dataset akhir memuat 1.581 citra yang terbagi ke dalam himpunan latih, validasi, dan uji, dengan distribusi jumlah *instance* tiap kelas sebagaimana telah disajikan pada Tabel 3.6. Sebaran spasial dan komposisi kelas pada himpunan latih divisualisasikan pada Gambar 4.14, yang memperlihatkan dua karakter penting dataset ini.

Karakter pertama adalah ketidakseimbangan kelas yang tajam, terlihat dari dominasi kelas orang dengan 10.694 *instance* dibanding kelas truk yang hanya 45 *instance*, yaitu rasio sekitar 237 berbanding 1. Karakter kedua adalah dominasi objek berukuran kecil, yang tampak pada panel sebaran lebar dan tinggi kotak di Gambar 4.14, dengan mayoritas kotak terpusat pada tinggi sekitar 0,1 dari tinggi citra. Kedua karakter ini menjadi tantangan utama yang tecermin pada hasil per kelas pada Subbab 4.4, dan menjadi dasar strategi penyetelan resolusi pada Subbab 4.5.

4.2 Hasil Pelatihan dan Perbandingan Delapan Varian

Delapan varian model, yaitu YOLOv8 dan YOLO11 pada empat ukuran (*nano*, *small*, *medium*, *large*), dilatih dengan konfigurasi identik sebagaimana diuraikan pada Subbab 3.6, lalu dievaluasi pada himpunan uji yang tidak pernah dilihat selama pelatihan. Perbandingan kedelapan varian ini menjadi langkah pertama untuk menjawab tujuan kedua penelitian, yaitu menentukan model deteksi yang paling sesuai untuk diterapkan pada sistem pemantauan. Subbab ini membandingkan performa akurasi dan efisiensi kedelapan varian secara berurutan, sebelum keduanya disandingkan sebagai dasar pemilihan model pada Subbab 4.2.3.



Gambar 4.14. Distribusi jumlah *instance* tiap kelas (kiri atas) serta sebaran posisi dan ukuran kotak pembatas pada himpunan latih.

4.2.1 Perbandingan Akurasi

Tabel 4.1 merangkum performa akurasi kedelapan varian pada himpunan uji, diurutkan menurun berdasarkan mAP50-95 sebagai metrik lokalisasi yang paling ketat. Seluruh nilai pada tabel ini dihitung pada himpunan uji agar perbandingan antar varian mencerminkan kemampuan generalisasi yang sesungguhnya, sedangkan himpunan validasi hanya digunakan selama pelatihan untuk *early stopping* dan tidak dipakai sebagai dasar perbandingan akhir. Susunan tabel dari mAP50-95 tertinggi ke terendah dipilih karena metrik ini paling ketat dalam menilai kualitas lokalisasi kotak pembatas, sehingga urutan ini menjadi acuan utama pembahasan pada subbab ini.

Tabel 4.1. Perbandingan akurasi delapan varian model pada himpunan uji.

Model	mAP50	mAP50-95	Precision	Recall	Rerata F1
yolo11l	0,6062	0,4012	0,7146	0,5438	0,5962
yolov8l	0,6175	0,3965	0,8355	0,5084	0,6222
yolo11m	0,6073	0,3861	0,7559	0,5227	0,6024
yolov8m	0,5875	0,3817	0,6341	0,5536	0,5796
yolov8s	0,5845	0,3730	0,6392	0,5247	0,5677
yolo11s	0,5717	0,3646	0,7114	0,4639	0,5488
yolov8n	0,5067	0,3122	0,5472	0,5193	0,5219
yolo11n	0,4698	0,2941	0,6039	0,4234	0,4877

Performa meningkat seiring bertambahnya ukuran model pada kedua keluarga, dengan varian *nano* menempati posisi terbawah dan varian *large* di posisi teratas. Tidak ada satu varian yang unggul pada seluruh metrik sekaligus. yolov8l mencapai mAP50 dan rerata F1 tertinggi, yaitu 0,6175 dan 0,6222, sedangkan yolo11l mencapai mAP50-95 tertinggi, yaitu 0,4012, yang berarti kotak prediksinya rata-rata paling presisi pada berbagai ambang IoU. Perbedaan antara dua varian *large* ini tergolong kecil pada akurasi, sehingga pertimbangan efisiensi turut menentukan pemilihan akhir.

4.2.2 Perbandingan Efisiensi Pelatihan dan Kecepatan Inferensi

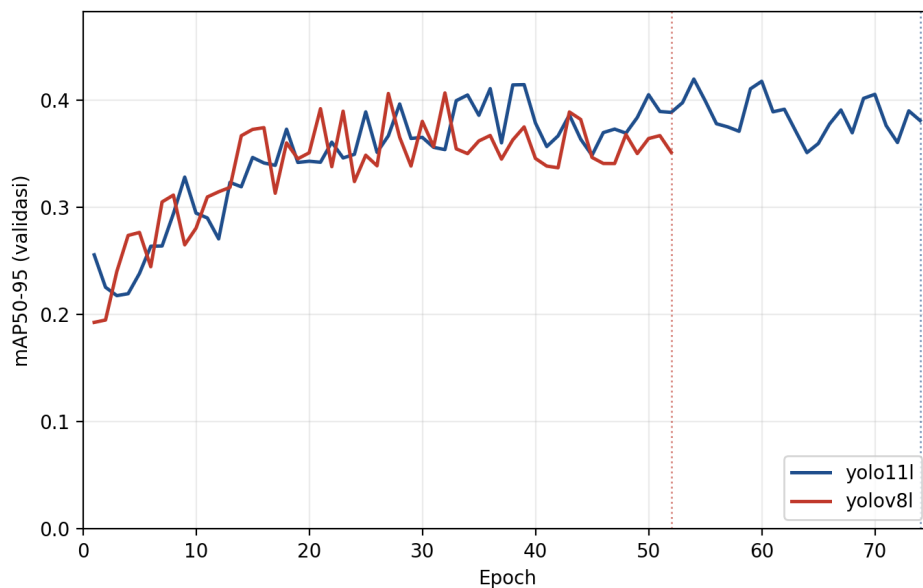
Tabel 4.2 merangkum sisi efisiensi tiap varian, mencakup jumlah epoch hingga konvergen, waktu pelatihan, ukuran model, serta kecepatan inferensi. Kecepatan pada tabel ini diukur pada GPU Tesla V100-SXM2 di lingkungan pelatihan, sehingga mewakili perbandingan relatif antar varian, bukan kecepatan pada perangkat penerapan yang dibahas pada Subbab 4.7. Jumlah epoch bervariasi karena *early stopping* menghentikan pelatihan saat performa validasi berhenti membaik.

Gambar 4.15 membandingkan pola konvergensi dua varian *large* sepanjang pelatihan, diplot dari nilai mAP50-95 pada data validasi tiap epoch. Kedua model

Tabel 4.2. Perbandingan efisiensi delapan varian model. Kecepatan diukur pada GPU Tesla V100.

Model	Epoch terlatih	Waktu latih (menit)	Ukuran (MB)	Waktu inferensi (ms/citra)	FPS (V100)
yolo11l	74	34,9	51,20	23,10	43,3
yolov8l	52	25,9	87,65	16,06	62,3
yolo11m	42	14,5	40,52	15,93	62,8
yolov8m	64	20,1	52,03	12,33	81,1
yolov8s	53	8,9	22,51	10,21	97,9
yolo11s	46	8,4	19,17	13,00	76,9
yolov8n	100	11,2	6,25	10,45	95,7
yolo11n	91	11,1	5,47	12,98	77,0

menunjukkan kenaikan tajam pada epoch-epoch awal yang kemudian disusul fluktuasi di sekitar nilai yang relatif stabil, menandakan model telah memanfaatkan sebagian besar kapasitas belajarnya. yolo11l berhenti pada epoch ke-74 dengan mAP50-95 validasi akhir di kisaran 0,38, sedikit di atas yolov8l yang berhenti pada epoch ke-52 dengan nilai akhir di kisaran 0,35, konsisten dengan keunggulan yolo11l pada metrik ini di Tabel 4.1.



Gambar 4.15. Perkembangan mAP50-95 pada data validasi sepanjang epoch untuk yolo11l dan yolov8l.

4.2.3 Pemilihan Model Terbaik

Hasil pada Tabel 4.1 dan 4.2 menunjukkan tidak ada satu varian yang unggul pada seluruh metrik sekaligus, sehingga pemilihan model terbaik memerlukan pertimbangan yang menyeimbangkan akurasi, ukuran, dan kecepatan, bukan sekadar memilih nilai

tertinggi pada satu kolom. Subbab ini menjelaskan pertimbangan tersebut secara eksplisit, dengan menyandingkan dua kandidat utama, yaitu yolo11l dan yolov8l, sebagai dua varian *large* yang sama-sama unggul dibanding varian lain pada keluarganya. Pemilihan ini dibuat secara jujur dengan menyadari yolov8l sedikit unggul pada mAP50 dan rerata F1 serta lebih cepat pada GPU V100, sehingga keputusan akhirnya tidak didasarkan pada satu keunggulan mutlak melainkan pada akumulasi beberapa pertimbangan yang saling melengkapi.

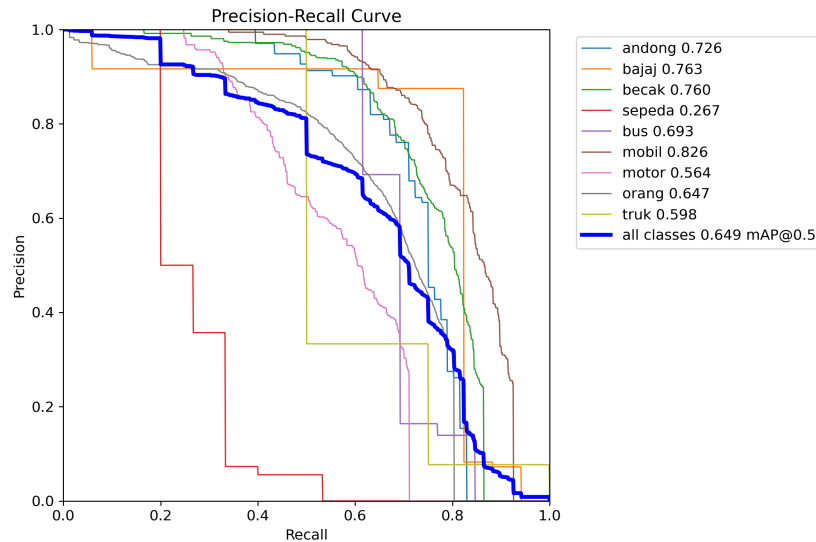
Pertimbangan pertama adalah lokalisasi. yolo11l unggul pada mAP50-95, yaitu metrik yang menuntut kotak pembatas presisi pada berbagai ambang IoU, yang menandakan lokalisasi objeknya lebih rapi dibanding yolov8l meski selisih mAP50 keduanya tipis. Pertimbangan kedua adalah ukuran model, yang menjadi penentu untuk perangkat penerapan dengan memori GPU terbatas. Ukuran yolo11l hanya 51,20 MB dibanding 87,65 MB milik yolov8l, yaitu sekitar 42% lebih ringkas, sehingga menyisakan ruang memori yang lebih lega untuk pemrosesan *stream* dan beban sistem lainnya pada laptop dengan GPU GTX 1650 bermemori 4 GB.

Pertimbangan ketiga muncul dari pengujian kecepatan inferensi pada perangkat penerapan sesungguhnya, yang justru membalikkan urutan pada GPU V100. Pada Subbab 4.7.1 ditunjukkan bahwa pada GTX 1650, yolo11l mencapai 9,73 FPS, lebih cepat dibanding yolov8l yang hanya 8,80 FPS, berkebalikan dengan hasil pada GPU V100 saat yolov8l lebih cepat. Atas dasar ketiga pertimbangan ini, yaitu lokalisasi yang lebih presisi, ukuran yang lebih ringkas, dan kecepatan yang ternyata lebih unggul pada perangkat target, yolo11l dipilih sebagai model yang diterapkan pada sistem. Pilihan ini kemudian diperkuat oleh hasil penyetelan pada Subbab 4.5, yang menunjukkan rerata F1 yolo11l naik menjadi 0,6739 setelah optimasi resolusi, melampaui capaian seluruh varian pada resolusi dasar termasuk yolov8l.

4.3 Analisis Kurva Evaluasi Model Terpilih

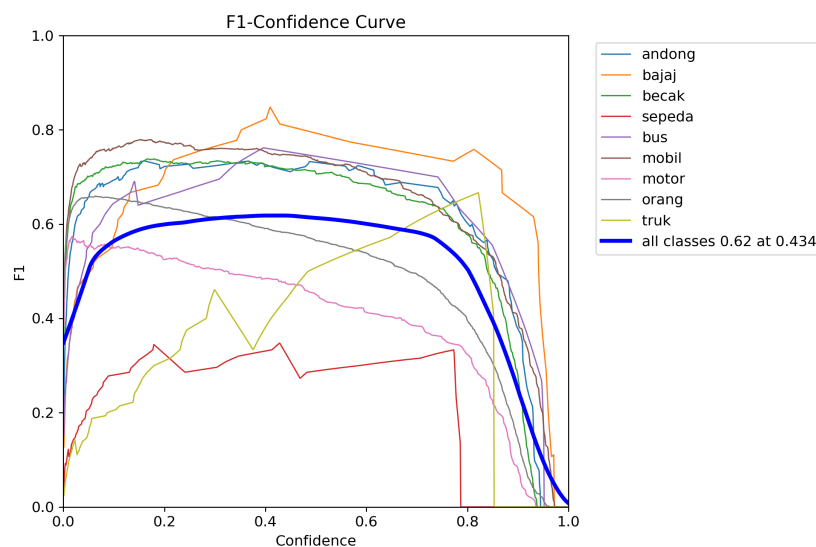
Selain metrik tunggal pada Tabel 4.1, perilaku model terpilih yolo11l pada berbagai ambang keyakinan dapat ditelaah melalui kurva evaluasi yang dihasilkan Ultralytics. Kurva *precision-recall* dan kurva F1 pada Gambar 4.16 dan 4.17 dihasilkan dari evaluasi otomatis pada himpunan validasi di akhir pelatihan, dan digunakan di sini untuk menelaah perilaku model pada berbagai ambang keyakinan serta menentukan titik operasi yang dipakai pada sistem. Hasil akhir model, yaitu performa pada data yang sama sekali tidak dilihat selama pelatihan, tetap diukur tersendiri pada himpunan uji dan disajikan melalui *confusion matrix* ternormalisasi pada Gambar 4.18 serta metrik per kelas pada Subbab 4.4.

Gambar 4.16 menunjukkan kurva *precision-recall* tiap kelas pada himpunan validasi. Kelas dengan kurva yang menjorok ke kanan atas, seperti mobil dan andong, mempertahankan *precision* tinggi hingga *recall* yang juga tinggi, menandakan model



Gambar 4.16. Kurva *precision-recall* model yolo11l untuk seluruh kelas pada himpunan validasi.

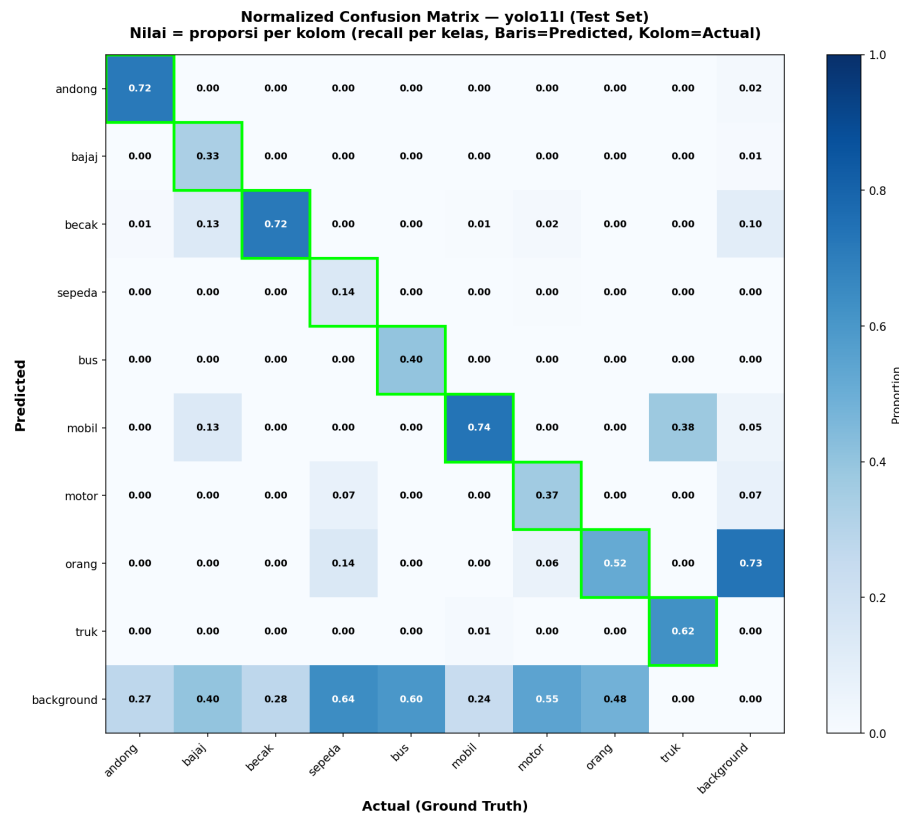
mengenalinya dengan keyakinan yang konsisten. Sebaliknya, kelas seperti sepeda menunjukkan kurva yang jatuh tajam segera setelah *recall* kecil, menandakan model hanya mampu mengenali sebagian kecil objek sepeda sebelum *precision*-nya merosot. Pola ini sejalan dengan hasil pada himpunan uji yang dibahas pada Subbab 4.4, meski kedua himpunan berbeda.



Gambar 4.17. Kurva *F1-score* terhadap ambang keyakinan model yolo11l untuk seluruh kelas pada himpunan validasi.

Gambar 4.17 memperlihatkan F1 pada himpunan validasi sebagai fungsi ambang keyakinan, yang berguna untuk menentukan titik operasi model. Puncak kurva rerata seluruh kelas menunjukkan ambang keyakinan yang mengoptimalkan F1 secara keseluruhan, dan ambang inilah yang dipakai sebagai titik operasi pada sistem yang

diterapkan. Lebar puncak yang relatif landai menandakan performa model tidak terlalu sensitif terhadap pemilihan ambang di sekitar titik optimalnya, sehingga sistem tetap stabil meski ambang digeser sedikit untuk menyesuaikan kebutuhan operasional. Metrik akhir yang dilaporkan pada Tabel 4.3 tetap dihitung pada himpunan uji menggunakan ambang ini, bukan diambil langsung dari kurva validasi.



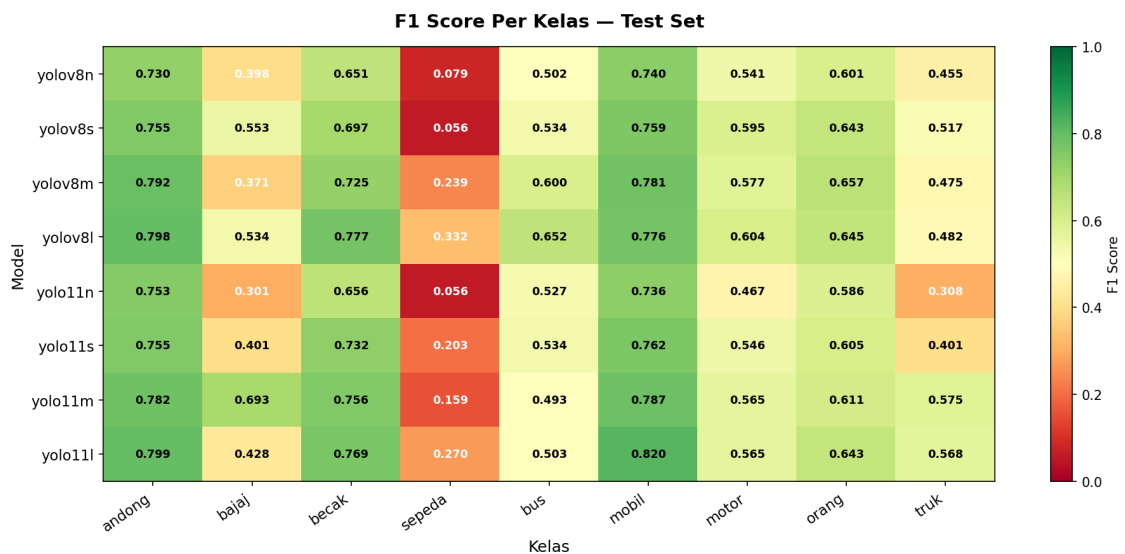
Gambar 4.18. *Confusion matrix* ternormalisasi model yolo11l pada himpunan uji.

Confusion matrix ternormalisasi pada Gambar 4.18 memetakan ke kelas apa suatu objek paling sering salah diklasifikasikan, atau apakah ia gagal terdeteksi sama sekali (kolom *background*). Diagonal utama yang gelap pada kelas mobil, andong, dan becak menegaskan deteksi yang andal pada kelas-kelas tersebut. Sebaliknya, baris kelas sepeda dan motor menunjukkan proporsi besar yang jatuh ke kolom *background*, berarti kesalahan dominan pada kedua kelas ini adalah objek yang gagal terdeteksi, bukan tertukar ke kelas lain. Pengamatan ini konsisten dengan pola *recall* rendah yang dibahas lebih rinci pada Subbab 4.4.

4.4 Analisis Performa per Kelas dan Pengaruh Ketidakseimbangan

Performa keseluruhan dan kurva evaluasi pada subbab sebelumnya belum cukup menggambarkan perilaku model pada tiap kelas, terutama pada dataset yang timpang seperti ini. Gambar 4.19 memetakan F1 tiap kelas untuk kedelapan varian, dan memperlihatkan pola yang konsisten lintas model, yaitu beberapa kelas selalu

mudah dideteksi sementara kelas lain selalu sulit. Tabel 4.3 merinci performa per kelas untuk model terpilih yolo11l, disandingkan dengan jumlah *instance* latih tiap kelas agar pengaruh ketidakseimbangan dapat ditelaah.



Gambar 4.19. F1 per kelas pada himpunan uji untuk kedelapan varian model.

Tabel 4.3. Performa per kelas model terpilih yolo11l pada himpunan uji, disandingkan jumlah *instance* latih.

Kelas	Instans latih	Precision	Recall	F1
mobil	1.694	0,889	0,761	0,820
andong	611	0,889	0,726	0,799
becak	2.277	0,816	0,727	0,769
orang	10.694	0,804	0,536	0,643
truk	45	0,458	0,750	0,568
motor	1.828	0,886	0,415	0,565
bus	92	0,678	0,400	0,503
bajaj	96	0,460	0,400	0,428
sepeda	162	0,553	0,179	0,270

Pola yang paling menonjol adalah rendahnya *recall* pada banyak kelas, sejalan dengan pengamatan pada *confusion matrix* di Subbab 4.3. Kelas motor, misalnya, memiliki *precision* 0,886 tetapi *recall* hanya 0,415, dan kelas orang memiliki *recall* 0,536 yang berarti hampir separuh objek orang tidak terdeteksi. Pola ini menandakan kesalahan dominan model adalah objek yang terlewat, bukan deteksi keliru, dan objek yang terlewat umumnya berukuran kecil atau berada jauh dari kamera.

Pengaruh ketidakseimbangan kelas terlihat, tetapi tidak bersifat tunggal. Kelas dengan performa terbaik adalah mobil, andong, dan becak dengan F1 di atas 0,76, yaitu

kelas yang berisi objek berukuran sedang hingga besar dengan bentuk yang khas dan jumlah *instance* memadai. Sebaliknya, kelas terlemah adalah sepeda dengan F1 hanya 0,270, diikuti bajaj dan bus, yaitu kelas dengan *instance* paling sedikit. Akan tetapi, hubungan ini tidak monoton. Kelas truk yang hanya memiliki 45 *instance*, paling sedikit di antara semua kelas, justru mencapai F1 0,568 yang melampaui sepeda dengan 162 *instance*.

Anomali ini menunjukkan bahwa ukuran dan kekhasan visual objek turut menentukan, di samping jumlah *instance*. Truk berukuran besar dan berpenampilan khas sehingga lebih mudah dipelajari meski contohnya sedikit, sedangkan sepeda berukuran kecil dan mudah tertukar dengan motor sehingga tetap sulit dikenali walau contohnya lebih banyak. Dengan demikian, ketidakseimbangan kelas adalah faktor yang berpengaruh, tetapi efeknya dimediasi oleh ukuran objek dan tingkat kemiripannya dengan kelas lain. Temuan inilah yang mendasari strategi penyetelan melalui peningkatan resolusi masukan, yang dibahas pada subbab berikutnya.

4.5 Hasil Penyetelan Hyperparameter Model Terpilih

Model terpilih yolo11l dioptimasi lebih lanjut melalui penyetelan resolusi citra masukan dari 640 menjadi 960 piksel, dengan dasar dan strategi sebagaimana diuraikan pada Subbab 3.8. Penyetelan ini diarahkan secara khusus pada kelemahan yang telah teridentifikasi pada Subbab 4.4, yaitu *recall* rendah pada objek berukuran kecil, sehingga hasil yang dipaparkan pada subbab ini sekaligus menjadi pengujian terhadap hipotesis tersebut. Model hasil penyetelan berhenti pada epoch ke-79 akibat *early stopping*, sebanding dengan model awal yang berhenti pada epoch ke-74, sehingga keduanya sama-sama mencapai konvergensi sebelum batas 100 epoch dan perbandingan performa keduanya tetap adil.

Tabel 4.4 membandingkan performa keseluruhan pada himpunan uji antara konfigurasi resolusi 640 dan 960. Seluruh metrik meningkat setelah resolusi dinaikkan, tanpa ada satu pun yang menurun, sebuah hasil yang konsisten dan tidak ambigu untuk menilai efektivitas penyetelan ini. Konsistensi kenaikan pada seluruh metrik sekaligus, alih-alih kenaikan pada satu metrik dengan kompensasi penurunan pada metrik lain, menunjukkan bahwa resolusi yang lebih tinggi memang memperbaiki kualitas deteksi secara menyeluruh, bukan sekadar menggeser titik seimbang antara *precision* dan *recall*.

Tabel 4.4. Perbandingan performa yolo11l pada resolusi 640 dan 960 (himpunan uji).

Konfigurasi	mAP50	mAP50-95	Precision	Recall	Rerata F1
Resolusi 640 (awal)	0,6062	0,4012	0,7146	0,5438	0,5962
Resolusi 960 (setelan)	0,6497	0,4413	0,7575	0,6217	0,6739
Selisih	+0,0435	+0,0401	+0,0429	+0,0779	+0,0777

Nilai mAP50-95 naik 0,0401 atau sekitar empat poin persentase, peningkatan yang besar untuk satu perubahan *hyperparameter* tunggal. Kenaikan terbesar terjadi pada *recall*, yaitu 0,0779, sesuai hipotesis bahwa resolusi lebih tinggi membuat lebih banyak objek kecil berhasil terdeteksi alih-alih terlewat, sebagaimana terlihat dari banyaknya kasus FN pada Tabel 4.3. Kenaikan *precision* yang menyertai *recall*, yaitu sebesar 0,0429, menunjukkan bahwa perbaikan ini tidak diperoleh dengan mengorbankan ketelitian deteksi, melainkan model benar-benar mengenali lebih banyak objek dengan tingkat kesalahan yang tetap terkendali.

Biaya komputasi turut meningkat sejalan dengan perbaikan akurasi ini. Waktu pelatihan naik dari 34,9 menit menjadi 84,2 menit akibat ukuran citra yang lebih besar, sedangkan ukuran model praktis tidak berubah pada 51,25 MB karena arsitektur tetap sama. Biaya pelatihan ini bersifat sekali jalan dan tidak menjadi beban berulang saat sistem dioperasikan. Pada GPU V100, kecepatan inferensi turun dari 23,10 ms (43,3 FPS) pada resolusi 640 menjadi 25,46 ms (39,3 FPS) pada resolusi 960, penurunan yang tergolong kecil mengingat kenaikan resolusi masukannya, dan masih jauh di atas kebutuhan *real-time*. Kecepatan pada perangkat penerapan yang sebenarnya dibahas pada Subbab 4.7.

4.6 Analisis Performa per Kelas Setelah Penyetelan

Kenaikan metrik keseluruhan pada Subbab 4.5 belum menjelaskan kelas mana yang paling berkontribusi terhadap perbaikan tersebut, sehingga diperlukan analisis yang lebih terperinci pada tingkat kelas. Tabel 4.5 merinci perbandingan F1 per kelas antara resolusi 640 dan 960, disusun untuk menelaah kelas mana yang paling diuntungkan oleh penyetelan resolusi dan kelas mana yang relatif tidak terpengaruh. Perbandingan pada tingkat kelas ini juga berfungsi memverifikasi apakah penyetelan benar-benar menyasar kelemahan yang telah teridentifikasi pada Subbab 4.4, yaitu kelas berukuran kecil dengan *recall* rendah.

Perbaikan paling jelas terlihat pada kelas yang sebelumnya paling lemah. Kelas sepeda naik 0,202 poin F1, sedangkan kelas bajaj naik 0,232 poin, peningkatan terbesar di antara seluruh kelas. Kedua kelas ini berisi objek berukuran kecil hingga sedang dengan jumlah *instance* sedikit, sehingga resolusi yang lebih tinggi memberi lebih banyak detail visual untuk dikenali. Kelas bus dan motor juga naik signifikan, masing-masing 0,117 dan 0,056 poin.

Sebaliknya, kelas andong dan mobil yang sudah terdeteksi baik pada resolusi 640 justru turun tipis, masing-masing 0,014 dan 0,006 poin. Penurunan yang sangat kecil ini konsisten dengan dasar teoretis bahwa kenaikan resolusi terutama membantu objek kecil dan nyaris tidak berpengaruh, atau berpotensi sedikit merugikan, pada objek yang sudah berukuran besar dan jelas. Pola menyeluruh ini menunjukkan penyetelan resolusi berhasil

Tabel 4.5. Perbandingan F1 per kelas yolo11l pada resolusi 640 dan 960 (himpunan uji).

Kelas	F1 (640)	F1 (960)	Selisih
andong	0,799	0,785	−0,014
bajaj	0,428	0,660	+0,232
becak	0,769	0,814	+0,045
sepeda	0,270	0,472	+0,202
bus	0,503	0,620	+0,117
mobil	0,820	0,814	−0,006
motor	0,565	0,621	+0,056
orang	0,643	0,654	+0,011
truk	0,568	0,625	+0,057
Rerata	0,596	0,674	+0,078

menyasar tepat kelemahan yang teridentifikasi pada Subbab 4.4, yaitu objek kecil yang sebelumnya banyak terlewat, tanpa mengorbankan performa pada kelas yang sudah baik.

4.7 Hasil Implementasi dan Pengujian Sistem

Model terpilih yolo11l diterapkan pada sistem pemantauan berbasis web yang dirancang pada Subbab 3.9, sebagai wujud capaian tujuan ketiga penelitian ini. Sistem tersebut diuji untuk memastikan dua hal, yaitu apakah kecepatan inferensinya memadai pada perangkat penerapan yang sesungguhnya, dan apakah seluruh komponen antarmuka berfungsi sebagaimana dirancang pada Bab III. Subbab ini melaporkan hasil kedua pengujian tersebut secara berurutan, dimulai dari kecepatan inferensi karena angka itu menentukan apakah sistem layak disebut *real-time*, dilanjutkan dengan bukti fungsional berupa tangkapan layar antarmuka yang sedang berjalan pada kondisi nyata.

4.7.1 Kecepatan Inferensi pada Perangkat Penerapan

Model terpilih yolo11l, beserta tujuh varian pembanding dan satu varian hasil penyetelan, diuji kecepatan inferensinya pada perangkat penerapan yang sebenarnya, yaitu laptop dengan GPU NVIDIA GeForce GTX 1650 bermemori 4 GB sebagaimana dirinci pada Tabel 3.1. Pengujian ini penting karena kecepatan pada Tabel 4.2 dan Subbab 4.5 diukur pada GPU Tesla V100 di lingkungan pelatihan, yang jauh lebih bertenaga dan tidak mewakili kondisi penerapan sesungguhnya. Setiap model diukur dengan merata-ratakan seratus kali inferensi berturut-turut saat sistem berjalan, sehingga angka yang dilaporkan mencerminkan kondisi pemakaian nyata, bukan pengukuran sekali jalan yang rentan terhadap variasi sesaat. Tabel 4.6 merangkum hasil pengukuran tersebut.

Pola kecepatan pada GTX 1650 secara umum sejalan dengan pola pada GPU V100, yaitu varian *nano* tercepat dan varian *large* terlambat pada tiap keluarga, karena kecepatan

Tabel 4.6. Kecepatan inferensi delapan varian model dan model hasil penyetelan pada GPU GTX 1650 (rata-rata 100 inferensi).

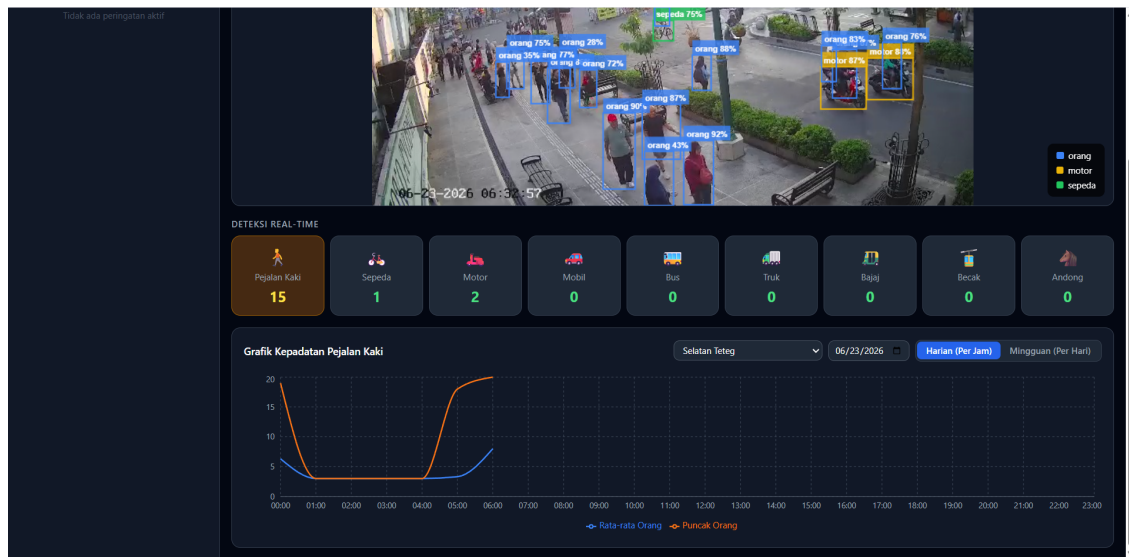
Model	Resolusi	Rerata (ms)	Std. (ms)	FPS
yolov8n	640	68,56	59,44	14,59
yolo11n	640	66,45	48,87	15,05
yolov8s	640	94,85	73,01	10,54
yolo11s	640	91,22	57,10	10,96
yolov8m	640	104,83	91,60	9,54
yolo11m	640	99,96	90,14	10,00
yolov8l	640	113,64	97,78	8,80
yolo11l	640	102,76	88,55	9,73
yolo11l (setelan)	960	112,47	90,98	8,89

inferensi memang berkorelasi dengan jumlah parameter model pada perangkat keras apa pun. Akan tetapi, satu temuan penting muncul khusus pada perangkat penerapan ini, yaitu urutan kecepatan antara dua model *large* berbalik dibanding pada V100. Pada GTX 1650, yolo11l mencapai 102,76 ms (9,73 FPS), lebih cepat dibanding yolov8l yang mencapai 113,64 ms (8,80 FPS), padahal pada Tabel 4.2 yolov8l justru lebih cepat pada V100.

Pembalikan ini diduga bersumber dari perbedaan karakteristik kedua GPU. GPU V100 memiliki banyak inti pemrosesan paralel dan memori yang besar, sehingga ukuran parameter yolov8l yang lebih besar tidak banyak menghambat kecepatannya pada perangkat tersebut. GTX 1650 memiliki kapasitas pemrosesan dan lebar pita memori yang jauh lebih terbatas, sehingga jumlah parameter yang harus dipindahkan dan diproses pada tiap inferensi menjadi faktor pembatas yang lebih dominan, dan pada kondisi ini ukuran yolo11l yang lebih ringkas (51,20 MB dibanding 87,65 MB) memberi keuntungan nyata. Temuan ini memperkuat keputusan pemilihan model pada Subbab 4.2.3, karena pertimbangan ukuran yang semula didasarkan pada keterbatasan memori kini terbukti juga relevan untuk kecepatan pada perangkat target, bukan sekadar pertimbangan keamanan memori semata.

Penyetelan resolusi ke 960 piksel pada model terpilih menurunkan kecepatan dari 102,76 ms menjadi 112,47 ms, atau penurunan FPS dari 9,73 menjadi 8,89, sekitar 8,6%. Penurunan ini tergolong kecil dibanding kenaikan resolusi yang diterapkan, dan kecepatan akhirnya masih berada pada kisaran yang sebanding dengan varian *medium* pada resolusi dasar. Standar deviasi yang relatif besar pada seluruh pengukuran, yang mendekati atau bahkan melampaui nilai rata-ratanya, menunjukkan kecepatan inferensi pada laptop berfluktuasi cukup besar antar *frame*, kemungkinan dipengaruhi proses latar belakang sistem operasi dan *thread* lain yang berjalan bersamaan, sehingga angka rata-rata sebaiknya dibaca sebagai estimasi tipikal dan bukan jaminan kecepatan yang tetap pada

Gambar 4.21 menunjukkan grafik kepadatan harian untuk kamera Selatan Teteg pada tanggal pengujian.



Gambar 4.21. Grafik kepadatan pejalan kaki harian (per jam) untuk kamera Selatan Teteg, menampilkan rata-rata dan puncak jumlah orang.

Gambar 4.21 memperlihatkan dua garis sesuai rancangan pada Subbab 3.9, yaitu rata-rata dan puncak jumlah orang per jam dalam zona waktu WIB. Kedua garis menunjukkan pola kepadatan rendah pada dini hari sebelum pukul 04.00 WIB, lalu naik tajam menjelang pagi, sesuai dengan karakter kawasan wisata yang ramai pada jam-jam aktif. Panel deteksi langsung pada bagian atas gambar yang sama menunjukkan kondisi pukul 06.32 WIB dengan lima belas orang dan beberapa kendaraan terdeteksi, konsisten dengan tren kenaikan yang mulai terlihat pada grafik di jam tersebut.

Riwayat peringatan diuji dengan membuka halaman riwayat dan memeriksa detail salah satu peringatan yang tersimpan, sebagaimana ditunjukkan pada Gambar 4.22. Pengujian ini memverifikasi bahwa fitur yang dirancang pada Subbab 3.9, yaitu penyimpanan bukti citra dan kotak deteksi setiap kali peringatan terpicu, benar-benar tersimpan dan dapat ditelusuri kembali oleh operator. Detail yang ditampilkan mencakup kamera asal, waktu dalam WIB, jenis peringatan, nilai pemicu, dan deskripsi singkat, seluruhnya konsisten dengan data yang tercatat pada panel peringatan kamera di Gambar 4.20.

Citra pada Gambar 4.22 menunjukkan kotak deteksi tergambar tepat di atas citra mentah yang tersimpan, sesuai rancangan pada Subbab 3.9 bahwa citra disimpan tanpa anotasi dan kotak deteksi digambar ulang dari data koordinat saat ditampilkan. Keberhasilan rekonstruksi kotak deteksi ini pada halaman riwayat membuktikan bahwa pendekatan penyimpanan hemat tersebut tidak mengorbankan kelengkapan bukti visual yang dapat ditinjau ulang oleh operator. Dengan demikian, ketiga pengujian pada subbab ini, yaitu antarmuka deteksi langsung, grafik kepadatan, dan riwayat peringatan,

diuntungkan oleh penyetelan resolusi. Hubungan ini menunjukkan bahwa strategi penyetelan yang dipilih, yaitu menaikkan resolusi masukan, secara tepat menyasar akar permasalahan yang teridentifikasi dari analisis sebelumnya, bukan perbaikan yang kebetulan.

Ketidakseimbangan kelas terbukti berpengaruh terhadap performa, tetapi penelitian ini juga menemukan bahwa pengaruh tersebut tidak berdiri sendiri. Anomali truk yang berperforma lebih baik daripada sepeda meski memiliki *instance* jauh lebih sedikit menunjukkan ukuran objek dan kekhasan visual turut memoderasi pengaruh ketidakseimbangan jumlah data. Temuan ini melengkapi pemahaman tentang ketidakseimbangan kelas pada deteksi objek multikelas yang heterogen secara visual, sebagaimana dibahas pada Subbab 2.2.4.

Hasil implementasi pada Subbab 4.7 melengkapi rangkaian temuan ini dengan menunjukkan bahwa pertimbangan ukuran model yang mendasari pemilihan yolo11l pada Subbab 4.2.3 terbukti relevan di luar sekadar keamanan memori. Pengujian kecepatan pada GTX 1650 menunjukkan yolo11l justru lebih cepat dibanding yolov8l pada perangkat penerapan, berkebalikan dengan urutan pada GPU V100, sehingga keunggulan ukuran yang ringkas memberi manfaat ganda berupa keamanan memori dan kecepatan yang lebih baik pada kondisi perangkat terbatas. Pengujian fungsional pada antarmuka, grafik kepadatan, dan riwayat peringatan turut menunjukkan bahwa rancangan sistem pada Bab III berhasil diwujudkan secara nyata, dengan notifikasi yang terpicu sesuai ambang dan kotak deteksi yang berhasil direkonstruksi dari data tersimpan. Secara keseluruhan, ketiga tujuan penelitian ini saling terkait erat, yaitu dataset yang terbentuk pada tujuan pertama menjadi dasar perbandingan model pada tujuan kedua, dan model yang terpilih serta dioptimasi pada tujuan kedua terbukti layak diterapkan pada sistem yang menjawab tujuan ketiga, baik dari sisi akurasi maupun kecepatan pada perangkat target.

BAB V

KESIMPULAN DAN SARAN

Bab ini menutup naskah dengan menyajikan kesimpulan penelitian dan saran untuk pengembangan lanjutan. Subbab 5.1 merangkum jawaban atas tiga rumusan masalah yang telah ditetapkan pada Bab I, disusun berurutan mengikuti alur dari ketersediaan data hingga penerapan sistem. Subbab 5.2 menguraikan arah pengembangan lanjutan yang diturunkan dari batasan penelitian dan keterbatasan yang teridentifikasi sepanjang proses penelitian.

5.1 Kesimpulan

Penelitian ini menjawab tiga rumusan masalah yang ditetapkan pada Bab I melalui pembentukan dataset, perbandingan dan penyetelan model deteksi, serta implementasi sistem pemantauan berbasis web. Ketiga jawaban tersebut diuraikan secara berurutan pada subbab ini, mengikuti urutan rumusan masalah pada Bab I agar keterkaitan antara permasalahan dan temuan dapat ditelusuri secara langsung. Subbab ini diakhiri dengan satu paragraf yang merangkum keterkaitan antar ketiga jawaban tersebut sebagai satu rangkaian yang saling menopang.

Pertama, dataset deteksi objek untuk sembilan kelas khas kawasan Malioboro berhasil dibangun melalui anotasi semi-otomatis berbantuan model *pretrained* dengan koreksi manual, menjawab rumusan masalah pertama yang menyatakan dataset semacam ini belum tersedia pada sumber data publik seperti COCO. Dataset akhir memuat 1.581 citra dengan sembilan kelas, yaitu andong, bajaj, becak, sepeda, bus, mobil, motor, orang, dan truk, dengan distribusi yang sangat tidak seimbang antara kelas orang sebanyak 10.694 *instance* dan kelas truk sebanyak 45 *instance*, atau rasio sekitar 237 berbanding 1. Ketidakseimbangan ini bukan kekurangan proses pengumpulan data, melainkan cerminan komposisi objek yang sebenarnya di lapangan, sehingga dataset yang terbentuk dapat dikatakan merepresentasikan kondisi nyata kawasan Malioboro secara jujur.

Kedua, perbandingan delapan varian model YOLOv8 dan YOLO11 berhasil menentukan model dengan performa terbaik, sekaligus mengungkap pengaruh ketidakseimbangan kelas terhadap performa deteksi tiap kelas, menjawab rumusan masalah kedua secara menyeluruh. Pada himpunan uji, yolov8l mencatat mAP50 dan rerata F1 tertinggi (0,6175 dan 0,6222), sedangkan yolo11l mencatat mAP50-95 tertinggi (0,4012) dengan ukuran model 42% lebih ringkas. Model yolo11l dipilih sebagai model terbaik berdasarkan keunggulan lokalisasi dan efisiensi ukuran tersebut, dan pilihan ini diperkuat oleh penyetelan resolusi citra masukan dari 640 menjadi 960 piksel, yang menaikkan rerata F1 model terpilih dari 0,5962 menjadi 0,6739, melampaui capaian

seluruh varian pada resolusi dasar termasuk yolov8l. Analisis per kelas menunjukkan ketidakseimbangan kelas berpengaruh pada performa, tetapi pengaruhnya dimoderasi oleh ukuran dan kekhasan visual objek, sebagaimana terlihat dari kelas truk yang berjumlah paling sedikit namun memperoleh F1 lebih tinggi daripada kelas sepeda yang berjumlah lebih banyak.

Ketiga, model terpilih berhasil diimplementasikan ke dalam sistem pemantauan kepadatan berbasis web yang memproses *stream* video kamera CCTV dan mengukur performa inferensinya, menjawab rumusan masalah ketiga. Pengujian fungsional menunjukkan antarmuka deteksi langsung, grafik kepadatan, dan riwayat peringatan berjalan sesuai rancangan, dengan notifikasi yang terpicu tepat sesuai ambang kepadatan yang ditetapkan. Pengukuran kecepatan inferensi pada perangkat penerapan, yaitu laptop dengan GPU GTX 1650, menunjukkan model terpilih mencapai 9,73 *frame* per detik pada resolusi 640, lebih cepat dibanding yolov8l yang hanya mencapai 8,80 *frame* per detik pada perangkat yang sama, berkebalikan dengan urutan kecepatan keduanya pada GPU Tesla V100 di lingkungan pelatihan. Temuan ini menegaskan bahwa pertimbangan ukuran model yang mendasari pemilihan yolo11l pada rumusan masalah kedua terbukti relevan secara langsung bagi kecepatan pada perangkat target, bukan sekadar pertimbangan keamanan memori semata.

Secara keseluruhan, ketiga jawaban di atas membentuk satu rangkaian yang saling menopang. Dataset yang terbentuk pada jawaban pertama menjadi fondasi bagi perbandingan dan pemilihan model pada jawaban kedua, dan model yang terpilih serta dioptimasi pada jawaban kedua terbukti layak diterapkan pada sistem yang menjawab rumusan masalah ketiga, baik dari sisi akurasi maupun kecepatan inferensi pada perangkat target. Dengan terjawabnya ketiga rumusan masalah tersebut, tujuan penelitian yang ditetapkan pada Bab I tercapai secara menyeluruh, mulai dari ketersediaan dataset hingga sistem pemantauan yang berfungsi.

5.2 Saran

Penelitian ini memiliki sejumlah batasan yang ditetapkan sejak awal agar lingkungannya tetap terarah, sebagaimana diuraikan pada Bab I, dan beberapa keterbatasan tambahan yang teridentifikasi sepanjang proses penelitian. Batasan dan keterbatasan tersebut bukan kekurangan yang perlu disesalkan, melainkan penanda arah yang jelas bagi penelitian berikutnya untuk melanjutkan dari titik yang tepat. Subbab ini menguraikan enam arah pengembangan lanjutan yang diturunkan langsung dari batasan dan keterbatasan tersebut, disusun agar penelitian berikutnya tidak mengulang capaian yang sudah diperoleh di sini.

Perluasan kalibrasi tingkat pelayanan ke seluruh titik kamera. Penetapan ambang notifikasi berbasis tingkat pelayanan pejalan kaki pada penelitian ini hanya diterapkan

pada dua titik kamera yang dikalibrasi melalui homografi sebagai contoh penerapan, sedangkan titik kamera lainnya menggunakan ambang yang ditentukan secara manual. Penelitian lanjutan dapat mengembangkan metode kalibrasi homografi yang lebih efisien atau bahkan otomatis untuk diterapkan pada seluruh 22 titik kamera, sehingga seluruh kawasan mendapat penetapan ambang yang berbasis standar kuantitatif, bukan hanya estimasi manual.

Penanganan kepadatan kendaraan dan kemacetan. Penelitian ini berfokus pada kepadatan pejalan kaki melalui konsep tingkat pelayanan, sedangkan penetapan ambang untuk kemacetan kendaraan berada di luar lingkup penelitian. Mengingat Malioboro juga mengalami kemacetan akibat lonjakan kendaraan sebagaimana diuraikan pada Bab I, dan sistem yang dibangun sudah mendeteksi kelas kendaraan seperti mobil, motor, bus, dan truk, penelitian lanjutan dapat mengembangkan standar kepadatan atau tingkat pelayanan jalan yang setara untuk kendaraan, sehingga sistem dapat memberi notifikasi kemacetan selain notifikasi keramaian pejalan kaki.

Penambahan kemampuan pelacakan antar-*frame*. Sistem yang dibangun melakukan penghitungan objek per *frame* tanpa pelacakan (*tracking*) antar-*frame*, sehingga tidak dapat membedakan apakah objek yang terdeteksi pada *frame* berurutan adalah objek yang sama atau berbeda. Penambahan algoritma pelacakan seperti DeepSORT pada penelitian lanjutan akan memungkinkan estimasi arus pergerakan, kecepatan rata-rata pejalan kaki, dan deteksi pola pergerakan yang tidak dapat diperoleh dari penghitungan per *frame* saja.

Peningkatan kecepatan inferensi pada perangkat dengan sumber daya terbatas. Pengukuran pada Bab IV menunjukkan kecepatan inferensi model terpilih pada GPU GTX 1650 belum mencapai laju *real-time* yang ideal, terutama setelah penyetelan resolusi ke 960 piksel. Penelitian lanjutan dapat mengeksplorasi teknik kompresi model seperti *pruning* atau kuantisasi, maupun konversi ke *runtime* inferensi yang lebih efisien seperti TensorRT, untuk mempercepat inferensi pada perangkat kelas konsumen tanpa mengorbankan akurasi secara signifikan.

Penanganan ketidakseimbangan kelas pada tahap pelatihan. Penelitian ini menunjukkan ketidakseimbangan kelas memengaruhi performa deteksi pada kelas minoritas seperti sepeda dan bajaj, dan mengatasinya melalui penyetelan resolusi yang menyasar objek berukuran kecil. Penelitian lanjutan dapat menambahkan strategi yang menyasar langsung ketidakseimbangan kelas, misalnya pembobotan fungsi kerugian per kelas atau augmentasi data yang lebih agresif khusus pada kelas minoritas, untuk menelaah apakah performa kelas seperti bajaj dan bus dapat ditingkatkan lebih lanjut tanpa mengorbankan performa kelas mayoritas.

Pengujian keterlibatan pengguna dan kebergunaan sistem. Penelitian ini berfokus

pada perbandingan model deteksi dan implementasi fungsional sistem, sedangkan perancangan berbasis *User-Centered Design* dan pengujian *usability* tidak termasuk dalam lingkup penelitian. Penelitian lanjutan dapat melibatkan calon pengguna sesungguhnya, seperti petugas pengelola kawasan Malioboro, untuk menilai kemudahan penggunaan antarmuka dan menyesuaikan rancangan sistem berdasarkan masukan tersebut, sehingga sistem tidak hanya terbukti berfungsi secara teknis tetapi juga nyaman digunakan dalam operasional sehari-hari.

DAFTAR PUSTAKA

- [1] Kementerian Pekerjaan Umum dan Perumahan Rakyat, “Pedoman perencanaan, penyediaan, dan pemanfaatan prasarana dan sarana jaringan pejalan kaki di kawasan perkotaan,” Peraturan Menteri PUPR No. 03/PRT/M/2014, 2014.
- [2] J. R. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González, “A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS,” *Machine Learning and Knowledge Extraction*, vol. 5, no. 4, pp. 1680–1716, 2023.
- [3] R. Khanam and M. Hussain, “YOLO11: An overview of the key architectural enhancements,” arXiv preprint arXiv:2410.17725, 2024.
- [4] H. N. Rahmah, M. J. Rahayu, and E. F. Rini, “Tingkat pelayanan jalur pedestrian di kawasan wisata Malioboro, Kota Yogyakarta,” *Desa-Kota: Jurnal Perencanaan Wilayah, Kota, dan Permukiman*, vol. 7, no. 2, pp. 102–111, 2025.
- [5] Pemerintah Kota Yogyakarta, “Jumlah pengunjung wisata menurut jenis pengunjung di Kota Yogyakarta tahun 2024,” Portal Satu Data Indonesia, 2024, [Daring]. Tersedia: <http://data.go.id/dataset/dataset/kyda2024-99>.
- [6] Badan Pusat Statistik Provinsi D.I. Yogyakarta, “Jumlah kunjungan wisatawan nusantara menurut tujuan (perjalanan) tahun 2025,” BPS Provinsi D.I. Yogyakarta, 2025, [Daring]. Tersedia: <https://yogyakarta.bps.go.id/id/statistics-table/2/NTE0IzI=/jumlah-kunjungan-wisatawan-nusantara-menurut-tujuan---perjalanan-.html>.
- [7] A. M. S. Lagalo and D. Talib, “Dinamika aktivitas pariwisata di kawasan Malioboro Yogyakarta saat mudik Lebaran: Studi wisata urban,” *Tulisan Ilmiah Pariwisata (TULIP)*, vol. 7, no. 2, pp. 144–148, 2024.
- [8] A. M. Saputra and Y. D. Artini, “Pengelolaan lalu lintas di kawasan Malioboro,” *Paradigma: Jurnal Ilmu Administrasi*, vol. 10, no. 2, pp. 136–153, 2021.
- [9] A. Khan, J. A. Shah, K. Kadir, W. Albattah, and F. Khan, “Crowd monitoring and localization using deep convolutional neural network: A review,” *Applied Sciences*, vol. 10, no. 14, p. 4781, 2020.
- [10] F. Caprotti and D. Liu, “Platform urbanism and the Chinese smart city: The co-production and territorialisation of Hangzhou City Brain,” *GeoJournal*, vol. 87, pp. 1559–1573, 2022.
- [11] S. L. B. Ginting, H. Maulana, R. A. Priatna, D. D. Fauzzan, and D. Setiawan, “Crowd detection using YOLOv3-Tiny method and Viola-Jones algorithm at mall,” *International Journal of Informatics, Information System and Computer Engineering*, vol. 2, no. 1, pp. 13–22, 2021.
- [12] A. Sharma, C. Mhatre, S. Gawali, R. Bokkasam, B. Sharma, V. Pattanaik, P. Rathore, R. Krishnapuram, V. G. Kovvali, A. Chakraborty, and Y. Simmhan, “BMD-45: A large-scale CCTV vehicle detection dataset for urban traffic in developing cities,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR) Findings*, 2026.

- [13] J. J. Fruin, "Designing for pedestrians: A level-of-service concept," *Highway Research Record*, no. 355, pp. 1–15, 1971.
- [14] M. Ş. Gündüz and G. Işık, "A new YOLO-based method for real-time crowd detection from video and performance analysis of YOLO models," *Journal of Real-Time Image Processing*, vol. 20, no. 1, p. 5, 2023.
- [15] W. T. D. T. Hansika, M. A. P. Madhawa, N. T. M. Sajith, M. N. M. Aashiq, and M. N. A. Hinas, "Privacy-preserving real-time crowd counting on IoT enabled edge devices using YOLOv8n and Norfair," in *Proc. IEEE Int. Research Conf. Smart Computing and Systems Engineering (SCSE)*. Kelaniya, Sri Lanka: IEEE, 2026.
- [16] S. Apeksha, M. Sohan, N. H. Gowda, R. Ramya, and M. Jadhav, "AI-transit alerts: A smart platform for tracking the number of people using public transportation in real time," in *Proc. 2nd Int. Conf. Computing for Sustainability and Intelligent Future (COMP-SIF)*. Bengaluru, India: IEEE, 2026.
- [17] V. W. H. Wong and K. H. Law, "Fusion of CCTV video and spatial information for automated crowd congestion monitoring in public urban spaces," *Algorithms*, vol. 16, no. 3, p. 154, 2023.
- [18] Y. Zhang, Z. Guo, J. Wu, Y. Tian, H. Tang, and X. Guo, "Real-time vehicle detection based on improved YOLO v5," *Sustainability*, vol. 14, no. 19, p. 12274, 2022.
- [19] K. Oksuz, B. C. Cam, S. Kalkan, and E. Akbas, "Imbalance problems in object detection: A review," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 10, pp. 3388–3415, 2021.
- [20] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, 2017.
- [21] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [22] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2117–2125.
- [23] Y. Zhao, W. Lv, S. Xu, J. Wei, Q. Dang, Y. Liu, and J. Chen, "DETRs beat YOLOs on real-time object detection," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 16 965–16 974.
- [24] R. Padilla, S. L. Netto, and E. A. B. da Silva, "A survey on performance metrics for object-detection algorithms," in *Proc. International Conf. Systems, Signals and Image Processing (IWSSIP)*, 2020, pp. 237–242.
- [25] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Proc. European Conf. Computer Vision (ECCV)*, 2014, pp. 740–755.

- [26] S. Tenckhoff, M. Koddenbrock, and E. Rodner, “Feedback-driven object detection and iterative model improvement for accurate annotations,” arXiv preprint arXiv:2411.19835, 2025.
- [27] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, 2000.
- [28] I. Fette and A. Melnikov, “The WebSocket protocol,” IETF RFC 6455, 2011.
- [29] G. K. Wallace, “The JPEG still picture compression standard,” *IEEE Transactions on Consumer Electronics*, vol. 38, no. 1, pp. xviii–xxxiv, 1992.
- [30] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, “YOLOv9: Learning what you want to learn using programmable gradient information,” arXiv preprint arXiv:2402.13616, 2024.
- [31] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, “YOLOv10: Real-time end-to-end object detection,” arXiv preprint arXiv:2405.14458, 2024.
- [32] Y. Tian, Q. Ye, and D. Doermann, “YOLOv12: Attention-centric real-time object detectors,” arXiv preprint arXiv:2502.12524, 2025.
- [33] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, and M. E. Kalfaoglu, “Ultralytics YOLO26: Unified real-time end-to-end vision models,” arXiv preprint arXiv:2606.03748, 2026.
- [34] N. Carion, L. Gustafson, Y.-T. Hu *et al.*, “SAM 3: Segment anything with concepts,” arXiv preprint arXiv:2511.16719, 2025.
- [35] Z. Luo *et al.*, “MIO-TCD: A new benchmark dataset for vehicle classification and localization,” *IEEE Transactions on Image Processing*, vol. 27, no. 10, pp. 5129–5141, 2018.

LAMPIRAN

L.1 Isi Lampiran

Lampiran bersifat opsional bergantung hasil kesepakatan dengan pembimbing dapat berupa:

1. Bukti pelaksanaan Kuesioner seperti pertanyaan kuesioner, resume jawaban responden, dan dokumentasi kuesioner.
2. Spesifikasi Aplikasi atau Sistem yang dikembangkan meliputi spesifikasi teknis aplikasi, tautan unduh aplikasi, manual penggunaan aplikasi, hingga screenshot aplikasi.
3. Cuplikan kode yang sekiranya penting dan ditambahkan.
4. Tabel yang terlalu panjang yang masih diperlukan tetapi tidak memungkinkan untuk ditayangkan di bagian utama skripsi.
5. Gambar-gambar pendukung yang tidak terlalu penting untuk ditampilkan di bagian utama. Akan tetapi, mendukung argumentasi/pengamatan/analisis.
6. Penurunan rumus-rumus atau pembuktian suatu teorema yang terlalu panjang dan terlalu teknis sehingga Anda berasumsi bahwa pembaca biasa tidak akan menelaah lebih lanjut. Hal ini digunakan untuk memberikan kesempatan bagi pembaca tingkat lanjut untuk melihat proses penurunan rumus-rumus ini.

LAMPIRAN

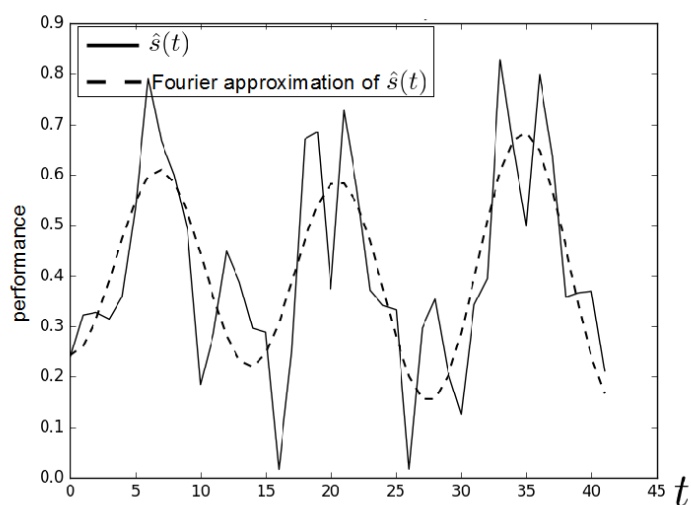
L.2 Panduan Latex

L.2.1 Syntax Dasar

L.2.1.1 Penggunaan Sitasi

Contoh penggunaan sitasi [?, ?] [?] [?] [?] [?, ?]

L.2.1.2 Penulisan Gambar



Gambar L.1. Contoh gambar.

Contoh gambar terlihat pada Gambar L.1. Gambar diambil dari [?].

L.2.1.3 Penulisan Tabel

Tabel L.1. Tabel ini

ID	Tinggi Badan (cm)	Berat Badan (kg)
A23	173	62
A25	185	78
A10	162	70

Contoh penulisan tabel bisa dilihat pada Tabel L.1.

L.2.1.4 Penulisan formula

Contoh penulisan formula

$$L_{\psi_z} = \{t_i \mid v_z(t_i) \leq \psi_z\} \quad (1)$$

Contoh penulisan secara *inline*: $PV = nRT$. Untuk kasus-kasus tertentu, kita membutuhkan perintah "mathit" dalam penulisan formula untuk menghindari adanya jeda saat penulisan formula.

Contoh formula **tanpa** menggunakan "mathit": $PVA = RTD$

Contoh formula **dengan** menggunakan "mathit": $PVA = RTD$

L.2.1.5 Contoh list

Berikut contoh penggunaan list

1. First item
2. Second item
3. Third item

L.2.2 Blok Beda Halaman

L.2.2.1 Membuat algoritma terpisah

Untuk membuat algoritma terpisah seperti pada contoh berikut, kita dapat memanfaatkan perintah *algstore* dan *algrestore* yang terdapat pada paket *algcompatible*. Pada dasarnya, kita membuat dua blok algoritma dimana blok pertama kita simpan menggunakan *algstore* dan kemudian di-restore menggunakan *algrestore* pada algoritma kedua. Perintah tersebut dimaksudkan agar terdapat kesinamungan antara kedua blok yang sejatinya adalah satu blok.

Algorithm 3 Contoh algorima

```
1: procedure CREATESET( $v$ )  
2:   Create new set containing  $v$   
3: end procedure
```

Pada blok algoritma kedua, tidak perlu ditambahkan caption dan label, karena sudah menjadi satu bagian dalam blok pertama. Pembagian algoritma menjadi dua bagian ini berguna jika kita ingin menjelaskan bagian-bagian dari sebuah algoritma, maupun untuk memisah algoritma panjang dalam beberapa halaman.

```
4: procedure CONCATSET( $v$ )  
5:   Create new set containing  $v$   
6: end procedure
```

L.2.2.2 Membuat tabel terpisah

Untuk membuat tabel panjang yang melebihi satu halaman, kita dapat mengganti kombinasi *table* + *tabular* menjadi *longtable* dengan contoh sebagai berikut.

Tabel L.2. Contoh tabel panjang

header 1	header 2
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar

L.2.2.3 Menulis formula terpisah halaman

Terkadang kita butuh untuk menuliskan rangkaian formula dalam jumlah besar sehingga melewati batas satu halaman. Solusi yang digunakan bisa saja dengan memindahkan satu blok formula tersebut pada halaman yang baru atau memisah rangkaian formula menjadi dua bagian untuk masing-masing halaman. Cara yang pertama mungkin akan menghasilkan alur yang berbeda karena ruang kosong pada halaman pertama akan diisi oleh teks selanjutnya. Sehingga di sini kita dapat memanfaatkan *align* yang sudah diatur dengan mode *allowdisplaybreaks*. Penggunaan *align* ini memungkinkan satu rangkaian formula terpisah berbeda halaman.

Contoh sederhana dapat digambarkan sebagai berikut.

$$\begin{aligned}
 x &= y^2 \\
 x &= y^3 \\
 a + b &= c \\
 x &= y - 2 \\
 a + b &= d + e \\
 x^2 + 3 &= y \\
 a(x) &= 2x
 \end{aligned}
 \tag{2}$$

$$b_i = 5x$$

$$10x^2 = 9x$$

$$2x^2 + 3x + 2 = 0$$

$$5x - 2 = 0$$

$$d = \log x$$

$$y = \sin x$$

LAMPIRAN

L.3 Format Penulisan Referensi

Penulisan referensi mengikuti aturan standar yang sudah ditentukan. Untuk internasionalisasi DTETI, maka penulisan referensi akan mengikuti standar yang ditetapkan oleh IEEE (*International Electronics and Electrical Engineers*). Aturan penulisan ini bisa diunduh di <http://www.ieee.org/documents/ieeecitationref.pdf>. Gunakan Mendeley sebagai *reference manager* dan *export* data ke format Bibtex untuk digunakan di Latex.

Berikut ini adalah sampel penulisan dalam format IEEE:

L.3.1 Book

Basic Format:

- [1] J. K. Author, "Title of chapter in the book," in Title of His Published Book, xth ed. City of Publisher, Country: Abbrev. of Publisher, year, ch. x, sec. x, pp. xxx–xxx.

Examples:

- [1] B. Klaus and P. Horn, Robot Vision. Cambridge, MA: MIT Press, 1986.
- [2] L. Stein, "Random patterns," in Computers and You, J. S. Brake, Ed. New York: Wiley, 1994, pp. 55-70.
- [3] R. L. Myer, "Parametric oscillators and nonlinear materials," in Nonlinear Optics, vol. 4, P. G. Harper and B. S. Wherret, Eds. San Francisco, CA: Academic, 1977, pp. 47-160.
- [4] M. Abramowitz and I. A. Stegun, Eds., Handbook of Mathematical Functions (Applied Mathematics Series 55). Washington, DC: NBS, 1964, pp. 32-33.
- [5] E. F. Moore, "Gedanken-experiments on sequential machines," in Automata Studies (Ann. of Mathematical Studies, no. 1), C. E. Shannon and J. McCarthy, Eds. Princeton, NJ: Princeton Univ. Press, 1965, pp. 129-153.
- [6] Westinghouse Electric Corporation (Staff of Technology and Science, Aerospace Div.), Integrated Electronic Systems. Englewood Cliffs, NJ: Prentice-Hall, 1970.
- [7] M. Gorkii, "Optimal design," Dokl. Akad. Nauk SSSR, vol. 12, pp. 111-122, 1961 (Transl.: in L. Pontryagin, Ed., The Mathematical Theory of Optimal Processes. New York: Interscience, 1962, ch. 2, sec. 3, pp. 127-135).

- [8] G. O. Young, "Synthetic structure of industrial plastics," in *Plastics*, vol. 3, *Polymers of Hexadromicon*, J. Peters, Ed., 2nd ed. New York: McGraw-Hill, 1964, pp. 15-64.

L.3.2 Handbook

Basic Format:

- [1] Name of Manual/Handbook, x ed., Abbrev. Name of Co., City of Co., Abbrev. State, year, pp. xx-xx.

Examples:

- [1] Transmission Systems for Communications, 3rd ed., Western Electric Co., Winston Salem, NC, 1985, pp. 44-60.
- [2] Motorola Semiconductor Data Manual, Motorola Semiconductor Products Inc., Phoenix, AZ, 1989.
- [3] RCA Receiving Tube Manual, Radio Corp. of America, Electronic Components and Devices, Harrison, NJ, Tech. Ser. RC-23, 1992.

Conference/Prosiding

Basic Format:

- [1] J. K. Author, "Title of paper," in Unabbreviated Name of Conf., City of Conf., Abbrev. State (if given), year, pp.xxx-xxx.

Examples:

- [1] J. K. Author [two authors: J. K. Author and A. N. Writer] [three or more authors: J. K. Author et al.], "Title of Article," in [Title of Conf. Record as], [copyright year] © [IEEE or applicable copyright holder of the Conference Record]. doi: [DOI number]

Sumber Online/Internet

Basic Format:

- [1] J. K. Author. (year, month day). Title (edition) [Type of medium]. Available: [http://www.\(URL\)](http://www.(URL))

Examples:

- [1] J. Jones. (1991, May 10). Networks (2nd ed.) [Online]. Available: <http://www.atm.com>

Skripsi, Tesis dan Disertasi

Basic Format:

- [1] J. K. Author, "Title of thesis," M.S. thesis, Abbrev. Dept., Abbrev. Univ., City of Univ., Abbrev. State, year.

[2] J. K. Author, "Title of dissertation," Ph.D. dissertation, Abbrev. Dept., Abbrev. Univ., City of Univ., Abbrev. State, year.

Examples:

[1] J. O. Williams, "Narrow-band analyzer," Ph.D. dissertation, Dept. Elect. Eng., Harvard Univ., Cambridge, MA, 1993. [2] N. Kawasaki, "Parametric study of thermal and chemical nonequilibrium nozzle flow," M.S. thesis, Dept. Electron. Eng., Osaka Univ., Osaka, Japan, 1993

LAMPIRAN

L.4 Contoh Source Code

L.4.1 Sample algorithm

Algorithm 4 Kruskal's Algorithm

```
1: procedure MAKESET( $v$ )
2:   Create new set containing  $v$ 
3: end procedure
4:
5: function FINDSET( $v$ )
6:   return a set containing  $v$ 
7: end function
8:
9: procedure UNION( $u, v$ )
10:  Unites the set that contain  $u$  and  $v$  into a new set
11: end procedure
12:
13: function KRUSKAL( $V, E, w$ )
14:   $A \leftarrow \{\}$ 
15:  for each vertex  $v$  in  $V$  do
16:    MakeSet( $v$ )
17:  end for
18:  Arrange  $E$  in increasing costs, ordered by  $w$ 
19:  for each  $(u, v)$  taken from the sorted list do
20:    if FindSet( $u$ )  $\neq$  FindSet( $v$ ) then
21:       $A \leftarrow A \cup \{(u, v)\}$ 
22:      Union( $u, v$ )
23:    end if
24:  end for
25:  return  $A$ 
26: end function
```

L.4.2 Sample Python code

```
1 import numpy as np
2
3 def incmatrix (genl1 , genl2):
4     m = len (genl1)
5     n = len (genl2)
6     M = None #to become the incidence matrix
7     VT = np.zeros ((n*m,1) , int) #dummy variable
8
9     #compute the bitwise xor matrix
10    M1 = bitxormatrix (genl1)
11    M2 = np.triu (bitxormatrix (genl2) ,1)
12
13    for i in range (m-1):
14        for j in range (i+1, m):
15            [r,c] = np.where (M2 == M1[i,j])
16            for k in range (len (r)):
17                VT[(i)*n + r[k]] = 1;
18                VT[(i)*n + c[k]] = 1;
19                VT[(j)*n + r[k]] = 1;
20                VT[(j)*n + c[k]] = 1;
21
22    if M is None:
23        M = np.copy (VT)
24    else:
25        M = np.concatenate ((M, VT) , 1)
26
27    VT = np.zeros ((n*m,1) , int)
28
29    return M
```

L.4.3 Sample Matlab code

```
1 function X = BitXorMatrix(A,B)
2 %function to compute the sum without charge of two vectors
3
4 %convert elements into unsigned integers
5 A = uint8(A);
6 B = uint8(B);
7
8 m1 = length(A);
9 m2 = length(B);
10 X = uint8(zeros(m1, m2));
11 for n1=1:m1
12     for n2=1:m2
13         X(n1, n2) = bitxor(A(n1), B(n2));
14     end
15 end
```